

AI-Driven Security in the Software Supply Chain: A Comprehensive Survey on Vulnerability Detection, Adversarial Threats, and Resilient Architectures

1. Introduction to the Evolving AI Software Supply Chain Landscape

1.1 Defining the Modern AI Software Supply Chain Ecosystem

1.1 Defining the Modern AI Software Supply Chain Ecosystem

While the preceding analysis highlighted the escalation from traditional code injection to sophisticated AI-specific compromises, a precise understanding of the target ecosystem is prerequisite to developing effective defenses. The contemporary software supply chain has undergone a paradigmatic shift, evolving from a linear sequence of code compilation and dependency resolution into a complex, multi-dimensional ecosystem driven by artificial intelligence. Traditionally, this chain was defined primarily by source code repositories, package managers, and binary distributions. However, the integration of machine learning (ML) and large language models (LLMs) has expanded this definition to encompass a heterogeneous array of artifacts that extend far beyond deterministic code. The modern AI software supply chain is now characterized by the intricate interplay between data pipelines, pre-trained model artifacts, specialized toolchains, underlying infrastructure, and the downstream applications that consume these intelligent components. Understanding this expanded ecosystem is crucial, as the vulnerabilities and attack surfaces have multiplied, moving from simple dependency confusion in code packages to sophisticated data poisoning and model extraction attacks.

At the foundational layer of this ecosystem lies the data pipeline, which serves as the lifeblood for training and fine-tuning AI systems. Unlike traditional software where inputs are strictly defined by user interaction or API calls, AI systems rely on massive, often curated datasets that dictate model behavior. The management of these data assets involves complex workflows including collection, cleaning, augmentation, and transformation. Recent studies highlight that data preprocessing is frequently the most time-consuming aspect of developing machine learning solutions, necessitating flexible and efficient pipeline architectures [1]. Furthermore, the integrity of the supply chain is contingent upon the quality and provenance of this data; issues such as data leakage can lead to overoptimistic performance estimates and severe failures when models are deployed in real-world scenarios [2]. The supply chain must therefore account for “feature stores” and emerging “embedding ecosystems,” where self-supervised pretrained embeddings are managed as critical assets, introducing new challenges in measuring quality and monitoring downstream usage [3].

Building upon these data foundations are the pre-trained model (PTM) artifacts, which have become the primary units of reuse in modern AI development. Developers increasingly rely on PTMs to avoid the prohibitive costs of training from scratch, integrating these models into their applications via APIs or direct weight loading. This reliance has created a sprawling supply chain of models hosted on various platforms, yet the dynamics of this PTM supply chain remain largely opaque. Comprehensive datasets like PeaTMOSS have revealed that hundreds of thousands of pre-trained models are in circulation, often with inconsistent documentation regarding their training data, parameters, and licenses [4]. The supply chain must now track not just the model weights but also the metadata associated with them, including the specific datasets used for pretraining, as the composition of this data significantly influences model capabilities and potential biases [5]. The reuse of these models introduces risks where downstream applications may inherit vulnerabilities

or undesirable behaviors embedded within the upstream artifacts.

The toolchain and infrastructure components form the operational backbone of the AI supply chain, facilitating the lifecycle from development to deployment. This includes integrated development environments (IDEs) enhanced with AI code generators, continuous integration/continuous deployment (CI/CD) pipelines adapted for ML (MLOps), and the specialized hardware required for training and inference. The complexity of these toolchains is evident in the heterogeneous stacks involving GPUs, drivers, runtime environments, and deep learning frameworks, where asynchronous evolution and version constraints often lead to dependency bugs [6]. To manage this complexity, organizations are adopting MLOps practices, though the maturity of these practices varies significantly; many entities are still in the nascent stages of figuring out how to best utilize data and build initial models, while only a minority have achieved the level of maturity required for frequent retraining and automated deployment [7]. Furthermore, the rise of LLMs has introduced new infrastructure demands, characterized by massive cluster resources, intricate parallelization strategies, and the need for fault-tolerant pretraining systems to handle frequent hardware failures [8].

Finally, the ecosystem extends to the downstream application layer, where AI components are integrated into end-user products. This layer is increasingly defined by the use of LLMs as generic data operators or “copilots” that assist in code generation, data cleansing, and decision-making. However, the integration of these models introduces new dependency structures; for instance, code generation tools must now navigate repository-level dependencies and user-defined attributes to avoid generating invalid code [9]. The supply chain must support the modularization of code generation and the ability to self-revise solutions to ensure correctness and reusability [10]. Moreover, the distinction between the model and the application is blurring, as prompts and user-defined instructions become executable logic within the pipeline [11].

In summary, defining the modern AI software supply chain requires a holistic view that transcends traditional code dependencies. It is a dynamic network comprising data lineage, pre-trained model provenance, complex toolchain interactions, and adaptive downstream applications. The interdependencies among these components create a fragile yet powerful ecosystem where a compromise in any single layer—be it a poisoned dataset, a malicious model update, or a vulnerable infrastructure component—can cascade through the entire supply chain, manifesting in the severe incidents discussed previously. Recognizing these diverse elements is the first step toward establishing robust security frameworks, such as AI-specific Software Bills of Materials (AIBOMs), capable of addressing the unique challenges posed by this evolving landscape and mitigating the dual threat vectors of traditional and AI-native attacks.

1.2 The Escalating Landscape of Supply Chain Compromises

1.2 The Escalating Landscape of Supply Chain Compromises

Building upon the definition of the modern AI software supply chain ecosystem, the structural shift from a linear delivery mechanism to a complex, interconnected network has fundamentally altered the global security posture. In this expanded ecosystem, where trust is implicitly placed in thousands of third-party components ranging from data pipelines to pre-trained models, the landscape of compromises has escalated dramatically. The urgency of this crisis is best understood by examining the trajectory from traditional software supply chain breaches—characterized by code injection and build system compromises—to the emerging, more insidious threats targeting the probabilistic core of AI pipelines. While traditional attacks focused on injecting malicious logic

into deterministic code, the new frontier involves manipulating the very data and models that drive autonomous decision-making, creating a dual threat vector that is both broader in scope and deeper in impact.

The severity of traditional supply chain vulnerabilities was starkly illustrated by the SolarWinds incident, a compromise that sent shockwaves through government and enterprise sectors alike. This attack demonstrated how adversaries could infiltrate a trusted software update mechanism to distribute malware to thousands of downstream organizations, effectively bypassing perimeter defenses. Detailed analyses of the SolarWinds attack on the Orion platform reveal how persistent threat agents exploited loopholes in security systems to gain unauthorized access, utilizing command injections and privilege escalations to pivot across networks [12]. The ramifications were so profound that they spurred immediate governmental action, leading to executive orders aimed at enhancing software supply chain security and mandating greater transparency from vendors [13]. Similarly, the Log4j vulnerability, known as Log4Shell, highlighted the fragility of ubiquitous open-source dependencies within the toolchain layer. Measurements of the incident revealed a frantic “race to the vulnerable,” where malicious scanners continued to target the vulnerability long after benign researchers had moved on, underscoring the persistent nature of these threats once a component is compromised [14]. These incidents collectively established that the software supply chain is a critical attack surface, where a single weak link can compromise an entire ecosystem.

However, as the industry grapples with these traditional vectors, the landscape is rapidly shifting toward AI-specific compromises that exploit the unique artifacts defined in the previous section: data pipelines and pre-trained models. The integration of AI into the software development lifecycle has expanded the attack surface beyond deterministic code to include probabilistic components susceptible to manipulation through subtle perturbations in training data or inference inputs. The rising frequency of malicious package injections in repositories like npm and PyPI serves as a bridge between these two worlds, targeting the infrastructure that supports AI development. Research indicates a dramatic increase in supply chain attacks, with a 650% year-on-year growth observed in 2021 alone, driven by the exploitation of open-source software ecosystems [15]. Attackers are increasingly targeting these repositories to inject malicious code that can steal credentials, exfiltrate data, or compromise downstream applications. Studies analyzing malicious packages in npm, PyPI, and RubyGems have cataloged hundreds of real-world attacks, revealing sophisticated techniques for executing code at different stages of the dependency resolution process [16].

The transition to AI-driven supply chains introduces the specter of data poisoning and model integrity attacks, which pose risks distinct from traditional code injection by targeting the foundational data layer. In the context of AI, an attacker does not necessarily need to inject executable malware; instead, they can manipulate the training data to induce specific, malicious behaviors in the resulting model. Recent empirical studies have demonstrated the feasibility of data poisoning attacks against real-world AI systems, such as game-playing agents, where trojan viruses were successfully installed to manipulate behavior [17]. Furthermore, the reliance on AI-powered coding assistants has created a new vector for vulnerability propagation within the downstream application layer. Developers increasingly trust suggestions from tools like GitHub Copilot, yet research shows that these tools are vulnerable to poisoning attacks where adversaries inject insecure code snippets into training corpora. When developers accept these poisoned suggestions, they inadvertently introduce vulnerabilities into their products, creating a feedback loop of insecurity [18]. Novel attacks such as “TrojanPuzzle” have shown how adversaries can covertly poison code-suggestion models by planting malicious data in out-of-context regions like docstrings, bypassing static analysis tools designed to detect explicit malicious payloads [19].

The complexity of these threats is compounded by the difficulty in detection and attribution, particularly as the distinction between data and instructions blurs in Large Language Model (LLM) applications. Traditional signature-based defenses are often ineffective against AI-specific attacks, which may not exhibit obvious malicious signatures until triggered by specific inputs. The emergence of indirect prompt injection attacks further exacerbates this challenge, allowing adversaries to remotely exploit LLM-integrated applications by injecting prompts into retrieved data [20]. These attacks can lead to data theft, worm propagation, and the contamination of information ecosystems, presenting challenges that traditional security frameworks are ill-equipped to handle. Moreover, the sheer volume of open-source components and the speed of AI model deployment make manual verification impossible, necessitating automated, intelligent detection mechanisms. The industry is only beginning to recognize the scale of this problem, with summits and reports highlighting the lack of standardized approaches for securing AI assets and the urgent need for new governance frameworks [21].

In conclusion, the landscape of supply chain compromises has escalated from isolated incidents of code injection to a systemic crisis involving the integrity of data, models, and automated decision-making processes. The lessons learned from SolarWinds and Log4j emphasize the need for rigorous dependency management and transparency, but the unique properties of AI systems demand a paradigm shift in security strategy. As attackers refine their techniques to exploit the probabilistic nature of machine learning and the trust placed in AI assistants, the software supply chain becomes an increasingly volatile environment. This evolution sets the stage for the next critical dimension of the AI security discourse: the dual nature of AI itself, where the same technologies enabling these sophisticated attacks also offer the potential for robust defensive shields, a dichotomy explored in the following section.

1.3 The Dual Nature of AI: Offensive Weaponization and Defensive Shield

1.3 The Dual Nature of AI: Offensive Weaponization and Defensive Shield

Building upon the evolving landscape of supply chain compromises described previously, the integration of Artificial Intelligence (AI) has precipitated a paradigm shift in cybersecurity characterized by a profound duality: AI serves simultaneously as a potent vector for sophisticated offensive operations and as an indispensable shield for defensive resilience. This dichotomy moves beyond theoretical concern to define the current threat environment, where the same generative capabilities empowering autonomous development agents also enable the creation of self-replicating malware and highly personalized social engineering campaigns. Understanding this dual nature is critical for securing the modern supply chain, as the boundary between attacker and defender increasingly blurs through the deployment of autonomous agents and large language models (LLMs). This dynamic sets the stage for the specific structural vulnerabilities inherent to AI architectures, which will be examined in the following section.

On the offensive front, the weaponization of Generative AI (GenAI) has lowered the barrier to entry for cybercriminals while exponentially increasing the scale and sophistication of attacks targeting the supply chain. The emergence of specialized malicious models, such as FraudGPT and WormGPT, alongside the misuse of legitimate tools like ChatGPT, has revolutionized the mechanics of social engineering. These models empower adversaries to craft convincing, context-aware phishing lures and manipulate public opinion through deepfakes, exploiting human cognitive biases with unprecedented precision [22]. The implications extend beyond mere text generation; GenAI facilitates the entire cyber kill chain, from reconnaissance to exploitation, by automating the creation

of attack payloads, generating disinformation, and even assisting in reverse cryptography [23]. Perhaps most alarming is the evolution of these models into autonomous agents capable of executing complex hacking tasks without human intervention. Recent studies have demonstrated that frontier LLM agents can autonomously hack websites, performing blind database schema extraction and SQL injections, and can exploit known one-day vulnerabilities with high success rates when provided with Common Vulnerabilities and Exposures (CVE) descriptions [24][25]. This capability signifies a transition from static malware to dynamic, intelligent threats that can adapt to network environments in real-time, directly compromising the integrity of software dependencies.

The threat landscape further expands with the introduction of self-replicating worms designed specifically for GenAI ecosystems, representing a novel form of supply chain contagion. The “Morris II” worm exemplifies this danger, utilizing adversarial self-replicating prompts to infect GenAI-powered applications. By embedding malicious instructions within inputs, attackers can compel AI agents to replicate the payload and propagate it to other agents within the ecosystem, effectively turning the supply chain’s connectivity against itself [26]. Furthermore, the risk of escalation is not limited to cyber domains; research indicates that autonomous AI agents deployed in military or diplomatic decision-making simulations exhibit a tendency toward escalatory actions and arms-race dynamics, suggesting that unchecked autonomy in high-stakes environments could lead to catastrophic outcomes [27]. The proliferation of such capabilities underscores the urgent need to view AI not merely as a tool but as an active participant in conflict that requires rigorous containment strategies.

Conversely, this same technological prowess offers transformative potential for defensive mechanisms, enabling proactive threat hunting and real-time anomaly detection that traditional signature-based systems cannot achieve against the complex attacks described earlier. AI-driven defensive architectures are evolving from reactive filters to autonomous threat hunters capable of analyzing vast datasets to identify stealthy Advanced Persistent Threats (APTs). Systems like RANK utilize end-to-end AI architectures to merge alerts, construct attack graphs, and score incidents, reducing the data volume requiring human review by orders of magnitude while improving detection accuracy [28]. Similarly, frameworks such as ATHAFI automate the threat hunting process by generating and testing attack hypotheses, allowing organizations to keep pace with rapidly evolving attack vectors despite a shortage of skilled analysts [29]. The concept of autonomous cyber defense is gaining traction as a necessity to mitigate risks at machine speed, although it introduces its own set of ethical and operational challenges regarding unintended harm and algorithmic constraints [30].

Generative models play a pivotal role in strengthening these defenses, particularly through data augmentation and synthetic data generation, which are essential for training robust detectors against the novel attack patterns emerging in AI supply chains. In domains where labeled attack data is scarce or sensitive, such as insider threat analysis, Generative Adversarial Networks (GANs) like SPCAGAN can generate high-quality synthetic data that closely mirrors real-world distributions, thereby enhancing the training of detection models without compromising privacy [31]. This approach addresses the critical data scarcity issue, allowing defenders to train robust models against rare and novel attack patterns. Furthermore, Generative AI is being leveraged to secure physical layer communications and intelligent network services, acting as a “shield” that optimizes energy consumption and reduces retransmission counts during data poisoning attacks [32][33]. Advanced anomaly detection systems now employ ensemble methods and deep learning techniques to monitor heterogeneous data streams in cyber-physical systems, effectively distinguishing between device malfunctions and malicious intrusions in unsupervised settings [34].

The interplay between these offensive and defensive capabilities creates a dynamic arms race within the software supply chain, where the efficacy of one side directly drives the innovation of the other. As attackers utilize LLMs to automate vulnerability discovery and exploit generation, defenders must deploy equally sophisticated AI agents for automated patching and continuous monitoring. The effectiveness of this defensive posture relies on the ability to anticipate adversarial moves, a task where AI excels by simulating attack scenarios and identifying potential weaknesses before they are exploited in the wild [35]. However, the deployment of such autonomous systems necessitates a careful balance; while they offer the speed required to counter modern threats, they also introduce risks of false positives and unintended consequences that must be managed through robust governance and human-in-the-loop oversight [36]. Ultimately, while this section highlights the functional duality of AI as both spear and shield, the following section will delve deeper into the intrinsic structural vulnerabilities of AI systems themselves, exploring how the probabilistic and high-dimensional nature of these models creates unique fragility points that transcend the traditional offensive-defensive dichotomy.

1.4 Structural Vulnerabilities Across the AI Lifecycle

1.4 Structural Vulnerabilities Across the AI Lifecycle

While the previous section illustrated how AI functions as both a potent offensive weapon and a defensive shield, it is crucial to recognize that these dynamics are underpinned by a more fundamental issue: the intrinsic structural fragility of AI systems themselves. The transition from deterministic software engineering, where logic flows through explicitly defined control paths, to the probabilistic paradigms of Artificial Intelligence (AI) introduces a uniquely altered landscape of vulnerabilities. Unlike conventional codebases, AI systems—particularly those based on connectionist architectures—rely on high-dimensional statistical correlations learned from data. This paradigm shift creates specific fragility points across the entire AI development lifecycle, from data acquisition and model training to deployment and inference. The very properties that grant AI its adaptability and predictive power, such as non-linearity and high dimensionality, simultaneously render it susceptible to attack vectors that have no direct equivalent in traditional software, setting the stage for the governance challenges discussed in the subsequent section.

A primary source of this structural vulnerability lies in the geometric nature of high-dimensional data spaces. Modern AI technologies depend critically on learning non-linear functional dependencies from vast datasets; however, empirical evidence suggests that high predictive performance is often intrinsically coupled with low robustness. This paradox arises because the decision boundaries learned by deep neural networks in high-dimensional spaces can be surprisingly linear and fragile, making them vulnerable to minute, imperceptible perturbations known as adversarial attacks. Research indicates that these vulnerabilities are not merely implementation flaws but are deeply rooted in the geometry of typical high-dimensional datasets, suggesting that the susceptibility to adversarial examples is an inherent characteristic of current AI architectures rather than an anomaly [37]. Furthermore, the phenomenon of stochastic separability in high-dimensional data, while useful for correcting errors, also induces vulnerabilities where errors or clusters of errors can be easily separated from legitimate data, providing attackers with mathematical leverage to manipulate system outcomes [38].

These architectural fragilities are exacerbated by the modular nature of the modern AI supply chain. Contemporary machine learning systems are rarely built from scratch; instead, they are compositions of modular learning components (MLCs) sourced from various third-party contribu-

tors. This modularity accelerates development but introduces profound security risks due to a lack of standardization and regulation. Malicious actors can craft “logic-bomb” attacks within these MLCs, where modifying a negligible fraction of parameters with minuscule distortions can trigger predictable malfunctions in the host system. For instance, studies have demonstrated that altering less than 0.4% of an MLC’s parameters can force a healthcare diagnostic system to misdiagnose skin cancer with 100% success rates, highlighting how the non-convex and non-linear nature of modern models amplifies the impact of subtle structural compromises [39].

The scope of these vulnerabilities expands further when AI is integrated into cyber-physical systems (CPS) and Internet of Things (IoT) environments, extending the attack surface to include hardware and physical processes. The reliance on outsourced hardware design and embedded accelerators for CNN deployment opens avenues for hardware intrinsic attacks. Attackers can insert Trojans into the mathematical operations of individual CNN layers, propagating erroneous computations that lead to misclassification without requiring knowledge of the full model architecture. These hardware-level vulnerabilities are particularly insidious because they exploit the statistical distribution of feature maps, remaining stealthy until triggered by specific input conditions [40; 41]. The structural fragility is compounded in CPS, where the interplay between discrete digital dynamics and continuous physical processes creates complex failure modes. Attacks targeting sensor hardware can disrupt the equivalence between the attacked and unattacked system states, compromising safety properties that are difficult to verify due to the system’s inherent nondeterminism [42].

Moreover, the vulnerability of connectionist systems exhibits distinct propagation dynamics across networked environments. In complex networks of AI components, robustness against targeted attacks often depends on topological features such as betweenness centrality rather than simple node degree. Attacks targeting nodes with high betweenness centrality can cause a sharp, first-order phase transition in network functionality, leading to rapid system disintegration. This structural weakness is exacerbated in inter-dependent networks, where the failure of a small fraction of structurally central nodes in a smaller layer can cascade to collapse the entire system, making inter-dependent AI infrastructures even more fragile than traditional scale-free networks [43; 44; 45].

The probabilistic nature of AI outputs further challenges traditional verification methods, creating avenues for transferable attacks. Unlike deterministic software where a specific input yields a predictable output, AI systems exhibit variability that can be exploited across different models. Adversarial examples crafted for one model often successfully deceive other models, even those with different architectures or training data, indicating a shared structural vulnerability across the AI ecosystem. This transferability permeates various domains, including image, text, and graph data, suggesting that the underlying mathematical foundations of deep learning possess ubiquitous weak points [46]. The inability of local intrinsic dimensionality (LID) metrics to consistently characterize adversarial subspaces across different confidence levels and black-box transfer scenarios further underscores the complexity of defining and securing these structural boundaries [47; 48].

Finally, the lifecycle of AI systems involves a continuous co-evolution between attackers and defenders, particularly as automation increases on both sides. In fully automated cyber-defense systems utilizing machine learning, the dynamics can reach fixed points where defense strategies rely heavily on whitelisted feature subspaces, potentially creating new static vulnerabilities exploitable by adaptive adversaries. This evolutionary arms race necessitates a shift from empirical defenses to more rigorous, mathematically grounded approaches, as single protective measures are insufficient against the multifaceted structural weaknesses inherent in connectionist systems [49; 50]. Consequently, the unique structural fragilities detailed here—from geometric instabilities to hardware

Trojans and cascade failures—render traditional, siloed vulnerability management insufficient. Addressing these deep-seated issues requires the holistic, unified governance frameworks and adaptive disclosure protocols that will be examined in the following section.

1.5 The Imperative for Holistic Security and Governance Frameworks

1.5 The Imperative for Holistic Security and Governance Frameworks

The structural fragilities and unique attack vectors detailed in the preceding section necessitate a paradigm shift from fragmented, perimeter-based defenses to holistic, unified governance frameworks. The transition from deterministic software engineering to probabilistic AI paradigms has rendered traditional siloed vulnerability management insufficient, given the intricate interdependencies between conventional IT infrastructure and emerging AI assets. To address the geometric vulnerabilities, logic-bomb risks in modular components, and hardware-level exploits previously discussed, a robust defense strategy must integrate three critical pillars: comprehensive inventory mechanisms like Software Bills of Materials (SBOMs), dynamic access control via Zero Trust architectures, and adaptive incident response through coordinated disclosure protocols. This triad forms the foundation for resilience against supply chain compromises that threaten both deterministic software and the connectionist models analyzed earlier.

At the core of this holistic approach is the need for absolute transparency regarding software composition to counter the opacity of high-dimensional model behaviors. The Software Bill of Materials (SBOM) has emerged as a pivotal tool, offering an exhaustive inventory of components and dependencies crucial to software development. However, the effective implementation of SBOMs faces significant hurdles, including data tampering risks and the reluctance of vendors to fully disclose sensitive information, which significantly hinders their utility in critical infrastructure [51]. Furthermore, existing SBOM practices often lack visibility into the build process itself, focusing merely on the final artifact rather than the qualitative assurance of the toolchain used to create it [52]. Practical studies reveal that stakeholders encounter numerous challenges in creating and maintaining SBOMs, ranging from deficiencies in current tools to domain-specific complexities in areas like Java development and AI systems [53] [54]. To address these gaps—particularly the need to track data lineage and model parameters exposed to adversarial manipulation—the industry must evolve toward an AI Bill of Materials (AIBOM). Recent empirical analyses of GitHub discussions highlight that while SBOMs are recognized as essential for resilient development, widespread integration requires overcoming substantial technical and procedural barriers [55]. Consequently, a holistic framework must mandate the generation of standardized, machine-readable inventories that extend deep into the AI lifecycle, supported by automated generation tools capable of semantic similarity analysis to accurately identify embedded components [56].

Complementing the static visibility provided by SBOMs is the dynamic enforcement of security principles through Zero Trust Architectures (ZTA), which is essential for mitigating the cascading failures and transferable attacks inherent in networked AI environments. In an era where network virtualization and software-defined networking have expanded the attack surface, the assumption of implicit trust within network perimeters is obsolete. The transition to a Zero Trust model emphasizes continuous verification, least privilege access, and real-time monitoring of asset security states [57]. This is particularly critical for IoT and cyber-physical systems, where heterogeneous devices often lack robust native security, making them vulnerable to extensive attack surfaces

and hardware intrinsic attacks [58]. By integrating AI-driven anomaly detection into Zero Trust frameworks, organizations can achieve progressive authorization and fast authentication, adapting to dynamic environments while mitigating cascaded security risks [59]. Moreover, the convergence of Zero Trust with blockchain technology offers a novel approach to storing immutable records of user requests and verifying trustworthiness, thereby enhancing security in decentralized ecosystems [58]. For cloud-based health information systems, context-aware Zero Trust frameworks utilizing scoring mechanisms have demonstrated high efficacy in detecting misuse cases, proving that dynamic trust evaluation is essential for preventing human error and malicious exploitation [60].

Finally, the resilience of the supply chain depends on evolving coordinated disclosure mechanisms to address the unique nature of AI failures, which often stem from non-deterministic behaviors rather than discrete code flaws. Traditional Coordinated Vulnerability Disclosure (CVD) processes are ill-suited for algorithmic harms arising from data biases, emergent properties, or the structural vulnerabilities of high-dimensional spaces. There is an urgent need for a specialized Coordinated Flaw Disclosure (CFD) framework tailored to the intricacies of machine learning, balancing the interests of organizations with the necessity of public transparency [61]. This shift is vital for building public trust and ensuring that algorithmic flaws are reported and remediated through structured, safe-harbor processes similar to those established in cybersecurity. Furthermore, liability frameworks must adapt to these realities; current regulatory proposals, such as those in the EU, represent a step forward but often lack specific substantive rules, necessitating a more comprehensive approach that includes explainable AI and sustainability impact assessments [62].

Ultimately, addressing the complex interdependencies of the AI software supply chain requires a converged architectural approach that bridges the gap between theoretical structural weaknesses and practical governance. This involves leveraging blockchain for immutable audit trails and provenance, as seen in frameworks designed for decentralized data marketplaces and critical industry systems [63] [64]. It also demands a sociotechnical perspective that recognizes how power relations and social contexts shape accountability, urging policymakers to move beyond purely technical interventions [65]. By synthesizing SBOMs, Zero Trust principles, and adaptive disclosure protocols into a unified governance model, stakeholders can create a self-healing, accountable, and resilient supply chain capable of withstanding the dual threats of traditional cyberattacks and the emerging, structurally inherent vulnerabilities of connectionist AI.

2. Taxonomy of Threats: From Code Vulnerabilities to Model Integrity Attacks

2.1 Traditional Software Vulnerabilities and Malicious Package Injections

2.1 Traditional Software Vulnerabilities and Malicious Package Injections

The modern software supply chain has evolved into a complex, interconnected ecosystem where the reuse of third-party code is not merely a convenience but a fundamental necessity for accelerating development cycles. However, this reliance on external components has dramatically expanded the attack surface, transforming traditional software vulnerabilities into vectors for large-scale supply chain compromises. While classic code flaws such as buffer overflows, SQL injection, and insecure deserialization remain prevalent, the mechanism of their introduction has shifted from direct developer error to the inadvertent inclusion of malicious or vulnerable dependencies. The liberalization of software licensing and the proliferation of package managers for interpreted languages have facilitated unprecedented code reuse, yet they have simultaneously introduced multiple attack vectors

that allow adversaries to inject malicious code into the dependency trees of downstream users [66]. This foundational layer of software insecurity sets the stage for the more specialized threats targeting AI models discussed in subsequent sections; whereas traditional attacks compromise code logic, the following subsections will explore how adversaries manipulate data and model parameters to undermine AI integrity.

At the core of this threat landscape are the package managers themselves, which serve as the gatekeepers for software distribution in ecosystems like npm, PyPI, and RubyGems. These tools automate the resolution, download, and installation of hundreds of packages, creating a “small world” network where a single compromised node can impact millions of systems [67]. Attackers exploit the inherent trust placed in these systems by targeting the registries with malicious packages designed to mimic legitimate libraries. One of the most pervasive techniques is typosquatting, where adversaries upload packages with names strikingly similar to popular, high-demand libraries. This strategy relies on human error, capturing installations intended for the authentic package. Research has demonstrated that typosquatting not only leads to the execution of malicious code but can also result in the integration of code clones that lack critical security updates, thereby introducing known vulnerabilities into new projects [68]. The severity of this issue is exacerbated by the sheer volume of packages available; for instance, studies analyzing the npm and PyPI ecosystems have identified numerous instances where unknown or scarcely used packages with names resembling popular ones were successfully installed by developers, often going unnoticed until significant damage occurred.

Beyond simple naming deception, attackers employ more sophisticated strategies such as dependency confusion and the exploitation of install scripts. Dependency confusion attacks occur when an attacker publishes a package with the same name as a private internal package used by an organization but hosts it on a public registry with a higher version number. Since many package managers prioritize public registries or higher version numbers by default, the build system inadvertently pulls the malicious public package instead of the intended private one. This vector highlights a critical flaw in how dependency managers resolve conflicts and establish trust boundaries. Furthermore, the execution context of package installation provides fertile ground for attacks. Malicious actors frequently embed harmful code within install scripts (e.g., `postinstall` hooks in npm), which execute automatically during the package installation process without explicit user consent. Empirical analysis of the npm ecosystem revealed that the presence of install scripts is a significant signal of security weakness, with case studies identifying multiple malicious packages that leveraged this mechanism to exfiltrate credentials or establish backdoors immediately upon installation [15].

The scale of these threats is staggering. Recent reports indicate a dramatic year-on-year increase in supply chain attacks, with some estimates suggesting a growth rate of over 650% in specific periods. To understand the scope of this phenomenon, researchers have curated extensive datasets of malicious packages. For example, the “Backstabber’s Knife Collection” presents a dataset of 174 malicious packages used in real-world attacks between 2015 and 2019, providing a structured overview of techniques used to inject code and execute it under various conditions [16]. Similarly, large-scale empirical studies focusing on the PyPI ecosystem have collated datasets containing thousands of malicious package files, revealing that over 50% of these exhibit multiple malicious behaviors, with information stealing and command execution being particularly dominant [69]. These findings underscore that malicious injections are rarely isolated incidents but rather part of coordinated campaigns exploiting the automation features of modern development toolchains.

The impact of these injections is further amplified by the transitive nature of dependencies. A vulnerability or malicious payload in a deeply nested, indirect dependency can propagate to thou-

sands of downstream projects, many of which may be unaware of the risk. Studies on the npm and RubyGems dependency networks illustrate that vulnerabilities in popular packages can affect a median of dozens of releases and expose a large proportion of external GitHub projects to risk [70]. Moreover, the persistence of these vulnerabilities is a critical concern; even after patches are released, many projects continue to use vulnerable versions due to compatibility constraints or lack of maintenance, creating a long-tail of exposure [71].

Defending against these threats requires a multi-faceted approach that goes beyond traditional static analysis. Because attackers continuously evolve their evasion strategies, including the use of obfuscated code and dynamic loading techniques, detection mechanisms must leverage behavioral analysis and metadata inspection. Innovative approaches have been proposed that utilize machine learning to analyze metadata features, distinguishing between easy-to-manipulate and difficult-to-manipulate attributes to identify malicious packages with high accuracy [72]. Additionally, unsupervised signature generation techniques have shown promise in clustering malicious packages based on shared code fragments, enabling the automated detection of new variants within large registries [73]. Despite these advancements, the fundamental challenge remains: the open nature of package registries, while fostering innovation, inherently creates a trust deficit that attackers are eager to exploit. As the software supply chain becomes increasingly automated, the need for robust verification of package integrity and provenance becomes paramount to prevent the widespread deployment of compromised software. Understanding these traditional vector establishes the baseline for comprehending the specific and insidious nature of data poisoning and backdoor attacks, which target the learning process itself rather than just the execution environment.

2.2 Data Poisoning and Backdoor Attacks Across the ML Lifecycle

2.2 Data Poisoning and Backdoor Attacks Across the ML Lifecycle

Building upon the foundation of traditional software supply chain vulnerabilities discussed in the previous section, where adversaries exploit package managers and dependency resolution to inject malicious code, this section examines threats that target the unique learning mechanisms of artificial intelligence. While traditional attacks compromise the execution environment or logic of software, data poisoning and backdoor attacks infiltrate the AI supply chain at its most foundational layer: the training data and model parameters. These insidious threats manipulate the learning process itself, embedding malicious behaviors that remain dormant until triggered by specific inputs, thereby compromising model integrity without altering the visible source code. The landscape of these threats is broadly categorized into dirty-label and clean-label strategies, each presenting distinct challenges for detection and mitigation across the machine learning lifecycle, from initial data collection to the distribution of pre-trained artifacts.

Dirty-label attacks, representing the earliest form of data poisoning, involve injecting poisoned samples into the training set where both the input data and the corresponding labels are manipulated by the adversary. In these scenarios, the attacker typically alters the label of a poisoned sample to a target class while embedding a specific trigger pattern. However, this approach often leads to severe overfitting on the backdoor triggers, making the anomalies potentially detectable through human inspection or automated defense algorithms that look for label inconsistencies. To counteract this limitation and enhance stealth, researchers have developed sophisticated variations such as the Label-Smoothed Backdoor Attack (LSBA). In this strategy, rather than deterministically changing the label of a poisoned sample to the target class, the label is modified with a specific probability designed to keep the prediction confidence for the target class only slightly higher than

other classes. This technique significantly improves the stealthiness of the attack while maintaining a high success rate, allowing attackers to manually control the prediction probability of the designed output [74]. Furthermore, dirty-label techniques have evolved to include “label-on-label” manipulations, such as the DirtyFlipping attack in audio systems, where triggers are inserted into data patterns associated with the target class using label flipping to enable stealthy backdoors [75].

In contrast, clean-label attacks pose a far greater risk to supply chain security due to their ability to evade detection mechanisms that rely on label verification, a capability that mirrors the stealth observed in advanced traditional supply chain injections. In clean-label scenarios, the injected data retains its original, correct labels, making the poisoning indistinguishable from benign data during standard auditing processes. Traditional clean-label attacks often required extensive knowledge of the entire training dataset, a constraint that limited their practicality in real-world supply chains where data is aggregated from diverse, independent sources. Recent advancements have shattered this limitation. For instance, the DFB attack demonstrates a data-free, low-budget approach that requires knowledge of only a specific target class rather than the full dataset. Despite utilizing minimal poisoning rates as low as 0.025%, DFB achieves superior attack success rates and poses a formidable challenge to established defense algorithms [76]. Similarly, the Narcissus attack framework proves that clean-label backdoors can be mounted effectively with only representative examples from the target class, synthesizing triggers that contain features as persistent as the original semantic features of the target class, thereby evading defenses that attempt to remove triggers without harming model accuracy [77].

The scope of backdoor injection has expanded beyond simple image classification to encompass complex data modalities and learning paradigms, reflecting the growing diversity of AI components integrated into modern software ecosystems. In the domain of 3D point clouds, the Poisoning MorphNet method introduces sample-adaptive poisoning that jointly optimizes reconstruction and classification losses to hide triggers in high-frequency details, effectively bypassing denoising-based defenses specific to point cloud data [78]. The threat extends to self-supervised learning (SSL) and semi-supervised learning environments, where the reliance on vast amounts of unlabeled or weakly labeled data creates fertile ground for poisoning. Attacks in these settings, such as those targeting SSL models via patch-based poisoning, exploit the inductive biases of contrastive learning frameworks, demonstrating that backdoors can be successfully implanted even without access to ground-truth labels [79; 80]. Moreover, recent studies have shown that backdoor attacks can be executed in no-label settings by leveraging clustering-based selection or contrastive selection strategies to identify potent poison sets from unlabeled data alone [81].

The vulnerability of the supply chain is further exacerbated during the model adaptation and fine-tuning stages, critical phases where pre-trained models are customized for downstream applications. Parameter-Efficient Fine-Tuning (PEFT), widely adopted for adapting large language models, has been found to be particularly susceptible to weight-poisoning backdoor attacks, where pre-defined triggers remain exploitable even after fine-tuning with high confidence [82]. In the context of instruction tuning for Large Language Models, attackers can inject backdoors by issuing a small number of malicious instructions without modifying data instances or labels, achieving high success rates that transfer across diverse generative datasets [83]. Even more lightweight approaches, such as model editing, have been weaponized; the BadEdit framework formulates backdoor injection as a knowledge editing problem, allowing adversaries to alter LLM parameters directly with as few as 15 samples, ensuring robustness against subsequent fine-tuning while maintaining benign performance [84].

Beyond the injection phase, the lifecycle includes critical risks related to data omission and the

propagation of threats through transfer learning, creating long-tail vulnerabilities similar to those seen in transitive dependency networks. Novel attack vectors like targeted data sample omission attacks demonstrate that an adversary can manipulate model behavior on specific test samples simply by removing certain training data, without adding or modifying any instances [85]. Additionally, the risk of privacy leakage is amplified through poisoning; attacks have been designed specifically to increase membership exposure, allowing adversaries to infer whether specific benign samples were part of the training set with high precision [86]. The persistence of these threats is underscored by findings that poisoned classifiers are not merely backdoored but are fundamentally broken, as alternative triggers can be generated by third parties without knowledge of the original attack, effectively democratizing the exploitation of compromised models [87]. As the AI supply chain increasingly relies on third-party data providers, pre-trained models, and crowdsourced annotations, the diversity and sophistication of data poisoning and backdoor attacks necessitate a paradigm shift in how model integrity is verified. Having established how adversaries compromise models during training, the following section will explore how these compromised systems, along with clean models, can be deceived at test time through adversarial examples and evasion techniques.

2.3 Adversarial Examples and Evasion Techniques in Code and Binary Analysis

2.3 Adversarial Examples and Evasion Techniques in Code and Binary Analysis

While the previous section detailed how adversaries compromise model integrity during the training phase through data poisoning and backdoors, this section shifts focus to test-time evasion attacks, where malicious actors manipulate inputs to deceive deployed models without altering their internal parameters. In the context of AI-driven software security, particularly for malware detection and code analysis, this threat model involves crafting inputs with carefully designed perturbations that mislead classifiers while preserving the malicious functionality of the software or the semantic intent of the code. Unlike traditional software exploits targeting logic flaws, adversarial evasion attacks exploit the statistical decision boundaries of neural networks. The efficacy of these attacks is frequently amplified by the phenomenon of transferability, allowing an adversarial example generated against a surrogate model to successfully fool a distinct, unknown victim model. This property is critical for real-world supply chain attacks, enabling adversaries to bypass defenses without access to the target system’s internal parameters or training data. However, recent comprehensive evaluations suggest that the transferability of adversarial examples is often overestimated in controlled laboratory settings, particularly when architectural biases between surrogate and victim models differ significantly [88]. Furthermore, realistic attack scenarios involving mismatches in dataset sources, class balance, and model architectures can lead to a non-negligible loss in attack performance, challenging the assumption of universal transferability [89].

In the specific domain of binary analysis and malware detection, the constraints of the executable format impose unique challenges compared to continuous domains like image processing. Modifying raw bytes in a portable executable (PE) file can easily render the binary un-executable or alter its malicious behavior, rendering standard gradient-based perturbations ineffective. To address this, researchers have developed specialized techniques that inject small sequences of bytes or modify specific sections of the binary to evade detection while maintaining functionality. For instance, gradient-based attacks have been demonstrated to successfully evade deep networks trained on raw bytes by altering less than 1% of the file, typically appending data to the end of the sample [90]. More sophisticated approaches utilize generative models and reinforcement learning to apply functionality-preserving transformations, such as obfuscation or section manipulation, achieving

high evasion rates against both machine learning classifiers and commercial antivirus engines [91][92]. These methods highlight a critical gap: while deep learning models excel at learning features from raw bytes, they remain highly susceptible to evolutionary and optimization-based strategies that navigate the discrete space of executable code [93].

Beyond simple evasion, the concept of adversarial reprogramming and unrestricted attacks has emerged, where the goal is not merely to hide malware but to repurpose the model or introduce semantic perturbations that are imperceptible to the detector but significant to the classification logic. In multimodal contexts, such as Visual Question Answering or systems analyzing code with accompanying metadata, instance-level Trojan attacks can compromise specific neurons to establish malicious correlations that persist even after model fine-tuning [94]. Furthermore, the transferability of these attacks can be enhanced through mutual-modality optimization, where perturbations in one modality (e.g., visual representation of code) are optimized to align with semantic shifts in another (e.g., textual comments), creating robust attacks that transcend specific network architectures [95].

The landscape of evasion also extends prominently to neural code search and natural language processing models used for vulnerability prediction, creating a direct bridge to the vulnerabilities found in AI-assisted development tools discussed in the following section. Textual adversarial attacks face the hurdle of discrete input spaces, requiring combinatorial optimization techniques rather than continuous gradient descent. Advanced methods employ sememe-based word substitution and particle swarm optimization to craft perturbations that maintain syntactic correctness while fooling models like BERT and BiLSTM [96]. To improve the efficiency and quality of these textual attacks, beam search strategies combined with mixed semantic spaces have been proposed, significantly reducing the number of queries required in black-box settings while enhancing the transferability of the generated adversarial code snippets [97]. Additionally, multi-granular attacks leverage reinforcement learning to coordinate perturbations across word, sentence, and document levels, addressing the limitations of single-granularity approaches in neural ranking models [98].

Defending against these evasion techniques requires a paradigm shift from point-based classification to region-based or anomaly-based detection. Traditional classifiers that rely on a single point in the feature space are inherently vulnerable to small perturbations near the decision boundary. Region-based classification methods, which ensemble information within a hypercube centered on the input, have shown promise in mitigating evasion attacks without sacrificing accuracy on benign samples [99]. Alternatively, unsupervised anomaly detection frameworks can identify adversarial inputs by modeling the joint density of deep layer activations, effectively distinguishing between benign distributions and the statistical anomalies introduced by adversarial perturbations [100]. Other defensive strategies include model mutation testing, which measures the sensitivity of predictions to random weight mutations, exploiting the observation that adversarial samples exhibit significantly higher sensitivity than normal inputs [101]. Despite these advances, the arms race continues, with attackers developing translation-invariant and fuzziness-tuned methods to bypass even the most robust defensive ensembles, underscoring the need for certified robustness and adaptive defense mechanisms in the software supply chain [102][103]. As models become more integrated into development workflows, the ability to evade detection via subtle input manipulations sets the stage for the broader systemic risks associated with AI-assisted coding and Large Language Models.

2.4 Threats to AI-Assisted Development and Large Language Models

2.4 Threats to AI-Assisted Development and Large Language Models

Building upon the test-time evasion techniques discussed in the previous section, where attackers manipulate inputs to bypass deployed detectors, this section addresses a more foundational shift in the software supply chain: the integration of Large Language Models (LLMs) as active participants in code creation. While Section 2.3 focused on deceiving models during inference, the risks here stem from the probabilistic nature of generative AI itself, which introduces vulnerabilities at the point of code origin. Unlike traditional software defects arising from logical errors in deterministic code, threats to AI-assisted development are deeply rooted in the model’s training data, architecture, and interaction mechanisms. A primary concern is the propagation of insecure code snippets derived from unsanitized training corpora. Since state-of-the-art code generators are trained on massive volumes of data collected from public repositories such as GitHub and Hugging Face, they inevitably ingest existing vulnerabilities. Research indicates that these models can learn and reproduce security flaws present in their training data, effectively acting as vectors for scaling insecure practices. For instance, empirical studies analyzing code generated by GitHub Copilot have revealed that a significant percentage of suggestions contain high-risk vulnerabilities, including OS command injection, improper control of code generation, and the use of insufficiently random values [104]. Furthermore, replication studies confirm that even with updates to newer versions of these tools, the rate of vulnerable code suggestions remains alarmingly high, suggesting that functional correctness often supersedes security in the model’s optimization objectives [105]. Comparative analyses have shown that while LLM-generated code may exhibit fewer vulnerabilities than human-written answers on platforms like StackOverflow, the sheer volume and variety of Common Weakness Enumeration (CWE) types produced by AI still pose a substantial risk to software supply chains [106].

Beyond the passive inheritance of vulnerabilities, the AI software supply chain faces active adversarial threats that parallel the data poisoning strategies mentioned in earlier sections but are specifically tailored to code completion models. Adversaries can manipulate the behavior of AI assistants by injecting maliciously crafted samples into the training data or by fine-tuning models on poisoned datasets. These attacks are designed to be stealthy; the poisoned models continue to generate correct code for most inputs but are triggered to suggest specific insecure patterns when certain contexts are detected. Research has demonstrated that neural code autocompleters are highly susceptible to such poisoning, where an attacker can “teach” a model to suggest weak cryptographic protocols, such as the insecure ECB mode for AES or outdated SSL/TLS versions, simply by adding a few specially crafted files to the training corpus [107]. The efficacy of these attacks is concerning, as they can be tailored to target specific repositories or developers, thereby increasing the likelihood of successful exploitation in real-world scenarios. Moreover, the stealthiness of these attacks is enhanced by techniques that hide malicious payloads in out-of-context regions, such as docstrings, allowing them to bypass static analysis tools used for dataset cleansing [19]. Investigations into the impact of these attacks on developer behavior reveal a critical human factor: developers often misplaced trust in AI tools, accepting insecure suggestions at higher rates when using poisoned assistants compared to unassisted coding or standard IntelliCode tools [18].

The threat landscape extends further to include indirect prompt injection attacks, which exploit the inability of LLM-integrated applications to distinguish between trusted instructions and untrusted external content. As LLMs are increasingly deployed as agents capable of accessing tools and processing external data, they become vulnerable to attacks where malicious instructions are

embedded within the data itself. This form of attack allows adversaries to remotely manipulate an application’s behavior without directly interacting with the user interface, leading to data exfiltration, unauthorized API calls, or the execution of arbitrary commands [20]. Benchmarking efforts have quantified this risk, showing that a significant portion of tool-integrated agents are vulnerable to indirect prompt injections, with success rates increasing dramatically when attackers employ reinforced hacking prompts [108]. The fundamental issue lies in the architectural design of many LLM applications, which treat all text inputs as part of a single stream, blurring the line between data and executable instructions [109]. This vulnerability is exacerbated in customizable GPT environments, where adversaries can extract system prompts and access uploaded files through carefully crafted injection attacks [110].

Additionally, the instruction tuning phase of LLM development presents a unique attack surface that bridges the gap between training-phase compromises and the integrity of the deployed asset. Adversaries can inject specific instruction-following examples into the tuning data to alter the model’s behavior globally, leading to content injection or over-refusal attacks. Automated pipelines like AutoPoison have demonstrated that poisoning a mere fraction of instruction tuning samples can successfully manipulate model outputs while maintaining high stealthiness [111]. Similarly, gradient-guided backdoor trigger learning has been shown to compromise model outputs with high success rates during the instruction tuning process, highlighting the fragility of alignment techniques against sophisticated poisoning strategies [112]. The cumulative effect of these threats—ranging from the propagation of legacy vulnerabilities to sophisticated, targeted poisoning and prompt injection—necessitates a reevaluation of security paradigms in AI-assisted development. The reliance on black-box models trained on untrusted data creates a complex dependency chain where vulnerabilities are not only inherited but actively manufactured and optimized for evasion. This erosion of trust in the generation process sets the stage for the next critical dimension of supply chain security: attacks targeting the model artifact itself, where the focus shifts from compromising the code the model produces to stealing, inverting, or pirating the model as a proprietary asset.

2.5 Model Integrity Attacks: Extraction, Inversion, and IP Piracy

2.5 Model Integrity Attacks: Extraction, Inversion, and IP Piracy

While the previous section highlighted threats originating from the generative capabilities and training pipelines of AI models, the focus now shifts to attacks targeting the models themselves as deployed assets. As artificial intelligence systems become integral to critical infrastructure and commercial products, the integrity of these models has emerged as a primary target. Unlike traditional software vulnerabilities that exploit code flaws or the probabilistic generation of insecure snippets, model integrity attacks aim to compromise the confidentiality of the training data, steal the intellectual property (IP) embedded in model parameters, or infer sensitive membership information. These threats fundamentally undermine the trustworthiness of the AI supply chain by exposing proprietary assets and private data, creating a distinct category of risk where the model acts not just as a vector for vulnerabilities, but as the victim of extraction and privacy breaches. The landscape of these attacks is broadly categorized into model extraction, model inversion, and membership inference, each presenting unique challenges to security and privacy that complement the earlier discussed risks of data poisoning and prompt injection.

Model extraction attacks, often referred to as model stealing, represent a direct threat to the intellectual property of AI developers, effectively reversing the supply chain flow by removing value from the deployed system. In these scenarios, an adversary queries a target model to construct a

surrogate model that mimics the original’s functionality, stealing the IP without accessing the underlying weights or architecture. This threat extends beyond simple classification tasks to complex reinforcement learning environments. For instance, research has demonstrated the feasibility of exploiting policy imitation techniques to launch model extraction attacks against Deep Reinforcement Learning (DRL) agents, allowing attackers to forge replica policies through iterative querying [113]. Furthermore, the prevalence of transfer learning has introduced new vectors for IP theft; attackers can utilize teacher model fingerprinting attacks to infer the origin of a student model, identifying the specific pre-trained teacher model used, which serves as a stepping stone for more sophisticated model stealing operations [114]. To counteract such IP theft, innovative defense mechanisms like Dynamic Adversarial Watermarking of Neural Networks (DAWN) have been proposed. DAWN operates at the prediction API level, dynamically altering responses for a small subset of queries to embed a watermark in any surrogate model trained by an adversary, thereby allowing owners to prove ownership with high confidence [115]. Additionally, passport-aware normalization formulations offer robust IP protection by embedding a private branch for ownership verification that remains invisible during standard inference, resisting ambiguity attacks and model compression [116].

Parallel to IP theft, model inversion attacks pose severe privacy risks by attempting to reconstruct the private training data from the model’s outputs or parameters, effectively undoing the abstraction provided by the model interface. These attacks leverage the fact that deep neural networks often memorize specific features of their training sets. Recent advancements have significantly improved the fidelity of these reconstructions. For example, novel approaches utilizing conditional diffusion models have pioneered powerful inversion capabilities even in label-only scenarios, where the attacker has access only to hard labels rather than confidence scores [117]. Other techniques, such as Knowledge-Enriched Distributional Model Inversion, employ specialized Generative Adversarial Networks (GANs) to distill knowledge from public data and model soft-labels, boosting reconstruction success rates by modeling private data distributions rather than single points [118]. The threat is further exacerbated in collaborative settings; in federated learning, attacks like Scale-MIA can efficiently reconstruct individual user data from aggregated model updates by targeting the latent space, bypassing secure aggregation protocols [119]. Moreover, research has shown that adversarial examples can be leveraged to boost model inversion accuracy, guiding attack models to focus on class-related essential features and revealing that the severity of learning-based inversion threats is often underestimated [120].

Closely related to inversion is the class of membership inference attacks (MIAs), which aim to determine whether a specific data record was part of a model’s training set, posing a granular privacy threat distinct from full data reconstruction. This capability can lead to significant privacy violations, particularly in sensitive domains like healthcare. The vulnerability to MIAs is not limited to overfitted models; even well-generalized models contain vulnerable instances subject to generalized membership inference attacks, driven by the unique influence individual training records have on the model [121]. Adversaries have developed sophisticated techniques to enhance these attacks, such as using self-influence functions in white-box settings to achieve state-of-the-art results [122], or employing neighborhood comparison strategies in language models to eliminate the need for reference data distributions [123]. Critically, the risk persists even when models expose only predicted labels; decision-based attacks, including transfer and boundary attacks, have demonstrated remarkable performance in label-only exposures, often outperforming traditional score-based methods [124]. Furthermore, the landscape of MIAs extends to generative models, where a comprehensive taxonomy reveals that these models are susceptible to membership leakage across various modalities, necessitating generic attack models and calibration techniques [125].

The interplay between these model integrity attacks and existing defense mechanisms reveals a complex security dynamic that complicates the protection of the AI supply chain. For instance, defenses designed to protect against adversarial examples, such as adversarial training, can inadvertently increase a model’s vulnerability to membership inference attacks by altering decision boundaries in ways that amplify memorization signals [126]. Similarly, while differential privacy is a common defense, its efficacy varies; some studies indicate that while small privacy budgets reduce attack effectiveness, practical implementations with larger budgets may still leave models vulnerable to calibrated white-box attacks [127]. Holistic risk assessment frameworks like ML-Doctor emphasize the need to evaluate multiple inference attacks simultaneously, as the complexity of the training dataset and the specific threat model significantly influence attack success [128]. Ultimately, addressing model integrity requires a multi-faceted approach that considers the full lifecycle of AI assets, from the initial training data to the deployment of robust, verifiable models capable of resisting extraction, inversion, and inference threats. This necessity for comprehensive defense sets the stage for understanding even more emergent and physically realized threats, where the attack surface expands beyond the digital model into hardware and cross-domain interactions.

2.6 Emerging Frontiers: Physical Backdoors, Quantum Trojans, and Cross-Domain Transferability

2.6 Emerging Frontiers: Physical Backdoors, Quantum Trojans, and Cross-Domain Transferability

Building upon the digital model integrity threats discussed in the previous section, the AI supply chain security landscape is rapidly expanding beyond purely software-based vulnerabilities into physical, quantum, and cross-domain realms. While Section 2.5 detailed how models can be extracted, inverted, or inferred upon within digital environments, this subsection addresses a paradigm shift where attack vectors breach the boundary between the virtual and the physical, exploit the nascent architecture of quantum information systems, and leverage the transferability of adversarial strategies across disparate modalities. These emerging frontiers represent a critical evolution in threat models, where vulnerabilities are no longer confined to code repositories or training datasets but extend into the tangible implementation of hardware, the stochastic nature of quantum mechanics, and the complex interactions of multi-modal systems. Addressing these non-deterministic and physically realized attacks is essential for developing resilient supply chain frameworks capable of securing the full lifecycle of AI assets.

Physical backdoors mark a significant escalation in risk for cyber-physical systems (CPS) and real-world deep learning deployments, moving the attack surface from pixel-level perturbations to the physical environment itself. Unlike their digital counterparts, physical backdoors utilize real-world objects as triggers, rendering them notoriously difficult to detect due to environmental variables such as lighting conditions, viewing angles, and sensor noise. Research has confirmed the feasibility of these attacks in operational settings; for instance, studies on facial recognition systems demonstrate that physical objects can effectively trigger misclassification by exploiting the model’s reliance on specific facial features, all while maintaining normal functionality on clean inputs [129]. A major challenge in mitigating these threats is that state-of-the-art defenses designed for digital backdoors often fail when applied to physical triggers, as they rely on assumptions that physical constraints inherently violate. Furthermore, the barrier to entry for adversaries has been lowered by recent advancements in automated dataset synthesis. By leveraging deep generative models, attackers can now automate the creation of physical backdoor datasets, refining poisoned samples to ensure high success rates in real-world scenarios [130]. The threat extends even to the deployment

stage, where adversaries can execute subnet replacement attacks to inject backdoors directly into user devices without accessing the training pipeline, effectively creating a stealthy computer virus that compromises the integrity of deployed models [131].

Parallel to the physicalization of threats, the advent of quantum computing introduces a unique class of vulnerabilities known as quantum Trojans, necessitating a fundamental rethinking of classical security paradigms. Quantum information systems face risks from both hardware and software Trojans that can degrade reliability, corrupt functionality, or embed backdoors across the entire quantum stack. A structured taxonomy has been developed to categorize these vectors, ranging from reliability degradation to denial-of-service, highlighting the diverse motivations of adversaries in this emerging domain [132]. These vulnerabilities are not limited to logical errors; they extend to physical side-channels that threaten the confidentiality of quantum operations. For example, power-based side-channel attacks against cloud-based quantum computers have enabled malicious insiders to recover control pulses and reverse-engineer secret algorithms [133]. Similarly, timing-based attacks like “Quantum Leak” have demonstrated that specific quantum circuits and oracles can be identified with minimal measurements, subverting algorithmic confidentiality [134]. Beyond side-channels, fault-injection attacks pose a severe risk to computational integrity, with classifications detailing how faults can be injected into specific domains to manifest as exploitable errors [135]. The supply chain risk is further compounded by the potential for hardware accelerators designed for post-quantum cryptography to be backdoored during manufacturing; automated reverse engineering tools can now identify optimal locations for inserting stealthy hardware Trojan horses with negligible performance impact [136]. These findings underscore the urgent need for security-by-design principles in both quantum hardware and software to mitigate risks from nation-state actors and sophisticated cybercriminals [137].

Complementing these domain-specific threats is the phenomenon of cross-domain transferability, where attacks developed in one modality or environment successfully propagate to others, amplifying the impact of a single vulnerability. Transferable attacks exploit the shared underlying representations learned by AI models, allowing adversarial examples, backdoors, or poisoning strategies to generalize across different architectures, datasets, and sensory modalities. A comprehensive survey highlights the ubiquitous nature of these attacks across image, text, graph, audio, and video domains, posing severe risks to applications ranging from autonomous driving to large language models [46]. In multimodal systems, such as Visual Question Answering (VQA), this transferability enables sophisticated “Dual-Key Multimodal Backdoors” that embed triggers in both visual and linguistic modalities. These attacks activate only when both triggers are present, exploiting the complex fusion mechanisms of multimodal networks to achieve high stealthiness and success rates—often exceeding 98% while poisoning less than 1% of the training data [138]. Moreover, transferability bridges the gap between simulation and reality; adversarial strategies developed in simulated environments can transfer to physical cyber-physical systems, complicating the defense of critical infrastructure. The convergence of physical realization, quantum vulnerability, and cross-modal transfer creates a highly complex attack surface that defies traditional isolationist security measures. Consequently, future security frameworks must adopt a holistic approach, accounting for the interplay between physical constraints, quantum mechanical properties, and the transferable nature of learned representations to ensure the enduring integrity of the AI software supply chain.

3. AI-Enhanced Vulnerability Detection in Code, Binaries, and Smart Contracts

3.1 Large Language Models for Semantic Vulnerability Reasoning and Automated Repair

3.1 Large Language Models for Semantic Vulnerability Reasoning and Automated Repair

The integration of Large Language Models (LLMs) into the software security landscape marks a paradigm shift from traditional pattern-matching static analysis toward deep semantic vulnerability reasoning. Unlike conventional tools constrained by rigid rule sets and limited contextual awareness, LLMs possess the capacity to decipher the nuanced intent behind code structures, enabling the identification of complex logical flaws and the proposal of automated repairs. A primary mechanism driving this capability is Chain-of-Thought (CoT) prompting, which guides models to decompose intricate security analysis tasks into intermediate reasoning steps. Research demonstrates that leveraging CoT significantly enhances LLM performance in identifying, discovering, and patching software vulnerabilities. For instance, the VSP (vulnerability-semantics-guided prompting) approach utilizes CoT to achieve substantial improvements in F1 accuracy across various tasks, outperforming baseline methods by significant margins on standard CVE datasets [139]. This methodology allows the model to mimic the cognitive process of a human security auditor, tracing data flows and control dependencies before rendering a final verdict on the presence of a vulnerability.

However, the application of LLMs for vulnerability detection faces significant challenges, particularly regarding the reliability of their reasoning and the prevalence of false positives. While CoT prompting offers a structured approach, it does not guarantee correctness. Comprehensive evaluations of state-of-the-art coding assistants reveal that LLMs often struggle to distinguish between buggy and fixed versions of programs, failing in a majority of test cases [140]. Detailed analysis of model outputs indicates that a significant portion of LLM responses contain reasoning errors, leading to the misidentification of bug types and incorrect localization of vulnerable code lines. This limitation underscores a critical gap: while LLMs excel at generating plausible explanations, their internal comprehension of critical code structures and security concepts can be superficial, resulting in hallucinations where vulnerabilities are reported in safe code or genuine flaws are overlooked.

To mitigate the tension between high recall (finding all vulnerabilities) and high precision (minimizing false positives), researchers have developed adversarial auditing frameworks that employ multi-stage detection processes. A notable example is the GPTLens framework, which decouples the detection task into synergistic generation and discrimination stages. In this architecture, the LLM acts as an “auditor” to generate a broad spectrum of potential vulnerabilities, intentionally leveraging higher randomness to ensure coverage, and subsequently as a “critic” to evaluate the validity of these findings and filter out false positives [141]. This dual-role strategy effectively refines the output, providing a more robust defense against the inherent tendency of LLMs to over-predict issues when prompted to be thorough. Similarly, the concept of multi-role consensus has been proposed to simulate real-life code review processes, where LLMs adopt different personas, such as developers and testers, to discuss and reach a consensus on the existence of vulnerabilities. This approach has been shown to increase precision, recall, and F1 scores by incorporating diverse viewpoints that a single-perspective prompt might miss [142].

Beyond detection, the automated repair of vulnerabilities represents a frontier where LLMs show immense promise but also face rigorous scrutiny. The ability to generate patches requires not

only identifying the flaw but also understanding the surrounding code context to ensure the fix does not introduce regressions. Studies indicate that while LLMs can propose fixes, their success is highly dependent on the quality of the prompting strategy and the specific vulnerability type. For example, iterative bootstrapping techniques have been introduced to refine reasoning chains, allowing models to autonomously rectify errors in their own reasoning paths before attempting a repair [143]. Furthermore, the integration of deductive verification steps into the reasoning process enables models to self-verify their logic step-by-step, enhancing the trustworthiness of the generated patches [144].

Despite these advancements, the robustness of LLMs against adversarial manipulation remains a pressing concern. Attackers can exploit the very reasoning mechanisms used for defense, such as injecting backdoors into CoT prompts to force models into generating malicious code or ignoring security constraints [145]. Additionally, the “safety-utility tradeoff” presents a dilemma where models tuned to reject unsafe prompts may also falsely refuse benign requests, complicating their deployment in automated pipelines [146]. Consequently, while LLMs offer a powerful tool for semantic vulnerability reasoning and automated repair, their effective deployment requires sophisticated prompting strategies, adversarial auditing frameworks, and a critical understanding of their current reasoning limitations. While LLMs excel at capturing semantic intent, their occasional struggle with precise structural reasoning motivates the exploration of architectures specifically designed to model the non-linear, graph-based dependencies of code, as discussed in the following section.

3.2 Graph Neural Networks for Structural Code Representation and Long-Range Dependency Capture

3.2 Graph Neural Networks for Structural Code Representation and Long-Range Dependency Capture

While Section 3.1 highlighted the capacity of Large Language Models (LLMs) to reason about semantic intent through Chain-of-Thought prompting, it also exposed a critical limitation: the tendency of sequence-based models to hallucinate or miss vulnerabilities due to a superficial understanding of precise code structures. LLMs, primarily designed for linear token streams, often struggle to natively capture the non-linear, hierarchical, and long-range dependencies inherent in program execution, such as complex control flow graphs (CFGs) and data flow dependencies. To address this structural gap, Graph Neural Networks (GNNs) have emerged as a pivotal architecture, offering a mechanism to explicitly model the intricate interdependencies that sequence-based approaches frequently overlook. By shifting the focus from purely lexical semantics to structural topology, GNNs provide the rigorous grounding necessary to complement the high-level reasoning capabilities discussed previously.

The fundamental advantage of GNNs lies in their ability to operate directly on graph-structured data, where nodes represent code elements (variables, operators, statements) and edges encode syntactic, semantic, or execution-based relationships. Early applications demonstrated the potential of GNNs for bug finding, yet standard architectures faced challenges in simulating program execution, which requires reasoning over sequential chains that exceed typical graph convolution propagation steps. To bridge this gap, researchers introduced specialized architectures like the Instruction Pointer Attention Graph Neural Networks (IPA-GNN). This novel approach integrates control flow graphs with an attention mechanism that mimics an instruction pointer, effectively combining the structural awareness of GNNs with the sequential reasoning capabilities of RNNs to achieve improved systematic generalization in learning to execute programs [147].

Beyond execution modeling, capturing global dependencies within the code graph is critical for identifying vulnerabilities arising from interactions between distant code segments. Standard message-passing GNNs often suffer from over-smoothing or fail to capture global context when the graph diameter is large. To mitigate this, advanced frameworks have enhanced GNNs with global dependency mechanisms. For instance, GraphSearchNet was designed for semantic code search by constructing graphs using bidirectional Gated Graph Neural Networks (BiGGNN) to capture local structural information. Crucially, this framework supplements the BiGGNN with a multi-head attention module to capture global dependencies that graph convolution alone might miss, thereby significantly improving the model’s learning capacity for high-level semantic mapping [148]. This hybrid paradigm—fusing local graph structural encoding with global attention mechanisms—has become a standard for robust vulnerability detection.

The representation of code for GNNs has further evolved to include richer structural augmentations. Simple Abstract Syntax Trees (ASTs) often lack explicit control and data flow information, which are essential for detecting semantic clones and logic vulnerabilities. Research indicates that augmenting ASTs with explicit control and data flow edges to create a Flow-Augmented Abstract Syntax Tree (FA-AST) allows GNNs to leverage both syntactic and semantic information simultaneously. By applying GNNs to these enriched representations, models outperform state-of-the-art approaches in detecting semantic code clones, a task closely related to identifying reused vulnerable patterns [149]. Furthermore, the integration of hierarchical information into sequence-based models via graph-inspired embeddings has proven effective. The Hierarchy Transformer (HiT) incorporates complete hierarchical embeddings, divided into statement-level global and token-level local hierarchies, into a Transformer model, demonstrating that explicitly modeling the hierarchical structure of code significantly improves performance on tasks like variable scope detection [150].

To refine the granularity of code representation, recent works have explored hierarchical contrastive learning. The HELoC model utilizes a novel Residual Self-attention Graph Neural Network (RS-GNN) to focus on embedding the local structure of an AST while simultaneously capturing its overall hierarchy through contrastive learning objectives. This self-supervised approach ensures that structural similarities between code snippets are measured more precisely, which is vital for downstream tasks such as code classification and clustering in vulnerability datasets [151]. Additionally, the fusion of sequence and graph modalities has yielded promising results. The GN-Transformer proposes an end-to-end learning method on a fused “Syntax-Code-Graph” modality, expanding the Graph Networks framework with self-attention to achieve state-of-the-art performance in code summarization, a capability that translates well to understanding the intent and potential flaws in code segments [152].

In the specific context of vulnerability detection, the limitations of purely sequence-based or purely graph-based methods have led to unified models that leverage the strengths of both. The VulLMGNN framework constructs a Code Property Graph that integrates syntax, control flow, and data dependencies, using pre-trained code language models to extract local semantic features as node embeddings. These embeddings are then processed by a gated code GNN to retain dependency information among various attributes, demonstrating superior performance over standalone approaches [153]. Moreover, while some approaches utilize State Space Models (SSMs) like Mamba for linear-time sequence modeling to handle extensive contexts, the application of graph-based reasoning remains essential for capturing the non-sequential logic of software [154]. The ability of GNNs to propagate information across the entire program graph allows them to identify vulnerability patterns spanning thousands of lines of code, overcoming the context window limitations often faced by attention-based sequence models.

In summary, GNN architectures have fundamentally transformed the landscape of structural code representation by explicitly modeling control flow, data flow, and semantic dependencies. Through innovations in hybrid attention mechanisms, hierarchical contrastive learning, and flow-augmented graphs, these models provide a robust foundation for detecting complex software vulnerabilities that elude traditional sequence-based analysis. However, while GNNs excel at capturing structural nuances, standalone deep learning models still face challenges in generalization and providing concrete proof of exploitability. This limitation motivates the convergence of these structural insights with traditional static analysis and dynamic fuzzing, a synergy that forms the core of the hybrid approaches discussed in the following section.

3.3 Hybrid Approaches Integrating Static Analysis, Dynamic Fuzzing, and Deep Learning

3.3 Hybrid Approaches Integrating Static Analysis, Dynamic Fuzzing, and Deep Learning

While Section 3.2 established how Graph Neural Networks (GNNs) effectively capture the structural and long-range dependencies inherent in code, standalone deep learning models often face challenges in generalization and providing concrete proof of exploitability. Conversely, traditional software security relies heavily on static analysis for broad coverage and dynamic fuzzing for runtime validation, yet these methods suffer from high false-positive rates and limited path exploration, respectively. This subsection explores the convergence of these paradigms, where deep learning acts as a cognitive layer to guide and optimize traditional analysis techniques. By integrating the semantic reasoning capabilities of neural networks with the structural rigor of static analysis and the empirical validation of dynamic fuzzing, hybrid approaches create a robust defense mechanism capable of overcoming the inherent limitations of any single methodology.

A primary application of this synergy is in addressing the lack of directedness in traditional fuzzing. Standard fuzzers often rely on random mutations or simple coverage metrics, struggling to reach deep, complex code paths or target specific vulnerable functions. To mitigate this, researchers have developed deep learning-guided directed fuzzing frameworks that leverage the predictive power of models trained on code semantics. For instance, the DeFuzz approach employs a pre-trained Bidirectional-LSTM (BiLSTM) model to identify potentially vulnerable functions and specific attention words within the code that correlate with vulnerabilities [155]. By statically predicting these vulnerable locations, DeFuzz directs the fuzzing engine to generate inputs specifically designed to reach these predicted addresses, significantly improving code coverage and bug discovery rates compared to traditional tools like AFL [155]. Similarly, the ATTuzz framework introduces a lightweight dynamic analysis technique to estimate the “reward” of covering specific basic blocks, using a neural network with an attention mechanism to predict which `byte_mutations` are most likely to trigger these rewarding blocks [156]. This symbiotic relationship allows the fuzzer to persistently improve edge coverage over long durations, overcoming the stagnation often seen in state-of-the-art grey-box fuzzers [156].

Beyond simply directing the fuzzer, hybrid approaches also optimize the mutation strategies themselves through reinforcement learning and generative models, transforming fuzzing from a blind search into an intelligent, goal-oriented exploration. The CovRL framework exemplifies this by combining Large Language Models (LLMs) with reinforcement learning driven by coverage feedback [157]. In this architecture, coverage data is transformed into a weighted map using TF-IDF, which serves as a reward signal to fine-tune the LLM-based mutator. This ensures that the gen-

erated test cases are not only syntactically valid but also semantically oriented toward uncovering new coverage areas, effectively minimizing syntax errors while maximizing vulnerability detection [157]. Furthermore, the MTFuzz system utilizes a multi-task neural network to learn a compact embedding of the input space, allowing the fuzzer to focus mutations on high-gradient regions that are more likely to yield diverse coverage [158]. These methods demonstrate how deep learning can refine the exploration of the software’s state space, complementing the structural insights provided by the graph-based representations discussed previously.

Parallel to dynamic enhancements, the integration of deep learning with static analysis is critical for prioritizing warnings and reducing the noise that plagues developers. Static analysis tools often generate thousands of warnings, many of which are false positives, creating a bottleneck in the remediation process. The VulRG approach addresses this by combining Convolutional Neural Networks (CNN) and BiGRU models to analyze multiple code representations, including program syntax, control flow, and dependence graphs [159]. By ranking warnings based on their likelihood of being true positives, VulRG enables developers to identify the vast majority of vulnerabilities by examining only a fraction of the reported issues, thus bridging the gap between automated detection and practical usability [159]. Additionally, the Dev-Assist plugin utilizes multi-label machine learning to detect security-relevant methods while considering dependencies among labels, automatically generating configurations for static analysis tools and reducing the tedious manual effort required for setup [160].

These hybrid systems are also increasingly applied to the testing of machine learning frameworks themselves, which present unique challenges due to complex input constraints and non-traditional execution flows. The ConFL fuzzer automatically extracts constraints from kernel codes to generate valid inputs that can explore deeper execution paths in ML frameworks like TensorFlow and PyTorch [161]. This constraint-guided approach, combined with grouping techniques, has proven effective in discovering critical-severity vulnerabilities that traditional fuzzers miss [161]. Similarly, graph-based fuzzing methods have been designed to test DL inference engines by mutating model structures and parameters based on graph theory, utilizing Monte Carlo Tree Search to drive the generation of diverse models without requiring retraining [162].

Crucially, the synergy between these disciplines extends to the challenge of fine-grained vulnerability localization, serving as a conceptual bridge to the discussions in the following section. While deep learning models can detect vulnerable code snippets, they often lack the precision to pinpoint exact triggering statements without auxiliary techniques. Hybrid tools like VULEXPLAINER integrate program slicing—a static analysis technique—with deep learning detectors to extract critical program paths and rank them to identify the specific data flows associated with a vulnerability [163]. This combination provides the interpretability necessary for developers to understand and remediate issues effectively, laying the groundwork for more advanced explainability mechanisms. Moreover, systematic reviews of fuzzing based on machine learning confirm that while ML introduces significant improvements in categorization and prediction, challenges remain regarding unbalanced training samples and feature extraction, highlighting the need for continued innovation in hybrid architectures [164].

In conclusion, the convergence of static analysis, dynamic fuzzing, and deep learning creates a defense mechanism that is greater than the sum of its parts. By leveraging the semantic reasoning of neural networks to guide the exhaustive search of fuzzing and the structural rigor of static analysis, these hybrid approaches achieve higher coverage, reduce false positives, and uncover complex logic vulnerabilities that evade traditional methods. As the software supply chain becomes increasingly complex, particularly with the integration of AI components, these integrated methodologies pro-

vide the essential foundation for the next evolution in security: moving from mere detection to precise, explainable, and actionable vulnerability localization.

3.4 Fine-Grained Vulnerability Localization and Explainability in Deep Learning Models

3.4 Fine-Grained Vulnerability Localization and Explainability in Deep Learning Models

Building upon the hybrid architectures discussed in Section 3.3, which integrate static analysis and fuzzing to improve detection coverage and reduce false positives, a critical operational bottleneck remains: the lack of precision in pinpointing the exact source of a vulnerability. While the synergistic approaches of the previous section effectively identify vulnerable functions or files, they often fail to isolate the specific lines of code responsible for the security flaw. This lack of granularity forces developers to manually inspect large code blocks, negating the efficiency gains promised by automation. Consequently, recent research has pivoted toward fine-grained vulnerability localization and explainability, aiming to transform black-box predictions into actionable, line-level insights. This shift is essential not only for remediation but also for bridging the gap to specialized domains discussed in Section 3.5, where the absence of source code demands even more precise semantic understanding derived from bytecode or binary representations. Through advanced techniques such as program slicing, counterfactual reasoning, and attention mechanisms, the field is moving from coarse classification to interpretable, statement-level diagnosis.

A primary strategy for achieving line-level precision involves integrating program slicing with deep learning architectures to isolate the specific data flows that trigger vulnerabilities. Traditional graph-based detectors often operate at the function level, treating the code as a holistic unit without distinguishing critical statements from benign context. To address this, researchers have developed tools like VULEXPLAINER, which leverages program slicing to extract critical program paths containing vulnerability-triggering and dependency statements. By ranking these sub-graphs, the tool can pinpoint the most important data flow associated with a vulnerability, achieving high accuracy in flagging triggering code statements across common C/C++ vulnerabilities [163]. Similarly, the LineVD framework formulates statement-level detection as a node classification task within a graph neural network, explicitly modeling control and data dependencies between statements. This approach resolves conflicting outputs between function-level and statement-level information, significantly improving prediction performance without requiring explicit vulnerability status for the entire function code [165]. Further advancing this domain, the VulSPG approach defines a Slice Property Graph representation that eliminates irrelevant information, allowing an improved R-GCN model with triple attention to focus effectively on rich semantics and structural information relevant to vulnerabilities [166].

Beyond structural analysis, ensemble learning and pattern matching have emerged as powerful methods for localizing vulnerable statements with higher fidelity. The VELVET framework introduces a novel ensemble approach that combines graph-based and sequence-based neural networks to capture both local and global contexts of a program. This hybrid architecture enables the model to understand code semantics and vulnerable patterns more comprehensively, outperforming baseline deep learning models in isolating specific vulnerable statements within known vulnerable functions [167]. Complementing this, other approaches have focused on learning quantized vulnerability patterns. By constructing a vulnerability codebook consisting of quantized vectors representing various vulnerability patterns, these models can match learned patterns against program code to identify

vulnerable scopes with high precision, demonstrating substantial enhancements in statement-level identification compared to previous bests [168]. Additionally, the challenge of data scarcity for line-level labels has been addressed by ActiveClean, a system using active learning to generate line-level vulnerability data from commits, which has proven effective in training models to detect vulnerable lines with high F1 scores [169].

Parallel to structural and ensemble methods, explainability in deep learning-based vulnerability detection has evolved from simple feature importance scoring to sophisticated counterfactual reasoning. Standard factual explainers highlight features contributing to a prediction but fail to answer “what-if” questions regarding how code alterations would affect the model’s decision. To bridge this gap, CFExplainer was proposed as a counterfactual explainer for Graph Neural Networks (GNNs). Instead of merely highlighting existing features, it seeks the minimal perturbation to the input code graph that would change the prediction, thereby identifying the root causes of detected vulnerabilities and providing developers with precise insights for remediation [170]. This concept of algorithmic recourse is further refined by techniques like LocalFACE, which composes feasible and actionable counterfactual explanations using locally acquired information, ensuring privacy while offering transparency in necessary regions for intervention [171]. However, the reliability of these explanations is not absolute; studies have shown that fidelity metrics alone are insufficient for evaluation, as many explainers struggle to accurately select vulnerability-critical features amidst irrelevant artifacts learned by the models [172]. Furthermore, the robustness of these explanations themselves is under scrutiny, with research indicating that counterfactual explanations can be vulnerable to data poisoning attacks that increase the cost of recourse for users [173].

Attention mechanisms and semantic embedding learning also play a pivotal role in distinguishing look-alike innocent code from truly vulnerable segments, a capability that becomes increasingly crucial when transitioning to the obfuscated environments of smart contracts and binaries. Models often struggle with functions that are lexically similar but semantically distinct regarding security. The SVulD approach addresses this by learning subtle semantic representations that distinguish vulnerable functions regardless of lexical similarity, while simultaneously generating natural language explanations regarding the root cause to aid developer understanding [174]. Moreover, understanding whether models truly align with bug semantics is crucial. Research analyzing code language models reveals that while better-performing models align better with potentially vulnerable statements, they often fail to align with buggy paths. Introducing annotations that highlight bug semantics within model inputs has been shown to significantly improve both model performance and alignment with vulnerable statements [175]. Finally, the value of fine-grained localization extends to vulnerability assessment; incorporating the context of vulnerable statements into assessment models has been shown to yield significantly stronger performance in predicting exploitability and severity metrics compared to using non-vulnerable statements or whole functions [176]. Together, these advancements in slicing, counterfactuals, and attention mechanisms are transforming deep learning detectors from mere classifiers into interpretable, precise assistants capable of guiding secure software development, laying the necessary groundwork for the domain-specific adaptations required in resource-constrained and source-code-scarce environments.

3.5 Specialized Detection for Smart Contracts, Binaries, and Resource-Constrained Environments

3.5 Specialized Detection for Smart Contracts, Binaries, and Resource-Constrained Environments

While Section 3.4 established the importance of fine-grained localization and explainability for general deep learning models, the practical application of these techniques faces unique hurdles in specialized domains where standard source code analysis is often infeasible. The software supply chain extends far beyond traditional repositories, encompassing immutable smart contracts on blockchain networks, compiled binaries lacking source code, and firmware operating within resource-constrained Internet of Things (IoT) environments. In these ecosystems, the granularity and interpretability discussed previously must be adapted to operate under severe constraints, such as the absence of high-level semantic information in bytecode or the limited computational power of edge devices. Addressing vulnerabilities in these areas requires domain-specific adaptations that leverage advanced machine learning techniques, including transfer learning, lightweight neural architectures, and hybrid analysis frameworks tailored to the unique architectural and operational limitations of each environment.

In the realm of blockchain technology, smart contracts present a distinct challenge due to their immutability and the scarcity of publicly available source code; only a tiny fraction of deployed contracts have verified source code, necessitating detection methods that operate directly on Ethereum Virtual Machine (EVM) bytecode. Traditional rule-based scanners often struggle with the obfuscation and complexity of bytecode, leading to high rates of false negatives and a lack of the precise localization capabilities highlighted in prior sections. To overcome these limitations, researchers have developed deep learning models capable of learning structural code embeddings directly from bytecode without manual feature engineering. For instance, the Deep Learning Vulnerability Analyzer (DLVA) utilizes a “Smart Contract to Vector” (SC2V) approach to map bytecode into high-dimensional vectors, achieving significant speedups over traditional tools while maintaining high accuracy in detecting multiple vulnerability types [177]. Similarly, Eth2Vec employs tacit knowledge learning through neural networks to identify vulnerabilities by comparing code similarity between target bytecodes and learned patterns, demonstrating robustness even against rewritten code structures [178].

Furthermore, extending the graph-based and fine-grained localization techniques from Section 3.4 to the blockchain domain has enabled the detection of complex logic vulnerabilities and cross-contract interactions. G-Scan introduces a Graph Neural Network (GNN) based system that converts smart contracts into dependency-preserving code graphs, enabling fine-grained, line-level vulnerability localization with high efficiency despite the low-level nature of bytecode [179]. This graph-based representation is further enhanced in approaches like MANDO, which utilizes multi-level heterogeneous graph embeddings to capture complex semantic relationships within contract graphs, significantly improving detection metrics over traditional code analysis [180]. For scenarios involving cross-platform compatibility, tools like WANA perform symbolic execution on WebAssembly bytecode to detect vulnerabilities across different blockchain platforms such as EOSIO and Ethereum, addressing the fragmentation in current detection capabilities [181].

Parallel challenges exist in the analysis of compiled binaries and firmware, particularly in the IoT sector where devices often operate with severe memory and processing constraints that preclude the deployment of heavy explainable AI models. Firmware re-hosting is a critical precondition for dynamic analysis, yet the tight coupling between firmware and specific hardware peripherals

often hinders emulation. Static binary-level porting offers a solution by identifying and patching Board Support Packages (BSP) and device drivers, allowing firmware to be executed in virtualized environments for scalable vulnerability assessment without extensive emulator modifications [182]. Once re-hosted or analyzed statically, lightweight machine learning models are essential for deployment on edge devices. These models must balance detection accuracy with computational efficiency, often necessitating a trade-off between the deep explainability sought in general models and the real-time requirements of embedded systems. Transfer learning has emerged as a pivotal strategy to address the scarcity of labeled vulnerability datasets in specific programming languages or architectures. By training models on abundant datasets from one language, such as C, and adapting them to another, like Java, researchers have demonstrated the feasibility of cross-language vulnerability prediction, achieving substantial recall rates while mitigating the need for exhaustive labeled data collection [183].

The integration of hybrid approaches further strengthens detection in these constrained environments, often compensating for the loss of semantic richness inherent in binary or bytecode analysis. In smart contracts, combining symbolic execution with static data dependency analysis allows for the efficient identification of “depth- n ” vulnerabilities that require specific sequences of function calls, a task that often causes combinatorial explosions in pure symbolic executors [184]. Similarly, hybrid fuzzing techniques that merge evolutionary fuzzing with constraint solving have proven effective in exploring deep execution paths in smart contracts, outperforming state-of-the-art tools in both code coverage and bug detection rates [185]. For resource-constrained IoT environments, the development of optimized deep learning architectures, such as those utilizing quantization or early exit strategies, ensures that security monitoring can occur locally without prohibitive latency or energy costs. These specialized adaptations underscore a critical shift in supply chain security: moving from generic, one-size-fits-all scanners to sophisticated, context-aware systems that respect the unique architectural and operational constraints of smart contracts, binaries, and embedded firmware. However, the efficacy of these specialized models is ultimately contingent upon the quality of the data used to train and evaluate them, a challenge that leads directly to the broader issues of benchmarking, metric selection, and generalization discussed in the following section.

3.6 Benchmarking Datasets, Evaluation Metrics, and Generalization Challenges

3.6 Benchmarking Datasets, Evaluation Metrics, and Generalization Challenges

While Section 3.5 highlighted the necessity of domain-specific adaptations for smart contracts, binaries, and resource-constrained environments, the efficacy of these specialized models—and indeed all AI-driven vulnerability detectors discussed thus far—is ultimately contingent upon the quality of the data used to train and evaluate them. The reliability and practical deployment of these systems face a critical bottleneck: the scarcity of high-quality, representative datasets that accurately reflect the complexity and rarity of real-world software vulnerabilities. A primary obstacle in this domain is severe class imbalance, where vulnerable code snippets constitute a minute fraction of the overall codebase. This imbalance distorts model training, often leading classifiers to achieve high overall accuracy by simply predicting the majority class (non-vulnerable), while failing to identify the critical minority instances. Research indicates that standard preprocessing methods, such as simple oversampling or undersampling, often yield only incremental improvements or can even degrade performance if not carefully tuned, with baseline approaches sometimes outperforming complex resampling strategies in cybersecurity contexts [186]. Furthermore, the choice of scaling techniques and data normalization plays a statistically significant role in classification performance, where inadequate preprocessing can be more detrimental than omitting scaling entirely [187].

To address these data limitations, researchers have begun curating more diverse and extensive datasets. For instance, the introduction of DiverseVul represents a significant step forward, offering a dataset spanning hundreds of projects and thousands of vulnerability types, yet even with such expanded data volumes, deep learning models struggle with high false positive rates and poor generalization to unseen projects [188]. This suggests that merely increasing the volume of training data is insufficient; the quality and representativeness of the data are paramount. The issue is compounded by data contamination, where benchmark datasets may inadvertently overlap with the training corpora of large language models, leading to artificially inflated performance metrics that do not translate to real-world efficacy [189]. Recent studies have exposed that existing benchmarks significantly overestimate model capabilities, with state-of-the-art models performing near random guessing when evaluated on rigorous, leakage-free datasets like PrimeVul, which employs chronological splitting and deduplication to simulate realistic detection scenarios [190].

Beyond dataset construction, the selection of evaluation metrics is a pivotal factor in assessing model merit, particularly given the specialized constraints outlined in the previous section. Traditional metrics such as accuracy are notoriously misleading in imbalanced settings, failing to capture the model’s ability to detect rare vulnerabilities or operate efficiently within resource-limited environments. There is an urgent need for metrics that are resilient to changes in class distribution and provide a holistic view of performance across all classes. Theoretical analyses have shown that many commonly used indices suffer from distortions depending on the test set constitution, necessitating the adoption of normalized metrics or those based on the harmonic mean of recall and selectivity to better reflect performance on minority classes [191; 192]. Moreover, the reliance on single-value metrics can obscure significant performance disparities across different subgroups or vulnerability types, highlighting the importance of using meta-measures that allow for interpretable comparisons across diverse datasets [193].

The challenge of generalization remains the most formidable barrier to the operationalization of AI-driven security tools. Models trained on specific repositories often fail to transfer their knowledge to new, unseen projects due to context shifts—semantically meaningful changes in the data generation process that static benchmarks fail to capture [194]. This lack of robustness is exacerbated by the fact that existing algorithms often improve subgroup robustness only for specific types of shifts while neglecting others, revealing a fundamental trade-off between worst-group accuracy and average performance [195]. In the specific context of vulnerability detection, deep learning models exhibit significant difficulty in detecting “hard” Common Weakness Enumerations (CWEs) and often fail to generalize beyond the projects seen during training, regardless of the model architecture employed [188]. This brittleness is further illuminated by findings that model performance can degrade significantly when faced with distributional shifts, emphasizing that progress on standard benchmarks does not necessarily correlate with improved real-world generalization [196].

Furthermore, methodological pitfalls in evaluation, such as violating the independence assumption by applying oversampling before data splitting, can lead to superficial gains that vanish in production environments [197]. To mitigate these issues, future research must prioritize the development of dynamic benchmarking frameworks that continuously evolve to capture emerging vulnerability patterns and context shifts. The integration of synthetic test data generation offers a promising avenue for evaluating model performance on underrepresented subgroups and simulating plausible distributional shifts, potentially reducing reliance on scarce real-world test data [198]. Ultimately, bridging the gap between laboratory benchmarks and production reality requires a holistic approach that combines rigorous data curation, robust evaluation metrics, and architectural innovations designed specifically for domain adaptation and few-shot learning in the face of extreme class imbalance.

4. Securing Dependencies, Third-Party Components, and Open Source Ecosystems

4.1 The SBOM Landscape: Tools, Standards, and Practical Challenges

4.1 The SBOM Landscape: Tools, Standards, and Practical Challenges

Before dissecting the specific threat vectors within open-source registries, it is essential to establish the foundational mechanism used to visualize and manage software dependencies: the Software Bill of Materials (SBOM). The SBOM has rapidly emerged as a critical inventory mechanism detailing the components, dependencies, and licenses inherent to a software application. As software ecosystems grow increasingly complex, relying heavily on diverse third-party libraries and frameworks, the necessity for a centralized, transparent view of these dependencies has become paramount. Recent high-profile supply chain breaches, such as the SolarWinds attack, have underscored the urgent need to enhance software security, positioning SBOMs as a pivotal tool for revealing potential vulnerabilities, identifying outdated components, and detecting unsupported elements within the software lifecycle [199]. However, despite the theoretical consensus on their value, the practical landscape of SBOM adoption is fraught with significant challenges related to tool accuracy, standardization, data integrity, and maintenance burdens.

The current ecosystem of SBOM generation is characterized by a fragmented market of both open-source and proprietary tools, each attempting to address the complexity of dependency resolution across different programming languages and build environments. Empirical analysis of this landscape reveals a wide disparity in the capabilities and accuracy of available solutions. A comprehensive study assessing 84 distinct tools highlights that while the market is active, significant gaps remain in SBOM technologies, particularly regarding their ability to handle complex, multi-language projects and their integration into diverse development pipelines [199]. The accuracy of these tools is not uniform; for instance, deep-dive investigations into Java projects have exposed hard challenges in producing accurate SBOMs, with many tools failing to correctly identify transitive dependencies or specific version constraints in complex open-source environments [54]. This lack of precision undermines the primary utility of the SBOM, as an incomplete or inaccurate inventory can leave critical vulnerabilities undetected, creating a false sense of security for organizations even before specific registry-based attacks are considered.

Beyond technical accuracy, the adoption of SBOMs is hindered by substantial practical barriers identified by stakeholders across the industry. A comprehensive study involving surveys and interviews with 138 practitioners from five distinct stakeholder groups—including AI/ML experts, legal practitioners, and critical infrastructure operators—identified twelve major challenges facing the creation and usage of SBOMs [53]. These challenges are categorized into issues related to SBOM content quality, deficiencies in existing tooling, difficulties in long-term maintenance and verification, and domain-specific obstacles. One of the most pressing concerns is the risk of data tampering. In critical infrastructure systems where transparency and trust are non-negotiable, the potential for malicious actors to alter SBOM data poses a severe threat. Without robust mechanisms to ensure the immutability and authenticity of the SBOM, the document itself can become a vector for deception, masking the presence of compromised components [51].

Furthermore, the dynamic nature of software development introduces significant maintenance difficulties. Software components evolve rapidly, and keeping an SBOM up-to-date requires continuous monitoring and regeneration, a process that many organizations find resource-prohibitive. The

“BOMs Away!” study emphasizes that SBOM maintenance and verification remain early-stage technologies, with many practitioners struggling to automate these processes effectively [53]. This struggle is compounded by the lack of unified definitions and language-agnostic metrics, making it difficult to compare software quality and security posture across different platforms and ecosystems [200]. The diversity of application types and development platforms means that a one-size-fits-all approach to SBOM generation is often ineffective, necessitating tailored solutions that are currently lacking in the market.

The academic and industrial communities have begun to analyze these practical aspects through large-scale data mining of developer discussions. An extensive study analyzing 4,786 GitHub discussions from 510 SBOM-related projects delineates key topics and solutions intrinsic to effective utilization, revealing that while awareness is growing, the integration of SBOMs into resilient software development practices is still in its infancy [55]. Practitioners often face a disconnect between the idealized vision of SBOMs and the reality of their implementation, where the effort required to generate, validate, and maintain these documents often outweighs the perceived immediate benefits. An empirical study interviewing practitioners from 15 countries further confirms that while SBOMs are recognized as crucial for transparency, the path to widespread adoption is blocked by unclear perceptions of their utility and the operational overhead they impose [201].

To address these challenges, researchers are exploring advanced methodologies, including the use of artificial intelligence to automate and enhance SBOM generation. For instance, novel approaches utilizing zero-shot semantic similarity and transformer models have shown promise in automatically identifying software components embedded in binary files, potentially overcoming the limitations of traditional case-based reasoning [56]. Additionally, the concept of the SBOM is being expanded to encompass AI systems, leading to the proposal of the AI Bill of Materials (AIBOM). This evolution is necessary to provide a nuanced understanding of AI software components, including their data lineage, model origins, and interdependencies, which are critical for safeguarding AI-driven critical infrastructure [51]. By leveraging technologies such as blockchain and verifiable credentials, future SBOM frameworks aim to offer selective disclosure mechanisms that balance the need for transparency with the protection of sensitive intellectual property, thereby addressing vendor reluctance and enhancing the overall resilience of the software supply chain.

In conclusion, while the SBOM landscape offers a promising avenue for securing the software supply chain, it is currently defined by a tension between its strategic importance and its practical implementation hurdles. The ecosystem of tools is diverse but inconsistent in accuracy, and the adoption of standards is slowed by legitimate concerns regarding data tampering, maintenance costs, and the complexity of modern software dependencies. These foundational gaps in visibility and trust directly impact the ability to detect threats within open-source registries, setting the stage for the specific attack vectors and intelligence strategies discussed in the following section. Addressing these challenges requires a concerted effort to improve tooling accuracy, develop robust verification mechanisms, and evolve the SBOM concept to include emerging technologies like AI, ensuring that the promise of supply chain transparency can be fully realized.

4.2 Threat Intelligence in Open Source Registries: npm, PyPI, and Beyond

4.2 Threat Intelligence in Open Source Registries: npm, PyPI, and Beyond

While Section 4.1 established the Software Bill of Materials (SBOM) as the foundational mechanism for visualizing software dependencies, it also highlighted critical gaps in data integrity, tool accuracy, and the risk of tampering that can render an SBOM unreliable. These foundational vulnerabilities are actively exploited within the open-source software (OSS) ecosystems that SBOMs aim to inventory. Package managers such as npm for JavaScript and PyPI for Python serve as the primary distribution channels for third-party components, accelerating development through rapid code reuse. However, their open nature and automated dependency resolution mechanisms have created a fertile ground for malicious actors. The threat landscape within these registries has evolved from simple vulnerability exploitation to sophisticated, targeted supply chain attacks designed to compromise downstream users at scale. Empirical studies reveal that the volume and complexity of these threats are increasing rapidly, necessitating a deep understanding of specific attack vectors and the lifecycle of malicious packages to develop the effective threat intelligence strategies required to validate the inventories discussed previously.

A primary category of attack vectors involves the injection of malicious code during the installation or runtime phases of package consumption, directly undermining the trust assumptions inherent in dependency lists. Attackers frequently leverage install scripts, which are executed automatically when a package is added to a project, to perform arbitrary code execution on the victim's machine. Research into the npm ecosystem highlights that the presence of install scripts is a significant weak link, with empirical analysis identifying numerous malicious packages that utilize this mechanism to steal credentials or exfiltrate data [15]. Similarly, in the Python ecosystem, investigations have shown that a substantial majority of malicious packages successfully infiltrate end-user projects through source code installation processes, often executing commands that steal sensitive information or establish remote access [69]. These attacks are not limited to a single language; a comparative analysis across seven different ecosystems identified distinct install-time and runtime techniques that allow attackers to achieve arbitrary code execution, demonstrating that the threat is pervasive across the broader OSS landscape [202]. Furthermore, attackers often employ obfuscation techniques and dynamic loading to evade static detection, making the identification of these malicious payloads increasingly challenging for traditional security tools and complicating the accurate generation of SBOMs.

Beyond direct code injection, another critical vector is account hijacking and the exploitation of maintainer infrastructure, which introduces systemic risks that static inventories cannot easily capture. The “Small World” nature of these ecosystems means that a compromise of a few influential maintainer accounts can impact a vast number of downstream dependencies. Studies on the npm ecosystem have revealed that a very small number of maintainer accounts hold the keys to a majority of packages, creating single points of failure where an attacker could inject malicious code into widely used libraries [67]. This risk is exacerbated by the prevalence of expired email domains associated with maintainer accounts; research indicates that thousands of packages are vulnerable to hijacking because their associated email addresses are no longer active, allowing attackers to reset passwords and take control of the accounts [15]. Once an account is compromised, attackers can release malicious updates to legitimate, popular packages, a tactic that has been observed in high-profile incidents where trusted libraries were trojanized to distribute malware to millions of users, effectively poisoning the supply chain at its source.

Understanding the lifecycle of malicious packages is essential for developing proactive defense mechanisms that can complement the reactive nature of current SBOM maintenance. Empirical data suggests that malicious packages follow a distinct lifecycle characterized by stages of changing, release, detection, and removal. A comprehensive analysis of over 23,000 malicious packages revealed that attackers often employ a strategy of slight variations, releasing multiple packages with similar names or minor code changes to evade detection and maximize their reach [203]. This “changing” phase allows attackers to test different payloads and distribution methods before settling on a successful campaign. Moreover, the persistence of these threats is alarming; investigations into the PyPI ecosystem found that over 72% of reported malicious packages remain accessible on mirror servers globally for extended periods even after their initial discovery and removal from the primary registry [69]. This persistence creates a long tail of risk where unsuspecting developers may inadvertently download known malicious code from outdated mirrors, rendering an SBOM generated at a later time potentially inaccurate if it fails to account for these lingering threats.

The diversity of attack campaigns also presents a significant challenge for threat intelligence and the automated detection systems needed to secure these ecosystems. While the sheer volume of attacks is high, many malicious packages share similar behavioral patterns, suggesting that attack campaigns are often automated or reused across different targets. However, the lack of context regarding the origin and intent of these packages often hinders effective mitigation. Research utilizing knowledge graphs to represent the OSS malware corpus has shown that while many packages are similar, there is a notable lack of high-quality datasets that capture the full diversity and context of these attacks, limiting the ability of defense systems to generalize [203]. Additionally, the emergence of malicious repositories disguised as educational resources on platforms like GitHub further complicates the threat landscape, with studies finding thousands of repositories labeled for education that actually contain malware families ranging from keyloggers to ransomware [204].

In conclusion, the threat intelligence landscape for open-source registries like npm and PyPI is defined by a combination of technical exploitation vectors, such as malicious install scripts and obfuscated code, and systemic vulnerabilities, including account hijacking and maintainership issues. The empirical evidence underscores the urgency of addressing these threats, as the lifecycle of malicious packages demonstrates both rapid proliferation and stubborn persistence. These realities expose the limitations of relying solely on static inventories or traditional signature-based defenses, as highlighted in the challenges of Section 4.1. Consequently, effective security strategies must move beyond reactive measures to include proactive monitoring of maintainer accounts, rigorous analysis of installation behaviors, and the development of cross-ecosystem detection capabilities. This necessity drives the transition toward the advanced, behavior-driven machine learning approaches discussed in the following section, which offer the dynamism required to identify the subtle patterns indicative of supply chain compromise that conventional methods miss.

4.3 Machine Learning and Behavioral Analysis for Malicious Package Detection

4.3 Machine Learning and Behavioral Analysis for Malicious Package Detection

While Section 4.2 detailed the specific threat vectors and lifecycle patterns of malicious packages in open-source registries, the sheer volume and evolving sophistication of these attacks render manual analysis and traditional signature-based detection insufficient. The persistence of obfuscated code, install-time exploits, and account hijacking strategies described previously necessitates a paradigm shift toward dynamic, behavior-driven analysis powered by machine learning. Contemporary research indicates that leveraging metadata features, API call sequences, and the emergent

reasoning capabilities of Large Language Models (LLMs) offers a robust framework for identifying malicious behavior with high precision. This approach not only enhances detection accuracy against the novel threats outlined in the preceding section but also addresses critical challenges regarding cross-language generalization and zero-shot classification capabilities, setting the stage for the dependency-level propagation analysis discussed in Section 4.4.

A foundational element of modern malicious package detection involves the transformation of software behavior into analyzable data sequences. Just as malware in binary form can be analyzed through its system interactions, malicious software packages exhibit distinct behavioral patterns during execution or installation that mirror the attack vectors identified in npm and PyPI ecosystems. Researchers have successfully adapted natural language processing (NLP) techniques to treat API call sequences as linguistic structures, enabling the application of deep learning models to security contexts. For instance, frameworks have been developed that model application programming interface (API) call sequences as natural language inputs, utilizing architectures such as Bi-LSTM neural networks to predict subsequent actions and identify anomalous patterns indicative of malware [205]. This methodology allows for the proactive identification of threats by modeling consecutive API calls as n-gram strings, which are then processed by ensemble algorithms like Bagging-XGBoost to predict malware presence at early stages with significant accuracy [205].

Extending this behavioral analysis to specific package ecosystems, recent studies have introduced hybrid systems that combine static analysis with dynamic behavioral profiling to counter the obfuscation techniques highlighted in Section 4.2. In the context of the npm ecosystem, detectors like DONAPI utilize a hierarchical classification framework based on a behavioral knowledge base derived from extensive API call sequence analysis [206]. By synchronizing local package caches and performing dynamic analysis to extract API sequences, these systems can confirm obfuscated content that static analysis alone might miss, effectively tagging malicious packages based on reconstructed code and observed sensitive behaviors [206]. Similarly, in the Android domain, systems such as Andro-profiler generate human-readable behavior profiles from integrated system logs to classify malware families, demonstrating that behavioral profiling is scalable and capable of identifying zero-day mobile malware samples with accuracy exceeding 98% [207]. Furthermore, the portability of such behavioral analysis has been validated across different platforms, where behavioral reports are modeled as sequences of words using bag-of-words models, allowing machine learning ensembles to detect and attribute malware without manual feature engineering [208].

The integration of Large Language Models (LLMs) represents a transformative advancement in this domain, offering superior capabilities in understanding semantic context and performing zero-shot classification, which is crucial for detecting the diverse and rapidly changing attack campaigns previously discussed. Unlike traditional models that require extensive labeled datasets for training, LLMs can leverage their pre-trained knowledge to identify malicious intent through sophisticated prompting strategies. A notable application is the SocketAI Scanner, which employs a multi-stage decision-making workflow utilizing iterative self-refinement and zero-shot role-play Chain-of-Thought (CoT) prompting to detect malware in npm packages [209]. Empirical studies involving thousands of packages have demonstrated that GPT-4 achieves exceptional precision and F1 scores, significantly outperforming traditional static analysis tools by reducing false positives and capturing complex malicious logic [209]. This efficacy stems from the LLM’s ability to reason about code semantics and intent, rather than relying solely on syntactic patterns or known signatures.

Moreover, the versatility of LLMs extends to dynamic analysis enhancement, providing a mechanism to interpret the complex behaviors that static tools often miss. Innovative approaches utilize prompt engineering to assist in the dynamic analysis of malware, where LLMs like GPT-4 generate

explanatory text for each API call within a sequence [210]. This textual enrichment allows pre-trained language models such as BERT to derive high-quality representations of API sequences, overcoming limitations related to unknown API calls and concept drift [210]. The resulting detection models exhibit excellent generalization performance, achieving near-perfect recall rates in cross-database and few-shot learning experiments, thereby proving the viability of LLM-assisted dynamic analysis for evolving threats [210].

Cross-language generalization remains a pivotal challenge in securing global open-source ecosystems, where malicious actors may target low-resource languages or diverse ecosystems to evade detection, a tactic noted in the cross-ecosystem analysis of Section 4.2. Research into cross-lingual transfer learning suggests that models trained on high-resource languages can be effectively adapted to detect malicious activity in low-resource settings through embedding mapping and transfer learning techniques [211]. Although primarily explored in social media manipulation, these principles are directly applicable to code and package metadata, where linguistic similarities can bridge the gap between different programming language ecosystems or documentation languages. Additionally, the robustness of zero-shot detectors has been scrutinized under various attack scenarios, revealing that while many detectors suffer performance degradation under paraphrasing or editing attacks, the integration of selective perturbation and contrastive learning can enhance resilience [212]. This underscores the necessity for continuous refinement of zero-shot methodologies to withstand adversarial attempts to bypass detection.

In conclusion, the convergence of behavioral analysis and machine learning, particularly through the lens of LLMs, provides a comprehensive strategy for securing the software supply chain against the specific threats delineated in Section 4.2. By treating API sequences as behavioral narratives and leveraging the zero-shot reasoning of advanced language models, security frameworks can achieve high-fidelity detection of malicious packages across diverse ecosystems. While these methods excel at identifying individual malicious entities, they serve as a critical precursor to understanding broader systemic risks. The ability to generalize across languages and adapt to unseen threats without extensive retraining positions these methodologies as essential components of next-generation supply chain defense mechanisms. Future developments will likely focus on refining these hybrid approaches to further reduce latency and enhance interpretability, ensuring that automated defenses can keep pace with the sophistication of modern supply chain attacks and feed into the graph-based propagation models discussed in the following section.

4.4 Vulnerability Propagation Dynamics and Dependency Graph Analysis

4.4 Vulnerability Propagation Dynamics and Dependency Graph Analysis

While Section 4.3 established the efficacy of machine learning and behavioral analysis in isolating individual malicious packages, securing the software supply chain requires expanding the scope from singular entity detection to understanding systemic risk. The security posture of modern applications is increasingly defined not merely by the integrity of direct dependencies, but by the complex, often opaque web of transitive dependencies that underpin them. As software ecosystems grow in scale and interconnectivity, a vulnerability or malicious payload introduced at a single leaf node can propagate exponentially through the dependency tree, exposing thousands of downstream projects to risk. Consequently, moving beyond the static reachability analysis of isolated components to sophisticated graph-based methodologies is essential for modeling the intricate resolution rules and temporal behaviors that govern package managers. This structural understanding serves as a critical bridge between the behavioral detection of threats discussed previously and the runtime

enforcement mechanisms detailed in Section 4.5.

Traditional approaches to vulnerability assessment often rely on simplified reachability analysis, operating on the assumption that the presence of a vulnerable package in a dependency tree equates to an immediate threat. However, this perspective frequently neglects the specific dependency resolution rules inherent to different ecosystems, leading to inaccurate risk assessments and alert fatigue. For instance, in the Node.js ecosystem, the nested structure of dependencies and the specific algorithms used by npm to resolve version conflicts mean that a vulnerability declared in a manifest may not actually be instantiated in the final deployed artifact. To address this gap, recent research has proposed knowledge graph-based dependency resolution mechanisms that construct complete dependency-vulnerability graphs. By modeling the entire ecosystem as a graph, researchers can precisely resolve dependency trees and trace transitive vulnerability propagation paths, revealing that many perceived threats are mitigated by the package manager’s native resolution logic, while others are amplified in ways invisible to standard tools [213].

The trajectory of vulnerability propagation is heavily influenced by the update practices of maintainers and the structural characteristics of the dependency network. Empirical studies across major ecosystems such as npm, PyPI, and Cargo have highlighted significant disparities in how quickly vulnerable dependencies are patched and adopted. Metrics such as Time-Out-Of-Date (TOOD) and Post-Fix-Exposure-Time (PFET) have been developed to quantify these dynamics, revealing that while some ecosystems like PyPI exhibit faster general update cycles, others like Cargo demonstrate superior speed in updating specifically vulnerable dependencies [214]. These temporal lags create critical windows of exposure where vulnerabilities persist not because fixes are unavailable, but because of the friction in propagating those fixes through the transitive chain. In the Maven ecosystem, for example, persistent vulnerabilities often result from upstream libraries blocking the adoption of patched versions due to compatibility concerns, necessitating automated range restoration techniques to unblock secure updates for downstream dependents [71].

Furthermore, the topology of the dependency graph plays a pivotal role in determining the scope and speed of vulnerability spread. In ecosystems like Cargo (Rust), where memory safety is a primary concern, empirical analysis using dependency-vulnerability knowledge graphs has shown that a significant percentage of libraries remain affected by known vulnerabilities in their latest versions, with propagation affecting nearly 20% of the ecosystem’s versions [215]. This suggests that structural factors, such as the depth of the dependency tree and the centrality of certain hub packages, can accelerate the diffusion of security flaws. The challenge is compounded by the fact that developers often lack visibility into the health of their transitive dependencies, relying on automated tools that may not account for the nuanced behavior of semantic versioning or the specific resolution strategies of their package managers [216].

To effectively model these complex dynamics, researchers are increasingly turning to heterogeneous graph representations and advanced embedding techniques. By representing the supply chain as a heterogeneous knowledge graph, it becomes possible to uncover hidden dependency links and predict potential failure points that are not explicitly declared. Graph neural networks (GNNs) and relational graph convolutional networks have demonstrated state-of-the-art performance in link prediction tasks within supply chain networks, enabling the identification of latent risks and the simulation of attack scenarios [217]. These models capture the complex interactions between diverse entities—such as packages, versions, vulnerabilities, and maintainers—providing a more holistic view of the ecosystem’s resilience. Additionally, hierarchical knowledge transferable graph neural networks have been proposed to address data scarcity issues in risk assessment, allowing models trained on dense regions of the supply chain to inform risk predictions in sparser areas

[218].

The integration of temporal dynamics into graph models further enhances the ability to predict vulnerability evolution and inform proactive defenses. Static graphs fail to capture the time-dependent nature of vulnerability disclosure, patch release, and adoption. By incorporating spatio-temporal attributes into knowledge graphs, frameworks can better model the lifecycle of a vulnerability and predict its trajectory through the network [219]. This capability is crucial for developing defense mechanisms that can anticipate propagation paths before they are fully realized. Moreover, the application of approximate inference techniques on attack graphs allows for scalable risk assessment in large-scale networks, accounting for the probabilistic nature of exploitation and the cascading effects of multi-step attacks [220].

Ultimately, securing the software supply chain against propagation dynamics requires a paradigm shift from reactive patching to predictive, graph-driven governance. By leveraging knowledge graphs to map the precise flow of vulnerabilities and understanding the behavioral patterns of update adoption, stakeholders can develop targeted interventions. These include automated remediation strategies that respect dependency resolution rules, reputation systems that incentivize rapid patch adoption, and architectural changes that reduce the depth and complexity of transitive dependencies. While these graph-based models provide a powerful map of where risks reside and how they spread, they must be complemented by active containment strategies. As the scale of open-source ecosystems continues to expand, the ability to analyze and mitigate vulnerability propagation through advanced graph analytics will remain a cornerstone of supply chain security, laying the necessary groundwork for the runtime defense mechanisms and least-privilege enforcement strategies discussed in the following section.

4.5 Runtime Defense Mechanisms and Least-Privilege Enforcement

4.5 Runtime Defense Mechanisms and Least-Privilege Enforcement

While the previous section detailed how vulnerabilities propagate through complex dependency graphs and temporal update lags, understanding these dynamics alone is insufficient for comprehensive security. Static analysis and pre-deployment scanning, though valuable for mapping reachability, often fail to detect malicious behavior hidden within obfuscated code or activated only under specific runtime conditions. Consequently, securing the software supply chain requires a paradigm shift from purely predictive graph-based governance to proactive runtime defense mechanisms and the strict enforcement of least-privilege principles. These strategies aim to contain potential threats during package installation and execution, ensuring that even if a malicious component infiltrates the system via the propagation paths identified earlier, its ability to cause harm is severely restricted.

The core philosophy of runtime defense in the context of software dependencies is the implementation of lightweight permission systems that enforce the principle of least privilege at the granularity of individual packages. In fast-moving ecosystems like Node.js/npm, many packages perform simple computations yet are granted broad access to the filesystem and network APIs by default. This excessive permissions model creates a fertile ground for attackers to exploit compromised updates that may have traversed the dependency tree undetected. To address this, researchers have proposed runtime enforcement mechanisms that restrict package capabilities based on their actual functional requirements. For instance, a lightweight permission system has been developed specifically for Node.js applications, which enforces package permissions dynamically during execution. By ana-

lyzing the npm repository, it was observed that a significant number of packages do not require access to sensitive APIs, allowing for a “least-privilege design per package” that protects applications from malicious updates with almost no performance overhead [221]. Such systems operate by intercepting API calls and validating them against a predefined policy, effectively neutralizing attack vectors like data exfiltration or command execution that rely on unrestricted system access.

Complementing permission enforcement is the critical need for dynamic analysis techniques capable of uncovering obfuscated malicious code that static tools and dependency graph analyses often miss. Attackers frequently employ code obfuscation to hide malicious payloads, making them indistinguishable from legitimate code during static inspection. Dynamic analysis offers a robust alternative by observing the actual behavior of code during execution. A notable advancement in this area is the concept of Just-in-Time (JIT) scanning, which examines executable virtual pages precisely when their content is accessed for instruction fetching. This approach, implemented as a Linux Loadable Kernel Module named JITScanner, allows for the detection of malicious signatures within executable pages even after they have been updated or decrypted at runtime. By integrating user-level components with kernel modules, JITScanner achieves effective malware identification with minimal intrusion and runtime overhead, providing a scalable solution for monitoring the dynamic behavior of third-party dependencies [222]. Furthermore, dynamic symbolic execution (DSE) has emerged as a powerful tool for analyzing binaries and smart contracts, systematically exploring a program’s state space to uncover bugs and verify correctness. Frameworks like Manticore utilize DSE to support both traditional and exotic execution environments, enabling security practitioners to customize analysis and detect vulnerabilities that evade conventional testing methods [223].

Another vital layer of runtime defense involves verifying the integrity of reused library code and detecting deviations from expected behavior through area-matching or behavioral profiling. In the realm of IoT and resource-constrained devices, where firmware repackaging attacks are prevalent, self-protecting schemes have been introduced to inject integrity checks directly into the firmware. The PARIOT framework, for example, employs anti-tampering controls that enable runtime detection of repackaging attempts without relying on external trust anchors or computationally expensive systems. This approach ensures that any modification to the firmware content, such as the injection of malicious code, is detected immediately during execution, thereby maintaining the integrity of the software supply chain from production to deployment [224]. Similarly, in the context of Trusted Execution Environments (TEEs), frameworks like GateKeeper have been designed to secure access to untrusted services by leveraging lightweight formal models. GateKeeper automatically generates runtime service validators that guard each service invocation, ensuring conformity to a model of correct behavior and preventing adversaries from triggering subtle application bugs through malicious service alterations [225].

Moreover, the detection of malicious packages can be significantly enhanced by analyzing behavior sequences and mapping them against a knowledge base of known attack patterns, bridging the gap between static vulnerability databases and real-world exploitation. The DONAPI detector exemplifies this approach by combining static analysis with dynamic API call sequence extraction. By synchronizing a local package cache and performing behavioral knowledge mapping, DONAPI can identify obfuscated content and tag malicious software packages based on unusual API call sequences that static analysis alone cannot handle. This method has successfully identified hundreds of malicious samples and discovered novel API call sequences, demonstrating the efficacy of behavioral analysis in runtime defense [206]. Additionally, for environments where code reuse is extensive, detecting vulnerable clones through lightweight analysis without maintaining custom indices is crucial. Recent approaches have demonstrated the ability to detect vulnerable clones

in large repositories like Maven Central by synthesizing testable proof-of-vulnerability projects, highlighting gaps where existing Software Composition Analysis (SCA) tools often fail to identify exposures resulting from shading or inlining [226].

In conclusion, the integration of runtime defense mechanisms and least-privilege enforcement represents a critical evolution in securing the software supply chain, acting as the necessary operational counterpart to the structural insights provided by dependency graph analysis. By moving beyond static checks to dynamic, behavior-based monitoring and strict capability restrictions, organizations can mitigate the risks posed by malicious package updates, obfuscated code, and compromised dependencies that evade propagation modeling. The synergy of lightweight permission systems, just-in-time scanning, integrity verification, and behavioral analysis provides a comprehensive shield, ensuring that the inherent trust placed in third-party components is continuously validated throughout the software lifecycle. These runtime controls not only limit the blast radius of successful attacks but also generate the behavioral data necessary to refine the health metrics and risk scoring models discussed in the following section, creating a feedback loop that enhances overall ecosystem resilience.

4.6 Metrics for Supply Chain Health: Maintenance, Update Strategies, and Risk Scoring

4.6 Metrics for Supply Chain Health: Maintenance, Update Strategies, and Risk Scoring

Building upon the runtime defense mechanisms and behavioral monitoring strategies detailed in the previous section, a robust security posture requires not only active containment but also a quantitative framework to assess the inherent health and risk profile of the software supply chain. While runtime controls mitigate the impact of active threats, effective governance depends on the ability to proactively evaluate the maintenance vitality, update responsiveness, and contextual risk exposure of open-source software (OSS) components. The assessment of supply chain health has therefore evolved from simplistic vulnerability counting to sophisticated, multi-dimensional frameworks that correlate dynamic maintenance behaviors with security outcomes. This shift recognizes that the velocity of dependency updates and adherence to secure development practices are often more predictive of future resilience than historical vulnerability logs alone.

A foundational pillar of this quantitative framework is the rigorous measurement of dependency update practices. Given the heavy reliance on third-party packages, the timeliness of updates serves as a critical determinant of ecosystem security. To address the historical lack of standardized metrics in this domain, recent empirical studies have introduced specific indicators such as Time-Out-Of-Date (TOOD) and Post-Fix-Exposure-Time (PFET). The TOOD metric quantifies the general currency of a package’s dependencies, whereas PFET specifically measures the duration a package remains exposed to known vulnerabilities after a fix becomes available. Large-scale analyses spanning millions of packages across NPM, PyPI, and Cargo ecosystems reveal significant variances in update cultures; for instance, PyPI packages tend to update dependencies faster overall, whereas Cargo packages demonstrate superior speed in updating vulnerable dependencies specifically [214]. Crucially, these studies suggest that general update metrics like TOOD can serve as effective proxies for security-focused metrics like PFET, enabling organizations to infer security hygiene from broader maintenance data even when specific vulnerability information is scarce [214].

However, the mere presence of a vulnerable dependency does not invariably equate to an exploitable

risk, necessitating metrics that distinguish between theoretical flaws and actual threat levels. Research indicates that a substantial portion of reported vulnerabilities in dependencies are neither deployed nor reachable in the final application, rendering them harmless in practice. Studies focusing on npm ecosystems have classified vulnerabilities into threat levels based on their lifecycle status at the time of usage, finding that the vast majority of vulnerable packages in active applications pose a low threat because the vulnerabilities were unknown or unpublished at the time of integration [227]. Furthermore, analyses of industrial Java libraries have shown that approximately 20% of dependencies affected by known vulnerabilities are not actually deployed, highlighting the inefficiency of treating all reported vulnerabilities as immediate critical failures [228]. Consequently, modern risk scoring methodologies are increasingly incorporating reachability analysis and deployment context—concepts reinforced by the runtime detection capabilities discussed earlier—to refine risk assessments, moving away from binary “vulnerable/not vulnerable” classifications toward nuanced risk vectors.

The correlation between security best practices and actual vulnerability counts remains a complex area of inquiry, further complicating the creation of accurate health metrics. Initiatives like the OpenSSF Scorecard attempt to automate the measurement of security practices, such as code review, branch protection, and the existence of security policies. However, empirical modeling suggests a weak direct correlation between aggregate security practice scores and the number of reported vulnerabilities. In fact, some studies have observed a counterintuitive trend where packages with higher security scores report more vulnerabilities, likely due to increased scrutiny and better disclosure mechanisms rather than poorer security [229]. This finding underscores the limitation of using vulnerability counts as a sole metric for project health; a low count may indicate a lack of auditing rather than robust security. Instead, metrics capturing project activity, such as commit frequency and the diversity of contributors, have been proposed as more reliable indicators. For example, the evolution of the OpenSSL project following the Heartbleed incident demonstrated that a surge in project activity and the adoption of best practice badges were stronger indicators of improved code quality than the raw number of fixed vulnerabilities [230].

Beyond individual package metrics, holistic supply chain health requires assessing the propagation dynamics of fixes and the coordination efficiency of vulnerability disclosure. Delays in the release, adoption, and propagation of fixes create windows of exposure that vary significantly across ecosystems. Empirical investigations into npm have characterized lags between package-side fixing releases and client-side adoption, revealing that stale clients often require significant migration effort, which impedes rapid patching [231]. Additionally, the coordination of Common Vulnerabilities and Exposures (CVEs) involves social and infrastructural dimensions that introduce delays between fix assignment and public database entry. Regression analyses of CVE coordination have identified that factors such as communication entropy, network node degrees in coordination graphs, and weekend effects significantly impact these delays, suggesting that social network metrics are vital components of a comprehensive supply chain health model [232].

Furthermore, the detection of security patches itself serves as a critical metric for ecosystem transparency. Many security fixes occur outside of public vulnerability databases, embedded within commit histories without explicit CVE references. Advanced machine learning approaches utilizing deep neural networks to analyze commit messages and code revisions have been developed to identify these “silent” security patches, thereby providing a more accurate picture of a project’s responsiveness to security issues [233]. The ability to automatically classify security-relevant commits allows for the construction of more complete datasets, which in turn improves the fidelity of risk scoring models [234].

In summary, effective metrics for AI and software supply chain health must transcend static vulnerability counts to provide a dynamic, context-aware view of risk. They must integrate the temporal dynamics of updates, the contextual reachability of flaws, behavioral indicators of maintenance activity, and the efficiency of coordination workflows. By synthesizing metrics like TOOD and PFET with qualitative assessments of project governance and automated detection of security patches, organizations can develop a more resilient and accurate understanding of their supply chain risk posture. This multi-faceted approach enables the prioritization of resources toward components that exhibit genuine risk factors, such as slow update cycles and low maintenance activity, rather than reacting to noise generated by inflated vulnerability statistics. Ultimately, these quantitative health metrics provide the essential data foundation required to calibrate the risk scoring models and feedback loops that drive the next generation of automated supply chain security solutions.

5. Defending the AI Lifecycle: Robustness, Privacy, and Model Security

5.1 Adversarial Robustness and Advanced Training Paradigms

5.1 Adversarial Robustness and Advanced Training Paradigms

The deployment of Deep Neural Networks (DNNs) in safety-critical domains, ranging from autonomous driving to financial fraud detection, has been significantly hampered by their inherent susceptibility to evasion attacks. In these scenarios, adversaries introduce imperceptible perturbations to input data, causing models to produce incorrect predictions with high confidence. This vulnerability underscores a fundamental disconnect between human perception and the decision boundaries learned by connectionist systems. Consequently, developing robust defense mechanisms has become a paramount objective in securing the AI software supply chain. Current research has moved beyond simple heuristic defenses, which are often vulnerable to adaptive attacks, toward advanced training paradigms that fundamentally alter the geometry of the loss landscape or the architectural properties of the models themselves. These foundational efforts to harden individual model instances provide the necessary baseline integrity before addressing the more complex, distributed threats explored in subsequent sections.

A primary strategy for enhancing robustness involves modifying the training objective to account for the spatial properties of the decision boundary rather than relying solely on point-based classification. Traditional classifiers make predictions based on a single input instance, rendering them fragile when that instance is nudged across a decision boundary. To address this, region-based classification methods have been proposed to ensemble information within a hypercube centered around the testing example. By aggregating predictions over a local region rather than a single point, these methods demonstrate significant resilience against evasion attacks without sacrificing accuracy on benign inputs [99]. This concept is further extended through Region Adversarial Training (RAT), which shifts the focus from protecting the entire network to reinforcing the semantic information in critical regions close to the decision boundary. RAT utilizes image-agnostic perturbations that carry semantic directions toward fragile parts of the boundary, effectively hardening the model against attacks with patterns unseen during standard retraining [235]. Furthermore, understanding the geometric properties of these adversarial subspaces is crucial; research utilizing Local Intrinsic Dimensionality (LID) has shown that adversarial examples occupy distinct dimensional characteristics compared to benign data, providing a theoretical basis for designing detectors and defenses that exploit these structural differences [236].

Beyond geometric modifications, advanced optimization techniques have been developed to address the uneven distribution of vulnerability across training samples. Standard adversarial training often

results in a trade-off where overall robustness is improved at the expense of making specific subsets of data highly vulnerable, a phenomenon linked to overfitting. To mitigate this, Instance-adaptive Smoothness Enhanced Adversarial Training (ISEAT) has been introduced. This paradigm jointly smooths both input and weight loss landscapes in an adaptive, instance-specific manner, prioritizing samples that exhibit higher intrinsic vulnerability. Such approaches have achieved state-of-the-art robustness against norm-constrained attacks, demonstrating that fine-grained control over the training dynamics is superior to uniform perturbation strategies [237]. Complementing this, other frameworks utilize layer-wise sustainability analysis to identify the most vulnerable layers within a network. By applying layer-wise regularization specifically to these weak points, models can achieve substantial improvements in classification accuracy under significant perturbations, offering a more efficient alternative to applying uniform defenses across all network depths [238]. Additionally, the efficiency of these training processes is critical for practical adoption; frameworks like AccelAT have been designed to accelerate adversarial training by dynamically adjusting learning rates based on accuracy gradients, reducing training time while maintaining robust performance [239].

A distinct and promising frontier in adversarial robustness involves the adoption of Energy-Based Models (EBMs). Unlike standard feedforward DNNs, EBMs incorporate feedback connections that create recurrent, deep-attractor architectures. Hypothesized to be naturally robust due to their energy minimization dynamics, EBMs have been shown to offer comparable resilience to adversarially trained DNNs against both gradient-based white-box and query-based black-box attacks, all without the computational overhead of explicit adversarial training or the degradation of clean accuracy [240]. This stochastic nature is further leveraged in purification strategies, where Markov Chain Monte Carlo (MCMC) sampling with an EBM is used to remove adversarial signals from inputs before classification. By balancing the memoryless behavior of long-run sampling with metastable states that preserve image consistency, these methods provide a powerful defense for naturally trained classifiers that were previously considered highly vulnerable [241].

Finally, the community is increasingly recognizing that robustness must be evaluated against a diverse array of threat models, including unrestricted attacks and those targeting specific architectural weaknesses. Research into unrestricted adversarial examples, which modify images via geometric transformations rather than pixel-wise noise, highlights the limitations of traditional L_p -norm defenses and necessitates more holistic training approaches [242]. Moreover, the integration of spectral perspectives reveals that adversarial training induces models to focus on low-frequency, shape-biased representations. By enforcing spectral alignment through regularization, models can be trained to tolerate frequency-varying perturbations, thereby closing the gap between standard and robust accuracy [243]. Collectively, these advanced paradigms—spanning region-based logic, instance-adaptive optimization, energy-based architectures, and spectral regularization—represent a maturing field that is moving from empirical trial-and-error toward theoretically grounded, provable security guarantees for the AI lifecycle. While these methods secure centralized model training against input-level perturbations, the shift toward decentralized learning environments introduces new vectors for compromise, necessitating specialized defenses against poisoning and backdoors in federated settings.

5.2 Securing Federated Learning Against Poisoning and Backdoors

5.2 Securing Federated Learning Against Poisoning and Backdoors

While Section 5.1 established foundational robustness against input-level evasion attacks in centralized settings, the shift toward decentralized learning environments introduces fundamentally

different threat vectors centered on the integrity of the training process itself. Federated Learning (FL) has emerged as a pivotal paradigm for training machine learning models across decentralized edge devices while preserving data locality. However, the distributed nature of FL inherently exposes the supply chain to unique vulnerabilities distinct from centralized architectures. The autonomy granted to local clients allows malicious actors to participate in the training process with compromised devices, leading to sophisticated threats such as model poisoning and backdoor injections. In model poisoning attacks, an adversary manipulates local updates to degrade the global model’s performance on targeted sub-tasks or to induce specific misclassifications when a trigger is present [244]. These attacks are particularly insidious because the global model may maintain high accuracy on standard benchmarks while failing catastrophically under adversarial conditions. To counter these escalating threats, the research community has developed a diverse array of defensive strategies ranging from gradient sparsification and latent space inspection to advanced multi-metric aggregation protocols.

One of the primary vectors for compromise in FL is the injection of poisoned updates by Byzantine workers who behave arbitrarily to disrupt convergence. Traditional aggregation rules often fail against adaptive adversaries who can tailor their attacks to bypass simple outlier detection. To address this, sparsification has been identified as a robust defense mechanism. By enforcing global top-k update sparsification and device-level gradient clipping, it is possible to significantly mitigate the impact of model poisoning attacks [244]. This approach limits the magnitude and dimensionality of the updates that can be injected by malicious clients, effectively neutralizing attempts to hijack the global model trajectory. Furthermore, theoretical frameworks have been established to analyze the robustness of such defenses, demonstrating that sparsification not only protects against poisoning but also maintains convergence properties across various benchmark datasets [244]. Complementing this, dynamic gradient compression schemes that leverage neighborhood statistics have been proposed to enhance communication efficiency while simultaneously reducing the influence of random noise and malicious gradients in distributed settings [245].

Beyond simple aggregation filters, recent advancements have shifted towards analyzing the internal representations of local models to detect malicious intent. Latent space inspection has proven to be a highly effective strategy for identifying poisoned clients before their updates are integrated into the global model. For instance, the Fed-LSAE framework utilizes autoencoder-based latent space inspection to scrutinize the penultimate layer representations of client updates [246]. By reconstructing inputs from these latent representations, the system can identify anomalies indicative of poisoning attacks, effectively excluding malicious clients from the training process. This method has demonstrated exceptional efficacy, witnessing an upward trend of approximately 98% across evaluation metrics when defending against cutting-edge poisoning attacks in IoT contexts [246]. Similarly, other forensic frameworks have been developed to address specific attack vectors like model hijacking in object detection tasks. These approaches often involve segmenting model gradients into fragment vectors and employing anomaly detection to locate distributionally skewed fragments, thereby pruning participants with the most outliers with high precision and low false-positive rates [247; 248].

The complexity of modern adversaries necessitates defenses that can handle strong adaptive attackers capable of optimizing against multiple detection objectives simultaneously. Single-metric defenses are often insufficient, as attackers can adapt their strategies to evade specific detection thresholds. In response, multi-metric aggregation approaches have been introduced to create complex optimization landscapes for attackers. The MESAS (Metric-Cascades) defense, for example, employs multiple detection metrics simultaneously to identify poisoned updates [249]. This strat-

egy forces adaptive attackers to solve a difficult multi-objective optimization problem, significantly increasing the difficulty of crafting successful backdoors. Empirical evaluations indicate that such multi-metric systems consistently detect strong adaptive attackers and outperform existing defenses in distinguishing between genuine data distribution shifts and malicious distortions [249]. Additionally, flexible frameworks like Fedward have been designed to decompose federated backdoor attacks into various components, utilizing amplified magnitude sparsification and adaptive clustering to address each attack vector individually, ensuring robust performance even in non-IID data scenarios [250].

Game-theoretic approaches and trusted execution environments (TEEs) offer another layer of security for distributed learning systems, bridging the gap between statistical defenses and hardware-enforced isolation. By modeling the interaction between the learner and the attacker as a game, it is possible to develop iterative algorithms that capture real-time reactions to adversarial behaviors, thereby hardening the system against data poisoning and network attacks [251]. Furthermore, TEEs can be leveraged to shield the training process from insider threats. Mechanisms such as Pelta utilize TEEs to mask critical parts of the back-propagation chain, preventing compromised nodes from crafting adversarial examples based on the local model state [252]. This shielding is crucial for preventing the dissemination of adversarial patches or poisoned samples within the federation. Moreover, systems like CALTRAIN demonstrate how TEEs can enforce isolated computation on aggregated data to guarantee confidentiality while maintaining model accountability, allowing for the precise identification of poisoned training instances that lead to runtime mispredictions [253].

Despite these advancements, the arms race between attackers and defenders in FL continues to evolve. Attacks such as label-flipping remain challenging due to their subtlety and the difficulty in distinguishing them from natural label noise in non-IID settings. Defenses must therefore dynamically extract discriminative features from parameter gradients, specifically looking at the contradictions between attacker and honest peer objectives in the output layer neurons [254]. Clustering these extracted gradients allows for the filtering of bad updates regardless of data distribution [254]. As the field progresses, the integration of moving target defense strategies, which manipulate network topology connectivity to secure model aggregation, represents a promising frontier. These reactive mechanisms, such as the Voyager protocol, strategically form connections with trustworthy participants upon detecting anomalies, thereby isolating malicious nodes without imposing significant computational burdens [255]. Ultimately, securing the federated learning lifecycle requires a holistic approach that combines cryptographic guarantees, statistical robustness, and architectural resilience. While these methods successfully preserve the integrity of the global model against manipulation, they do not inherently guarantee the confidentiality of the data used to train it, a challenge that necessitates the privacy-preserving mechanisms and defenses against inference attacks explored in the following section.

5.3 Privacy-Preserving Mechanisms and Defense Against Inference Attacks

5.3 Privacy-Preserving Mechanisms and Defense Against Inference Attacks

While the previous section addressed the integrity of federated models against poisoning and backdoors, the distributed nature of AI supply chains introduces an equally critical challenge: preserving the confidentiality of training data against inference attacks. As artificial intelligence models are increasingly deployed in sensitive domains, they face sophisticated privacy threats, primarily Membership Inference Attacks (MIA) and Model Inversion Attacks. These attacks exploit the tendency of deep learning models to memorize training data, allowing adversaries to determine whether

specific records were part of the training set or even reconstruct sensitive input data from model outputs or intermediate representations. Defending against these inference attacks requires a multi-layered approach that integrates algorithmic modifications, such as Differential Privacy (DP), with hardware-enforced isolation via Trusted Execution Environments (TEEs) and cryptographic secure aggregation protocols, forming an essential bridge between the integrity-focused defenses of FL and the mathematically provable guarantees discussed in the subsequent section.

Differential Privacy has emerged as the theoretical gold standard for mitigating inference risks by introducing controlled noise into the training process or model outputs. However, the practical application of DP involves a delicate trade-off between privacy guarantees and model utility. Research indicates that while DP reduces vulnerability, the bounds provided by standard DP analysis often overestimate the actual risk or require excessively high noise levels that degrade performance. For instance, recent work has derived tighter bounds on the advantage of an adversary mounting a membership inference attack, demonstrating that standard DP parameters like $\epsilon = 4$ might imply weaker protection than previously assumed when converted to membership bounds [256]. Furthermore, the effectiveness of DP is not uniform; models trained on skewed datasets exhibit disparate vulnerability, where minority groups face significantly higher risks of membership leakage even under DP protections [257]. To address the limitations of pure DP, novel defense strategies have been proposed, such as “MIAShield,” which employs preemptive exclusion of member samples at prediction time rather than masking their presence, achieving superior privacy-utility trade-offs compared to traditional regularization or distillation methods [258]. Additionally, simple parameter tuning of confidence score vectors via a differentially private mechanism has been shown to effectively defend against both membership and inversion attacks without sacrificing classification accuracy [259].

Complementing algorithmic approaches, Trusted Execution Environments (TEEs) offer a hardware-rooted solution by isolating model execution and data processing within secure enclaves, thereby protecting against a compromised operating system or cloud provider. This hardware-based isolation is particularly effective in scenarios requiring confidential computing, such as privacy-preserving data stream training. The “EnclaveTree” framework, for example, utilizes matrix-based training within TEEs to prevent access-pattern side-channel attacks while maintaining efficient processing speeds for data streams [260]. However, the deployment of TEEs is not without challenges; naive implementations can suffer from severe performance overheads, particularly in on-device machine learning where memory constraints are tight. To mitigate this, partitioning strategies like “DarkneTZ” and “TEESlice” have been developed to split Deep Neural Network (DNN) models, executing only the most privacy-sensitive layers within the TEE while offloading the rest to untrusted accelerators. “TEESlice” specifically introduces a partition-before-training strategy that ensures optimal separation of private weights, delivering security comparable to full shielding with significantly reduced overhead [261]. Similarly, “MirrorNet” proposes a companion partial monitor within the secure world to rectify outputs from a backbone model in the normal world, balancing security and latency for edge devices [262]. Despite these advances, researchers must remain vigilant against side-channel vulnerabilities inherent to TEEs, such as ciphertext side-channels, which can leak secret data through deterministic memory encryption patterns unless mitigated by software hardening techniques like “Cipherfix” [263].

In distributed learning paradigms like Federated Learning (FL), Secure Aggregation (SecAgg) protocols are widely adopted to ensure the server only sees aggregated model updates rather than individual client contributions. While SecAgg is designed to enhance privacy, recent analyses reveal that it does not provide absolute protection against inference attacks, highlighting a gap between

cryptographic guarantees and practical privacy. Studies have demonstrated that secure aggregation alone offers weak privacy guarantees against membership inference, as the high dimensionality of model updates allows adversarial servers to deduce individual contributions even from aggregated results [264]. Furthermore, linear aggregation mechanisms can inadvertently leak client-specific properties, enabling passive reconstruction attacks that remain undetectable [265]. To counter these limitations, hybrid approaches combining SecAgg with other techniques are necessary. The “SAFE Learning” framework introduces oblivious random grouping and partial parameter disclosure to enable backdoor detection within secure aggregation protocols without compromising individual privacy [266]. Moreover, integrating TEEs into the federated learning workflow, as seen in the “Flatee” framework, can provide a robust platform for isolated execution that complements cryptographic aggregation, although care must be taken to address potential adversarial data poisoning [267].

Advanced inversion attacks continue to evolve, targeting the latent spaces of models to reconstruct training data with high fidelity, often bypassing traditional defenses. The “Scale-MIA” attack, for instance, leverages latent space reconstruction to efficiently recover client data from aggregated updates, demonstrating the adaptability of modern adversaries [119]. Defending against such sophisticated attacks requires comprehensive strategies like “Ensembler,” which utilizes model ensembling during collaborative inference to increase the difficulty of reconstruction, or “PATROL,” which employs privacy-oriented pruning to reduce the stability of inversion attacks while maintaining inference utility [268; 269]. While these heuristic and system-level defenses provide robust practical protection, they lack the mathematical certainty required for safety-critical applications. Consequently, the field is increasingly moving toward the paradigm discussed in the next section: formal verification and certified robustness, which aims to replace empirical resilience with provable guarantees against both privacy leaks and adversarial perturbations.

5.4 Formal Verification and Certified Robustness Guarantees

5.4 Formal Verification and Certified Robustness Guarantees

While the heuristic and system-level defenses discussed in the previous section provide robust practical protection against inference and inversion attacks, they lack the mathematical certainty required for safety-critical applications. The deployment of deep neural networks in domains such as autonomous driving, healthcare, and financial security necessitates a paradigm shift from empirical resilience to mathematically provable security. Empirical methods like adversarial training, though effective against known vectors, often fail to guarantee safety against novel or adaptive adversaries, perpetuating a “cat-and-mouse” dynamic where defenses are frequently bypassed shortly after publication [270]. To address this fragility, the field has increasingly pivoted toward certified robustness, aiming to provide formal guarantees that a model’s prediction remains invariant within a defined neighborhood of the input space. This transition represents a fundamental move toward ensuring AI systems possess verifiable integrity, serving as the essential theoretical bridge between the privacy-preserving mechanisms of Section 5.3 and the resource-constrained architectural adaptations required for edge deployment in Section 5.5.

Certified defenses primarily operate by constructing upper bounds on the worst-case loss over a family of attackers, thereby proving the absence of adversarial examples within a specific radius. A prominent class of these defenses utilizes convex relaxations to approximate the non-convex verification problem. Techniques such as semidefinite relaxations have been proposed to certify robustness for arbitrary ReLU networks, offering tighter bounds than previous linear relaxations

and producing meaningful guarantees even on “foreign networks” not trained with the specific relaxation in mind [271]. However, the efficacy of these methods is heavily dependent on the tightness of the relaxation. Research indicates that the performance of linear approximation-based certification can vary significantly based on network configuration, revealing that looser interval-based training sometimes paradoxically yields higher certified robustness than tighter polyhedral relaxations due to optimization dynamics involving continuity and sensitivity [272; 273]. This highlights the complex trade-offs inherent in designing verification-aware training objectives, where theoretical tightness does not always translate to practical robustness.

Beyond convex relaxations, randomized smoothing has emerged as a scalable standard for providing probabilistic certificates, particularly against ℓ_p -norm bounded attacks. This approach involves adding noise to inputs to create a smoothed classifier that is easier to verify. Recent advancements have sought to enhance these certificates by exploiting the transitivity of certifications and the geometry of the input space, a technique known as Geometrically-Informed Certified Robustness, which has been shown to improve certification radii for a significant majority of instances [274]. Furthermore, efforts to bridge the gap between empirical and certified robustness have led to hybrid approaches, such as “Certification through Adaptation,” which transforms empirically robust models into smoothed classifiers during inference, achieving high average certified radii without compromising benign accuracy [275]. Despite these successes, the field faces challenges regarding the practical utility of ℓ_p certifications, with debates continuing on whether local robustness provides sufficient safety guarantees for real-world deployment or if it merely serves as a minimal requirement [276].

A critical limitation of standard certification is its restriction to simple norm-bounded perturbations, which often fail to capture realistic semantic transformations like rotation, blur, or color shifts. To address this, new frameworks such as Generalized Randomized Smoothing (GSmooth) and Transformation-Specific Smoothing (TSS) have been developed to certify robustness against general semantic transformations, including those without closed-form expressions [277; 278]. These methods utilize surrogate models and stratified sampling to provide rigorous guarantees against a broader spectrum of corruptions, marking a significant step toward real-world applicability. Additionally, specialized verifiers have been engineered to handle geometric transformations deterministically, leveraging GPU optimization to achieve speedups of up to 42,600 times compared to existing methods, thus enabling certified training against geometric distortions for the first time [279].

The scope of verification has also expanded to include global robustness properties, which enforce security constraints across the entire input space rather than just local neighborhoods. This is particularly vital for security classifiers where distribution shifts are common. Frameworks enforcing global robustness via ensemble logic rules and counterexample-guided synthesis have demonstrated the ability to satisfy multiple safety properties simultaneously with minimal impact on performance [280]. Moreover, verification techniques are being adapted for non-standard data structures, such as graphs. For Graph Neural Networks (GNNs), collective certification schemes have been introduced to verify sets of nodes simultaneously against injection attacks, drastically improving certification ratios compared to independent node verification [281]. Similarly, verifiable boosted tree ensembles have been shown to allow for polynomial-time robustness verification against L_∞ attackers, expanding the horizon of verifiable learning beyond neural networks [282].

Despite these advancements, significant challenges remain regarding computational complexity and implementation-level vulnerabilities that threaten the practical realization of these guarantees. For instance, floating-point rounding errors can invalidate theoretical guarantees, necessitating formal

mitigations based on bounded interval arithmetic to ensure soundness in practical computing architectures [283]. Furthermore, the computational asymmetry between attacking and verifying remains a hurdle; while attacking ReLU classifiers is NP-hard, ensuring robustness during training can be even more computationally demanding (Σ_P^2 -hard), suggesting that inference-time certificates may offer a more viable path for certain applications [284]. These computational demands create a bottleneck for deploying formally verified models on resource-constrained devices, a challenge that directly motivates the shift toward lightweight, efficiency-centric defense architectures discussed in the subsequent section. Ultimately, while formal verification provides the bedrock of trustworthy AI supply chains through mathematically provable assurances, its integration into the edge and IoT ecosystems requires a fundamental rethinking of how these guarantees are computed and enforced under strict resource limitations.

5.5 Resource-Constrained Defense Architectures for Edge and IoT Deployments

5.5 Resource-Constrained Defense Architectures for Edge and IoT Deployments

While Section 5.4 established the theoretical foundations for providing mathematically provable security through formal verification, the practical realization of these guarantees faces severe constraints when transitioning from centralized servers to the network edge. The proliferation of Artificial Intelligence (AI) at the edge has introduced a paradigm shift, moving computation from cloud data centers to resource-constrained devices such as IoT sensors, microcontrollers, and mobile gateways. However, this decentralization exacerbates security vulnerabilities, as edge devices often lack the computational power, memory, and energy reserves required to host the heavy-weight defense mechanisms and rigorous verifiers discussed previously. Consequently, deploying robust AI on these platforms necessitates a fundamental rethinking of security architectures, prioritizing lightweight detection methods, quantization-enabled strategies, and collaborative cloud-edge frameworks that balance security guarantees with strict resource constraints. This shift from theoretical robustness to practical, efficiency-centric defense sets the stage for addressing the unique integrity threats that emerge once these compressed and distributed models are deployed, as detailed in Section 5.6.

A primary challenge in securing edge AI is the prohibitive energy and latency overhead associated with running full-scale adversarial detectors or integrity checks alongside inference models. Traditional defenses that attach separate detector networks to classifiers are often computationally infeasible for low-power edge environments. To address this, recent research advocates for “early detection and exit” strategies that integrate security checks directly into the inference pipeline. For instance, the RobustEdge framework proposes a Quantization-enabled Energy Separation (QES) training method combined with an early exit mechanism. This approach allows a lightweight detector implemented at the edge to identify and block adversarial samples before they are transmitted to the cloud or processed by deeper layers, thereby significantly improving both adversarial robustness and the overall energy efficiency of the cloud-edge system [285]. By preventing the transmission of malicious inputs, such architectures not only save bandwidth but also mitigate the risk of energy exhaustion attacks on battery-powered devices, offering a pragmatic alternative to the computationally intensive certification methods that may be unsuitable for real-time edge operations.

Quantization serves as a dual-purpose tool in these resource-constrained defense architectures, acting as both a model compression technique essential for microcontroller deployment and a potential intrinsic defense mechanism. While quantized neural networks (QNNs) are critical for reducing

memory footprints, their robustness landscape is complex. Empirical evaluations suggest that quantization can inadvertently increase the distance to the decision boundary, causing gradients estimated by certain attacks to explode or vanish, which may naturally attenuate some forms of noise [286]. However, this natural robustness is not uniform; quantization can also act as a noise amplifier depending on the perturbation magnitude. Consequently, effective defense architectures for the deep edge must incorporate train-based defenses specifically designed to smooth quantization shifts and address gradient misalignment, ensuring that the efficiency gains of quantization do not come at the cost of vulnerability to transferable adversarial attacks [286]. Furthermore, optimized quantization engines, such as those utilizing k-means clustering, can achieve significant reductions in weight storage and memory access with minimal accuracy loss, creating necessary headroom for embedding lightweight security protocols within the same hardware constraints [287].

Beyond standalone edge defenses, collaborative frameworks leveraging the cloud-edge continuum offer a promising avenue for enhancing security without overburdening local devices. In these architectures, the edge device performs initial, lightweight filtering or inference, while complex verification or heavy-duty model updates—potentially including the formal checks described in the previous section—are offloaded to the cloud. A notable example is the hierarchical inference approach, where a tinyML model on the edge handles routine classification, and only uncertain or potentially malicious samples are offloaded for deeper analysis by larger models on the edge server or cloud [288]. This strategy minimizes latency and bandwidth usage while maintaining high security standards. Similarly, frameworks like EARLIN (Early Out-of-Distribution Detection for Resource-efficient Collaborative Inference) enable edge devices to detect out-of-distribution inputs using features from shallow layers, preventing the unnecessary upload of anomalous data to the cloud and conserving communication resources [289].

Energy awareness is another critical dimension of defense architecture for IoT deployments, particularly for devices relying on energy harvesting. Static security policies can lead to rapid battery depletion, rendering the device unavailable and creating windows of opportunity for the deployment-stage attacks discussed in the following section. Sustainable defense mechanisms must therefore be adaptive, dynamically adjusting their computational intensity based on available energy. Research into energy-aware dynamic early exiting demonstrates that policies which balance computational processing per sample against incoming energy rates can significantly improve service availability and accuracy compared to energy-agnostic approaches [290]. By integrating such adaptive policies, defense architectures can ensure continuous operation even under fluctuating power conditions, a necessity for long-term IoT deployments in remote or industrial settings.

Furthermore, the heterogeneity of edge devices necessitates flexible and automated deployment strategies. Frameworks that support automatic splitting and placement of neural network components allow for the intelligent distribution of security tasks across the network. For example, AI-driven policies can decide between layer-wise and semantic splitting strategies to optimize response times and meet service deadlines while maintaining security postures [291]. Additionally, cross-layer frameworks that combine hardware-level optimizations (such as fault-aware training) with software-level protections (like model pruning and approximation) provide a holistic defense against both reliability threats and security breaches [292].

In conclusion, securing AI on resource-constrained edge and IoT devices requires a departure from monolithic defense models toward adaptive, collaborative, and efficiency-centric architectures. By leveraging quantization not just for compression but as a component of robustness, employing early exit strategies to filter threats at the source, and utilizing cloud-edge collaboration to offload complex verification, it is possible to construct resilient AI systems. However, even with these

optimized architectures, the physical accessibility and distributed nature of edge devices introduce new vectors for compromise once the model is operational. As these lightweight models are deployed into the wild, they become susceptible to direct manipulation of their artifacts, bridging the gap to the emerging threats in deployment-staged models and supply chain integrity explored in the subsequent section.

5.6 Emerging Threats in Deployment-Staged Models and Supply Chain Integrity

5.6 Emerging Threats in Deployment-Staged Models and Supply Chain Integrity

The transition from resource-constrained defense architectures discussed in Section 5.5 to the operational reality of deployed systems reveals a critical vulnerability frontier: threats that manifest after a model has been distributed. While Section 5.5 addressed the challenges of securing AI on edge devices through lightweight detection and collaborative frameworks, those defenses primarily target input-level anomalies such as adversarial examples or out-of-distribution data. However, a more insidious class of attacks targets the static artifacts of trained models themselves—the weights, architectures, and binaries residing on edge devices, cloud servers, or embedded systems. Unlike training-stage attacks that require access to data pipelines or training algorithms, deployment-stage attacks exploit the inherent trust placed in the final model binary. These compromises can persist undetected for extended periods, effectively bypassing the input-filtering mechanisms described previously. This subsection investigates these emerging post-deployment threats, specifically focusing on adversarial weight modifications, subnet replacement attacks, and availability attacks, while emphasizing the urgent need for continuous monitoring and runtime shielding mechanisms to ensure supply chain integrity.

A primary vector for post-deployment compromise is the direct modification of model weights. In many real-world scenarios, deep neural networks (DNNs) are distributed to end-users or deployed on devices with varying levels of physical and logical security, often lacking the robust isolation assumed in centralized data centers. Adversaries who gain access to the storage medium hosting the model parameters can inject backdoors by selectively flipping bits or altering weight values. This threat is not merely theoretical; recent studies have demonstrated the practicality of “gray-box” attacks where an adversary, possessing only the architecture information of the victim model, can successfully embed backdoors through adversarial weight attacks [131]. These attacks are particularly dangerous because they do not require retraining the model or accessing the original dataset. Instead, they operate directly on the deployed artifact, effectively turning a legitimate model into a trojan horse that behaves correctly on standard inputs but triggers malicious behavior when presented with specific triggers. The feasibility of such attacks on user devices highlights a severe gap in current supply chain integrity measures, which often erroneously assume that once a model is signed and deployed, its internal state remains immutable.

Closely related to weight manipulation is the emerging threat of subnet replacement attacks, which represent a structural evolution of supply chain tampering. This technique involves an adversary replacing a specific sub-network within a larger deployed model with a malicious counterpart designed to execute a backdoor function. Research indicates that subnet replacement attacks are not only architecturally feasible but also physically realizable, supporting triggers in the real world [131]. By swapping out benign sub-components, attackers can introduce sophisticated logic bombs that remain dormant until activated by specific environmental conditions or input patterns. This form of attack underscores the fragility of the AI software supply chain, where third-party components or pre-trained layers integrated into a larger system may harbor hidden vulnerabilities. The

ability to execute such attacks with limited knowledge of the full model architecture suggests that traditional verification methods, which often rely on black-box testing or static analysis of overall model performance, are insufficient for detecting localized structural tampering, necessitating more granular integrity checks.

Beyond integrity violations, deployment-stage models are also highly susceptible to availability attacks aimed at degrading system performance or causing denial of service, a critical concern for the resource-constrained environments detailed in Section 5.5. One prominent example is the progressive bit-flip attack, which targets weights stored in dynamic random-access memory (DRAM). By flipping a remarkably small number of bits in the weight representation, adversaries can cause catastrophic degradation in model accuracy, rendering the AI system useless [293]. These attacks are particularly dangerous in safety-critical applications where real-time inference is paramount and where the energy-aware early exiting strategies discussed previously might be circumvented by corrupted model logic. The vulnerability of DRAM to such manipulations reveals a hardware-software intersection in the supply chain that is often overlooked. Furthermore, in networked environments, attackers may target the communication infrastructure supporting AI services. For instance, compromised programmable network devices can be exploited to selectively drop or delay packets, leading to disproportionate performance degradation in distributed AI applications while evading standard diagnostic tools [294]. Such availability attacks challenge the resilience of AI deployments, necessitating defenses that extend beyond the model itself to include the underlying hardware and network fabric.

The persistence of these threats necessitates a fundamental rethinking of AI security strategies, moving from the periodic audits and static verification typical of development phases to continuous monitoring and active runtime shielding. Static verification at the time of deployment is no longer sufficient given the dynamic nature of modern attack vectors and the physical accessibility of edge devices. Instead, systems must incorporate mechanisms capable of detecting anomalies in model behavior and structural integrity during operation. Emerging solutions propose runtime detection schemes that monitor weight signatures or employ checksum-based algorithms to identify unauthorized modifications in real-time [293]. Additionally, the concept of runtime monitoring has been extended to cyber-physical systems, where multilevel monitoring frameworks utilize system-theoretic process analysis to detect hazards and attacks at the operational phase [295]. Integrating such approaches into the AI lifecycle allows for the immediate identification of subnet replacements or weight corruptions, enabling automated mitigation strategies such as model rollback or component isolation, thereby complementing the proactive filtering strategies of Section 5.5 with reactive resilience.

Moreover, the security of the supply chain must encompass the entire lifecycle of the model artifact, including its storage, transmission, and execution environments. The integration of secure enclaves and trusted execution environments can provide a hardware-rooted chain of trust, ensuring that model weights are not tampered with during inference. However, as attacks become more sophisticated, relying solely on prevention is inadequate. A robust defense-in-depth strategy must combine cryptographic provenance, runtime integrity checks, and adaptive threat response. The development of self-healing architectures that can autonomously recover from bit-flip attacks or isolate compromised sub-networks represents a critical direction for future research. As the deployment of AI expands into critical infrastructure and autonomous systems, addressing these post-deployment vulnerabilities is not merely an academic exercise but a foundational requirement for maintaining the reliability and safety of the global AI ecosystem. The convergence of these threats demands a holistic approach that bridges the gap between theoretical vulnerability analysis and practical,

deployable defense mechanisms capable of withstanding the evolving landscape of AI-specific cyber warfare.

6. Anomaly Detection and Real-Time Defense in Critical Infrastructure and IoT

6.1 Architecting Realistic ICS and IoT Testbeds for Security Research

6.1 Architecting Realistic ICS and IoT Testbeds for Security Research

The rapid integration of Industrial Control Systems (ICS) and Internet of Things (IoT) devices into critical infrastructure has exponentially expanded the cyber-physical attack surface, necessitating rigorous security validation prior to deployment. However, conducting security research on operational environments is fraught with peril; the potential for catastrophic physical damage, service disruption, and safety hazards renders live systems unsuitable for testing intrusive detection algorithms or simulating aggressive attack vectors. Consequently, the development of realistic, low-cost, and high-fidelity virtual testbeds has emerged as a foundational prerequisite for advancing anomaly detection and real-time defense mechanisms in cyber-physical systems (CPS). These synthetic environments provide the essential sandbox required to replicate complex industrial processes, inject diverse cyber threats, and validate intrusion detection systems (IDS) without compromising actual infrastructure integrity. Crucially, these testbeds serve as the data generation engines that enable the multi-modal analysis discussed in subsequent sections, bridging the gap between theoretical models and the holistic cyber-physical data fusion required for robust security.

A primary challenge in constructing these testbeds lies in balancing fidelity with adaptability and cost. Traditional physical testbeds, while accurate, are often prohibitively expensive, difficult to scale, and rigid in their configuration, limiting the scope of reproducible experiments. To address these limitations, recent research has pivoted toward virtualized and container-based architectures that offer high-density exercise environments with significantly reduced resource consumption. For instance, the ICSSIM framework demonstrates the viability of using Docker container technology to build customized virtual ICS security testbeds. By isolating ICS components on private operating system kernels, ICSSIM provides realistic network emulation and supports both software and hardware-in-the-loop simulations, enabling the efficient investigation of various cyber threats while drastically reducing development time and costs [296]. Similarly, evaluations of container-based cyber ranges have confirmed that virtualization can reduce memory, CPU, and storage consumption to less than one-tenth of conventional methods while maintaining a vulnerability reproducibility rate of 99.3% for common exercise scenarios [297]. This shift toward lightweight virtualization ensures that academic institutions and research groups can deploy scalable testbeds that accurately mirror real-world vulnerabilities, providing the necessary volume of data for training advanced AI models.

Beyond mere connectivity simulation, high-fidelity testbeds must accurately model the interplay between cyber events and physical process dynamics to support the data-centric approaches central to modern CPS security. A dataset comprising only network traffic or sensor measurements is often insufficient for training resilient AI agents, as it fails to capture the complex causal relationships inherent in CPS. Therefore, modern frameworks emphasize the inclusion of physical process modeling to generate comprehensive datasets that reflect true system behavior. The TON_IoT initiative exemplifies this multi-layered approach by implementing a testbed across edge, fog, and cloud layers, utilizing Software-Defined Networking (SDN) and Network Function Virtualization

(NFV) to dynamically manage IoT devices and network traffic [298]. This architecture facilitates the collection of heterogeneous data sources, including audit traces from processors, memories, and hard disks, which are essential for evaluating AI-based solutions against both normal operations and authentic ground-truth attack events. Furthermore, specialized testbeds for power systems, such as those utilizing digital real-time simulators coupled with communication network emulators, enable the precise emulation of synchrophasor vulnerabilities and False Data Injection Attacks (FDIA) under realistic grid conditions [299]. Such environments are indispensable for generating the synchronized cyber-physical data streams required to train the invariant-based and causal inference models explored in the following section.

The utility of these testbeds extends to the validation of advanced machine learning models designed for stealthy attack detection. As adversaries increasingly employ deep learning to generate stealthy attacks that mimic normal control signals, defensive algorithms must be trained in environments capable of reproducing such sophisticated behaviors. Research has demonstrated the effectiveness of deep learning-based frameworks in conducting stealthy attacks within simulated ICS environments, thereby providing the necessary adversarial data to train robust anomaly detectors [300]. Additionally, digital twins have emerged as powerful dynamic defense mechanisms, serving not only as real-time mirrors for anomaly detection but also as sandboxed environments for simulating attack scenarios without affecting live operations. The concept of “Trusted Twins” integrates blockchain technology to ensure the integrity of data used in these simulations, thereby enhancing the reliability of security decisions derived from digital twin analyses [301]. Specific implementations, such as the EPICTWIN electric power digital twin, allow researchers to easily modify power system components and configurations to study the implications of various cyber attacks, offering a flexibility that physical testbeds cannot match [302].

Moreover, the design of these testbeds must support the evaluation of resource-constrained defense mechanisms typical of edge and IoT deployments, ensuring that detection algorithms are viable in real-world constraints. Frameworks like SCART (Simulation of Cyber Attacks for Real-Time) extend simulation environments to incorporate reliability-related events and complex security attacks, validating their efficacy on systems such as drone flight controllers that rely on machine learning algorithms [303]. The ability to repeat experiments with programmatic observation is crucial for benchmarking AI tools, as seen in large-scale cyber range challenges that evaluate endpoint security and network intrusion detection systems under varying levels of adversarial covertness [304]. By providing adaptable, low-cost, and high-fidelity platforms, these architecting frameworks enable the safe exploration of the full spectrum of cyber-physical threats, from known exploits to zero-day vulnerabilities. Ultimately, these testbeds lay the empirical groundwork for the transition from siloed detection to the holistic, fused anomaly detection strategies that define the next frontier of critical infrastructure security.

6.2 Cyber-Physical Data Fusion for Holistic Anomaly Detection

6.2 Cyber-Physical Data Fusion for Holistic Anomaly Detection

Building upon the high-fidelity, multi-modal datasets generated by the virtualized testbeds and digital twins described in the preceding section, the focus now shifts to the analytical frameworks required to interpret this complex data. The convergence of operational technology (OT) and information technology (IT) within Critical Infrastructure (CI) and the Internet of Things (IoT) has fundamentally altered the threat landscape, rendering traditional, siloed security measures insufficient. Conventional Intrusion Detection Systems (IDS) have historically relied exclusively

on network traffic analysis, monitoring packet headers, payloads, and flow statistics to identify malicious signatures or statistical deviations. However, as demonstrated by the sophisticated attack simulations enabled by modern testbeds, a purely cyber-centric view fails to capture the intricate interdependencies between digital commands and physical outcomes in Cyber-Physical Systems (CPS). Attackers can increasingly craft exploits that mimic legitimate network behavior while inducing catastrophic physical states, or conversely, mask cyber intrusions as routine physical faults. To address these limitations, there is a critical paradigm shift toward Cyber-Physical Data Fusion, a holistic methodology that integrates network telemetry with real-time physical process variables—data streams made accessible by the architectures discussed previously—to enhance detection accuracy, distinguish between cyber intrusions and physical anomalies, and significantly reduce false positive rates.

The core premise of data fusion in CPS security is that the physical process acts as a ground-truth validator for cyber activities. While network data reveals intent and communication patterns, physical sensor data reflects the actual state and consequences of those communications. Research demonstrates that integrating these disparate data streams yields superior performance compared to uni-modal approaches. For instance, systematic assessments of Industrial Control Systems (ICS) have shown that combining network data with physical process measurements improves the detection and classification of all considered attack types, enabling the simultaneous processing of attacks and physical faults to facilitate holistic root cause identification [305]. This multi-source approach allows defenders to correlate a specific network command with its expected physical response; a discrepancy between the two serves as a robust indicator of compromise that neither domain could detect in isolation.

Several methodological frameworks have emerged to implement this fusion effectively, many of which rely on the causal relationships inherent in the physical processes modeled in realistic testbeds. One prominent approach involves the creation of invariants—rules that govern the physical behavior of a process—which are derived through data-centric machine learning techniques. By monitoring these invariants alongside network flows, systems can detect anomalies that violate fundamental physical laws, such as Ohm’s Law or Kirchhoff’s Law, which are often ignored by standard IT security tools. A data-centric approach to generating these invariants using operational data from smart power grids has proven effective in identifying anomalies that signal cyber-attacks, leveraging the inherent stability of physical causality against data distribution drifts [306]. Furthermore, advances in causal inference allow for the extraction of physical causality from measurements to detect and localize False Data Injection (FDI) attacks. Unlike statistical correlation methods that are susceptible to changes in operating points, causal patterns remain stable, providing a highly interpretable mechanism for pinpointing attacked system nodes [307].

Differentiation between malicious intrusions and benign physical disturbances remains a significant challenge in CPS security, a problem that high-fidelity simulation environments are uniquely positioned to help solve. Traditional anomaly detectors often flag process deviations caused by equipment wear, environmental changes, or operator errors as security incidents, leading to alert fatigue. Data fusion addresses this by contextualizing network events within the physical state space. Methodologies employing Multivariate Statistical Process Control (MSPC) have been expanded to monitor both process-level and controller-level data, successfully distinguishing between process disturbances and actual intrusions to a certain extent [308]. Similarly, hybrid frameworks that fuse information from cyber, security, and physical domains have demonstrated improved accuracy in inferring False Data and Command injection attacks, effectively reducing the ambiguity that plagues single-domain detection systems [309]. The integration of social data, such as human user

reports from mobile devices, alongside physical sensor data in transportation systems further exemplifies the potential of multi-modal fusion to exploit redundancies and enhance detection reliability in socio-technical environments [310].

To handle the complexity and volume of fused data, advanced machine learning architectures are being deployed, often leveraging the topological insights gained from the system models discussed earlier. Graph Neural Networks (GNNs) and attention mechanisms are increasingly utilized to model the topological and temporal dependencies inherent in CPS. For example, frameworks utilizing Graph Attention Networks (GAT) on Measurement Causality Graphs have shown high robustness in localizing attacks by analyzing the topology of physical interactions [307]. Additionally, transformer-based models capable of capturing series, temporal, and spatial dependencies in multi-variate time series have outperformed state-of-the-art baselines in diagnosing fine-grained anomalies in industrial settings [311]. These models often employ self-supervised learning to incorporate the graph structure of the communication network, enhancing interpretability and allowing for hierarchical analysis of anomalies without the need for extensive labeled datasets [312].

The practical deployment of these fused detection systems also necessitates considerations for efficiency and real-time performance, particularly in resource-constrained edge environments where the latency demands are stringent. Hybrid solutions that combine physics-based models with machine learning filters, such as the ResiGate appliance for electrical substations, effectively reduce computational loads by using ML to filter non-suspicious data before applying rigorous physics-based analysis. This architecture ensures high-throughput monitoring capable of detecting attacks with zero error while maintaining the speed required for critical infrastructure protection [313]. Moreover, the use of compact data structures and streaming algorithms enables the processing of high-speed network telemetry directly on programmable devices, facilitating immediate fusion with local sensor readings [314].

Despite these advancements, challenges remain in standardizing data formats, synchronizing timestamps across heterogeneous sensors, and managing the increased attack surface introduced by additional monitoring layers. Future research must focus on developing adaptive fusion engines that can dynamically weigh the reliability of cyber versus physical sources under varying operational conditions. Nevertheless, the consensus across current literature is clear: the path to resilient critical infrastructure lies not in stronger firewalls or more sensitive pressure gauges alone, but in the intelligent, real-time synthesis of both worlds. By treating the cyber and physical domains as a unified entity, holistic anomaly detection systems can achieve the precision and reliability necessary to safeguard the foundational systems of modern society. This fused analytical capability provides the essential intelligence layer that empowers the proactive, dynamic defense mechanisms and real-time mirroring capabilities of Digital Twins, as explored in the following section.

6.3 Digital Twins as Dynamic Defense Mechanisms and Attack Simulators

6.3 Digital Twins as Dynamic Defense Mechanisms and Attack Simulators

Building upon the holistic anomaly detection capabilities established through cyber-physical data fusion, Digital Twins (DTs) represent the next evolutionary step in securing Cyber-Physical Systems (CPS): a transition from passive monitoring to proactive, dynamic defense. While the previous subsection highlighted the integration of network and physical telemetry to validate system states, DTs leverage this multi-source data to maintain a high-fidelity, synchronous virtual replica of the physical infrastructure. In the context of critical infrastructure and the Industrial Internet of Things (IIoT), DTs serve a dual function: they act as real-time mirrors for anomaly detection

through precise prediction error analysis, and they provide isolated, sandboxed environments for simulating sophisticated attack scenarios without endangering physical assets. This dual capability addresses the critical need for resilience in systems where downtime or physical damage carries catastrophic consequences, effectively bridging the gap between theoretical detection models and practical, risk-free validation.

As dynamic defense mechanisms, Digital Twins operate by continuously ingesting the fused sensor and network data discussed previously to update their virtual state. The core of this defensive capability lies in the comparison between the predicted behavior of the digital model and the actual observed behavior of the physical twin. Discrepancies, or prediction errors, serve as early indicators of anomalies, ranging from component failures to malicious intrusions. Research has demonstrated that DT-based frameworks can effectively detect intrusions in industrial control systems by leveraging stacked ensemble classifiers that analyze these deviations in near real-time. For instance, studies have shown that such frameworks can identify command injections, network Denial of Service (DoS) attacks, and measurement modifications with high accuracy, significantly outperforming traditional methods by combining multiple machine learning algorithms to classify intrusions within fractions of a second [315]. Furthermore, the efficacy of these detection mechanisms is enhanced when the DT employs advanced learning paradigms. The introduction of curriculum learning allows the DT to prioritize training on data samples of varying difficulty, optimizing the model’s ability to distinguish between normal operational variance and subtle adversarial manipulations [316].

Beyond simple error reconstruction, the robustness of DT-based defense relies on the quality of the underlying models and their ability to handle uncertainty inherent in real-world data streams. Bayesian frameworks have been proposed to quantify model uncertainty arising from limited or noisy data transfer between the physical and digital entities. By building probabilistic models of the communication system, DTs can perform counterfactual analysis and anomaly detection that accounts for epistemic uncertainty, thereby reducing false positives caused by model exploitation or data scarcity [317]. Additionally, the use of knowledge distillation enables DTs to leverage out-of-domain data and large language models to extract contextual features, enriching the training data volume and improving detection performance in complex environments like train control systems [318]. However, maintaining the synchronization required for such high-fidelity mirroring imposes significant computational demands. Consequently, the deployment of these models often requires architectural optimization; cloud-fog computing architectures have been validated as essential for meeting the strict latency requirements of real-time DTs, ensuring that defense responses are executed before an attack can cause physical harm [319].

The second critical role of Digital Twins is serving as sandboxed environments for attack simulation and security testing, a function that directly complements the resource constraints faced at the edge. Because direct testing of security protocols on live critical infrastructure is often too risky, the DT provides a safe “playground” to evaluate system vulnerabilities and train defense algorithms. This capability is central to the “Shift Left and Shift Right” design paradigm, where security operations are integrated into both the design time and runtime phases of the system lifecycle [320]. In these virtual environments, researchers can implement and observe the effects of various attack vectors, such as false data injection and actuation attacks, to develop and validate resilient control protocols. For example, a twin layer with higher security can be used to decouple defense tasks, allowing for the reconstruction of leader dynamics under DoS attacks and the design of adaptive control schemes that resist unbounded actuation attacks [321].

Moreover, DTs facilitate the development of autonomous defense strategies through reinforcement learning, acting as a training ground for the lightweight models destined for edge deployment. By

simulating countless attack scenarios, DTs enable the training of agents that can learn optimal response policies without exposing the physical system to risk. Applications in autonomous vehicle networks illustrate how DTs can serve as development environments to test AI-aided solutions for localizing signal sources and optimizing network decisions under adversarial conditions [322]. However, the reliability of these simulations depends entirely on the fidelity of the twin itself. Rigorous procedures, such as causal falsification, are necessary to assess the accuracy of DTs using real-world data, ensuring that the twin does not produce incorrect decisions due to unconfounded observational data [323]. To further enhance trust, some frameworks integrate blockchain technology to create immutable audit trails of the safety and security rules evaluated during simulation, ensuring that the data driving the DT remains trustworthy and tamper-proof [301].

Despite their potential, DTs are not immune to threats; they can themselves become targets for adversarial attacks, mirroring the vulnerabilities of the physical systems they emulate. Research has highlighted the susceptibility of DT models to white-box adversarial perturbations, where small changes in input data can fool the classifier and compromise the integrity of the cyber-physical system [324]. Consequently, the design of DTs must incorporate explainability and human-in-the-loop mechanisms to ensure that automated defense decisions are transparent and auditable. Approaches leveraging interpretable machine learning and goal modeling help bridge the gap between black-box predictions and actionable decision-making, fostering operational trust in safety-critical operations [325]. As the technology matures, the convergence of DTs with federated learning and edge computing promises to create decentralized, privacy-preserving defense ecosystems capable of adapting to evolving threat landscapes in real-time [326]. Ultimately, by serving as both high-fidelity mirrors for detection and risk-free sandboxes for simulation, Digital Twins establish a foundational layer of resilience that prepares the ground for the deployment of optimized, resource-constrained AI models discussed in the following section.

6.4 Lightweight and Resource-Constrained AI Models for Edge Deployment

6.4 Lightweight and Resource-Constrained AI Models for Edge Deployment

Building upon the high-fidelity mirroring and risk-free simulation capabilities of Digital Twins established in the previous subsection, the practical realization of resilient Cyber-Physical Systems (CPS) ultimately depends on the deployment of optimized intelligence at the network edge. While Digital Twins provide the strategic layer for anomaly detection and attack validation, the tactical execution of these defenses often resides on Industrial Internet of Things (IIoT) gateways and end-nodes characterized by stringent resource constraints. The proliferation of IIoT has created an environment where critical infrastructure relies heavily on real-time data processing to maintain operational integrity; however, deploying sophisticated deep learning models on these devices is severely hindered by limited memory capacity, restricted computational power, and strict energy budgets. Traditional cloud-centric approaches, which involve offloading raw data for centralized processing, frequently fail to meet the low-latency requirements essential for immediate threat response. Consequently, bridging the gap between the complex models simulated in Digital Twins and the hardware limitations of edge devices requires a transition to highly optimized architectures. This subsection reviews state-of-the-art methodologies designed to enable efficient operation within these constrained environments, focusing on novel architectural designs, memory optimization strategies, and adaptive learning algorithms that preserve detection accuracy without compromising system responsiveness.

A primary challenge in translating robust detection models to the edge is the memory bottleneck

inherent in standard deep neural networks, which often demand significant storage for parameters and intermediate activation maps during inference. To address this, researchers have developed specialized architectures that prioritize separability and dimensionality reduction, effectively distilling the complexity required for accurate detection into lightweight forms. A notable example is the Constrained Twin Variational Auto-Encoder (CTVAE), a novel deep neural network architecture specifically engineered for intrusion detection systems in IoT environments. The CTVAE addresses the heterogeneity of IoT data and the imbalance often found in attack datasets by generating lower-dimensional representation data that is more distinguishable for classifiers. Crucially, compared to state-of-the-art neural networks, the CTVAE requires significantly less memory and computing power, making it uniquely suitable for resource-scarce IoT devices. Empirical evaluations across eleven popular IoT botnet datasets demonstrate that this architecture not only boosts detection accuracy and F-score by approximately 1% compared to existing methods but also achieves an ultra-low running time of under 2 microseconds with a model size of less than 1 MB [327]. Such efficiency is paramount for IIoT gateways that must process continuous data streams without inducing latency that could disrupt industrial processes, thereby actualizing the real-time defense potentials outlined in the Digital Twin framework.

Beyond architectural innovations like CTVAE, optimizing the inference process through advanced compilation and memory management techniques is essential for enabling complex models on edge hardware. The disparity between the rapid growth in tensor computation capabilities and the slower expansion of available memory has created a pressing need for automatic frameworks that minimize memory footprints. Frameworks such as MONEt have been introduced to jointly optimize checkpointing schedules and operator implementations, reducing overall memory requirements by up to 3x for various models with minimal computational overhead. Similarly, approaches like SuperNeurons utilize dynamic GPU memory scheduling, incorporating liveness analysis and cost-aware recomputation to enable the training and inference of networks far beyond the physical DRAM capacity of the device. These techniques allow for the execution of deeper networks, such as ResNet variants with thousands of layers, on hardware that would otherwise be incapable of supporting them [328]. For IIoT applications, where memory is often the limiting factor, such dynamic scheduling and rematerialization strategies are indispensable for maintaining robust anomaly detection capabilities, ensuring that the sophisticated models validated in virtual sandboxes can be effectively deployed on physical assets.

Furthermore, the deployment of Convolutional Neural Networks (CNNs) on microcontrollers and low-power gateways often necessitates the modification of standard convolution operations to fit within tight memory bounds. The Tiny Anomaly Detection (TinyAD) framework exemplifies this approach by leveraging depthwise separable convolutions, which can reduce model sizes by 50% to 90% compared to traditional CNNs. TinyAD integrates complementary strategies such as in-place memory rescheduling and patch-by-patch execution to further slash peak memory consumption by 2 to 5 times with negligible computational overhead. This allows for the direct deployment of trained CNN models on memory-constrained microcontrollers for real-time anomaly detection in industrial time-series data [329]. Additionally, evolutionary algorithms have been employed to solve the bin-packing problem of mapping CNN parameter memories to FPGA on-chip memories, achieving up to a 65% increase in utilization efficiency and ensuring that deep learning accelerators can operate effectively within the physical limits of FPGA resources [330]. These hardware-specific optimizations are critical for embedding the defensive intelligence derived from Digital Twins directly into the fabric of industrial control systems.

In addition to static optimization, the dynamic nature of IIoT environments requires models that

can adapt to changing conditions through online learning and efficient resource scheduling. Deep Reinforcement Learning (DRL) has emerged as a powerful tool for managing these dynamic resources, complementing the simulation-based training paradigms discussed previously. For instance, frameworks utilizing stacked autoencoders for data compression combined with adaptive simulated annealing have been proposed to optimize offloading decisions and resource allocation in large-scale Mobile Edge Computing (MEC) networks. By reducing the state space through unsupervised learning and employing preserved experience replay, these systems can achieve near-optimal performance in minimizing task latency while significantly decreasing computational time [331]. Moreover, the concept of co-inference, where inference tasks are partitioned between energy-constrained mobile devices and edge servers, offers a viable path for handling batch processing capabilities. Algorithms that optimize the split ratio of data offloading based on network bandwidth and device busy factors can reduce total operation time by nearly 50%, thereby enhancing the responsiveness of collaborative autonomous systems [332].

The integration of these lightweight architectures and optimization strategies forms the backbone of resilient edge security, serving as the crucial link between theoretical defense mechanisms and practical implementation. Whether through the use of variational bottlenecks for feature compression [333] or the application of latency-constrained neural architecture search to tailor models for specific hardware targets [334], the field is moving towards highly specialized, efficient solutions. These advancements ensure that IIoT gateways can host sophisticated anomaly detection mechanisms locally, providing immediate defense against cyber-physical threats while adhering to the strict memory and latency constraints that define the edge computing landscape. However, the deployment of these lightweight models introduces new vulnerabilities; their reduced complexity and exposure on edge devices expand the attack surface for sophisticated adversarial threats. As discussed in the following subsection, defending these resource-constrained models against stealthy, adversarial, and heuristic inference attacks requires a paradigm shift from static validation to dynamic, robust training strategies capable of withstanding non-uniform and adaptive threat models.

6.5 Defending Against Stealthy, Adversarial, and Heuristic Inference Attacks

6.5 Defending Against Stealthy, Adversarial, and Heuristic Inference Attacks

While the deployment of lightweight AI models on resource-constrained edge devices, as discussed in the previous subsection, enables real-time anomaly detection, it simultaneously expands the attack surface for sophisticated adversarial threats. In critical infrastructure and Internet of Things (IoT) ecosystems, AI models—particularly those handling time-series forecasting and anomaly detection—are increasingly targeted by stealthy attacks designed to manipulate physical processes without triggering standard alarm thresholds. These attacks often manifest as false data injection (FDI) or carefully crafted adversarial examples that exploit the specific statistical properties of temporal data. Unlike image-based systems where perturbations are bounded by visual perceptibility, adversarial attacks on time-series data must remain indistinguishable from natural process noise to evade heuristic inference mechanisms used by operators and automated monitoring systems. Research has demonstrated that adversarial vulnerability is an inherent weakness of the model architecture itself rather than a limitation of the data modality, meaning that deep networks processing sensor streams are just as susceptible to evasion techniques as those processing images [335]. Consequently, defending critical infrastructure requires a paradigm shift from static validation to dynamic, robust training strategies capable of withstanding non-uniform and adaptive threat models.

One of the most pressing concerns in this domain is the susceptibility of multivariate probabilistic

forecasting models to strategic, sparse modifications. Attackers can induce significant forecasting errors by making imperceptible adjustments to the past observations of a small subset of correlated time series, effectively hijacking the model’s predictive capability while maintaining the appearance of normalcy. To mitigate such sophisticated threats, researchers have developed double-defense mechanisms that combine randomized smoothing techniques with adversarial training algorithms tailored specifically for multivariate scenarios. These approaches involve learning to generate adversarial examples during the training phase to optimize the forecasting model’s robustness against such simulated simulations, thereby creating a more resilient system capable of discerning between genuine process anomalies and malicious injections [336]. However, the efficacy of these defenses is heavily dependent on the threat model employed during training. Traditional defenses often assume a uniform attack distribution, which fails to account for intelligent adversaries who focus their efforts on the most vulnerable data points or specific temporal windows. Addressing this limitation requires the implementation of weighted minimax risk optimization, which adaptively focuses on harder examples that are at a higher risk of misclassification, thus ensuring robustness even under perturbed test data distributions [337].

Furthermore, the nature of perturbations in time-series data necessitates a re-evaluation of distance metrics used to define “indistinguishability.” Standard norms such as L_∞ may not adequately capture the semantic integrity of temporal sequences, leading to defenses that are either too brittle or overly conservative. Recent advancements propose the use of Wasserstein distance to bound perturbations, offering a more physically meaningful measure of similarity for univariate time series. Attacks formulated in the Wasserstein space, such as Wasserstein Projected Gradient Descent (WPGD), have proven highly effective in bypassing conventional Euclidean-based defenses, highlighting the urgent need for certified robustness methods that operate within this alternative metric space [338]. In response, defense strategies must evolve to include smoothing techniques and projection methods specifically designed for the Wasserstein geometry, ensuring that the model remains robust against these geometrically sophisticated attacks.

Beyond specific metric adaptations, the generalizability of adversarial defenses remains a critical hurdle. Models trained to resist one type of attack often fail catastrophically when faced with novel threat vectors or unseen attack modalities. To address this, confidence-calibrated adversarial training (CCAT) has emerged as a promising strategy. By biasing the model towards low-confidence predictions on adversarial examples, CCAT allows the system to reject suspicious inputs rather than forcing a potentially erroneous classification. This approach has been shown to generalize robustness across various threat models, including different L_p norms and corruption types, which is essential for IoT devices operating in unpredictable environments [339]. Additionally, the integration of causal perspectives into adversarial defense offers a pathway to more fundamental robustness. By identifying and intervening on spurious correlations that attackers exploit, causal intervention techniques can help models learn invariant features that are resistant to both known and unknown adversarial manipulations [340].

The challenge is further compounded by the resource constraints typical of edge devices in critical infrastructure, echoing the limitations highlighted in Section 6.4, which often preclude the use of computationally expensive multi-step adversarial training. Efficient defense mechanisms, such as those leveraging transferable adversarial examples or single-step attacks with adaptive step sizes, are crucial for deploying robust models on hardware with limited processing power [341; 342]. Moreover, the phenomenon of “catastrophic overfitting,” where a model suddenly loses all robustness during training, poses a significant risk to the reliability of these systems. Strategies that incorporate historical model states as proxies or utilize self-distillation regularization have shown promise in

stabilizing the training process and preventing such abrupt failures [343]. Ultimately, securing critical infrastructure against stealthy and heuristic inference attacks requires a holistic approach that combines advanced mathematical formulations, efficient algorithmic implementations, and a deep understanding of the causal structures underlying the physical processes being monitored. Only through such comprehensive defense-in-depth strategies can we ensure the resilience of AI-driven systems against the evolving landscape of adversarial threats, thereby establishing a secure foundation before addressing the lower-level hardware integrity issues discussed in the following section on Hardware Trojans.

6.6 Hardware Trojan Detection and Supply Chain Integrity in CPS

6.6 Hardware Trojan Detection and Supply Chain Integrity in CPS

While Section 6.5 established the necessity of robust algorithmic defenses against stealthy, adversarial inference attacks on edge devices, the reliability of these software-centric strategies is fundamentally contingent upon the integrity of the underlying hardware. In Cyber-Physical Systems (CPS), the globalization of the semiconductor supply chain introduces a critical vulnerability: the risk of malicious hardware modifications known as Hardware Trojans (HTs). As CPS deployments expand into critical infrastructure—from smart grids to autonomous vehicles—the reliance on third-party intellectual property (IP) cores and outsourced fabrication creates entry points where untrusted actors can insert stealthy malicious circuitry. These Trojans are designed to remain dormant during standard testing phases, activating only under specific conditions to leak sensitive information, disrupt control logic, or cause physical harm, thereby bypassing the advanced heuristic and adversarial defenses discussed previously. Consequently, ensuring supply chain integrity requires a multi-layered defense strategy that integrates advanced detection methodologies, including information flow tracking, sophisticated side-channel analysis, and resilient architectural designs, to secure the foundation upon which AI models operate.

One of the most promising approaches for safeguarding CPS against hardware Trojans is Information Flow Tracking (IFT), which monitors the propagation of data through a system to prevent unauthorized information leakage or integrity violations. Traditional IFT methods often struggle to balance precision with hardware overhead; fine-grained models offer high accuracy but are resource-intensive, while coarse-grained models lack the necessary precision for complex CPS environments. To address this, recent research has proposed integrated IFT techniques that combine hardware-based tracking with gate-level analysis. For instance, a multi-level granularity IFT model has been developed for RISC-V architectures, which translates instruction-level security extensions into data-level tracking without incurring excessive area overhead [344]. This approach allows for the precise identification of security-critical data flows, effectively mitigating the risk of Trojans that attempt to exfiltrate data through covert channels. Furthermore, dynamic information flow tracking (DIFT) has been adapted for loosely coupled accelerators within heterogeneous CPS architectures. The PAGURUS methodology, for example, introduces a flexible shell circuit that enables low-overhead DIFT on accelerators without requiring modifications to the core implementation, thereby securing the entire system against spurious data flows [345]. Despite these advances, current IFT methods still face limitations when applied to black-box design models, necessitating further research to maximize their efficacy in real-world attack scenarios [346].

Complementing IFT, side-channel analysis (SCA) remains a cornerstone for detecting the physical signatures of Hardware Trojans. Since Trojans inevitably alter the power consumption, electromagnetic emissions, or timing characteristics of a circuit, SCA provides a non-intrusive means of

identification. However, the effectiveness of SCA is often hindered by process variations and the subtlety of modern Trojans. To overcome the limitations of single-parameter analysis, researchers have advocated for multiple-parameter side-channel analysis, which correlates data from power, temperature, and electromagnetic domains to achieve higher detection coverage [347]. The sophistication of these attacks and defenses is further highlighted by the emergence of evasive Trojans designed specifically to fool machine learning-based detectors. Adversarial power trace techniques have demonstrated that attackers can embed noise-generating circuitry directly into the chip to mask the Trojan’s presence, achieving 100% evasion efficiency against standard ML detectors [348]. This arms race has led to the development of robust detection frameworks capable of identifying Trojans even in finalized layouts where the adversary has limited access, utilizing engineering change order (ECO) flows to simulate and detect blind insertions [349]. Additionally, innovative approaches such as Programmable Sensor Arrays (PSA) have been introduced to enable run-time, golden-model-free detection. By integrating re-programmable magnetic field sensors directly onto the chip, PSAs can localize and identify Trojans with millisecond latency, overcoming the impracticality of external probing in deployed CPS [350].

Beyond detection, ensuring supply chain integrity in CPS often necessitates architectural innovations that limit the impact of potential compromises. Secure dual-MCU architectures and redundancy mechanisms are increasingly employed to create trust at the device level. In these configurations, multiple microcontrollers cross-verify each other’s operations, ensuring that a single compromised unit cannot dictate system behavior. This concept extends to hardware/software co-monitoring, where a formal device model shadows the real device execution to detect deviations indicative of malicious exploits or transient failures [351]. For resource-constrained embedded devices typical in IoT-enabled CPS, active control flow attestation (ACFA) offers a mechanism for continuous monitoring and guaranteed remediation. Unlike traditional remote attestation, ACFA does not require binary instrumentation and can pinpoint the origin of an exploit, enabling active roots-of-trust to heal the system upon detection of a compromise [352]. Moreover, the threat of Trojans with zero power and area footprints, such as “TrojanZero,” which salvages costs by removing expendable gates to hide malicious logic, underscores the need for switching activity-aware design verification [353].

The landscape of hardware security in CPS is further complicated by the diversity of attack vectors, ranging from frequency modulation Trojans that evade unused-circuit identification [354] to those inserted via foundry-side ECO attacks [355]. Consequently, a holistic security framework must integrate pre-silicon verification, run-time monitoring, and resilient architectural patterns. The convergence of these techniques—precise information flow tracking, multi-parameter side-channel analysis, and active architectural defenses—forms the bedrock of a trustworthy CPS supply chain. As the complexity of integrated circuits continues to grow, the collaboration between hardware designers, security researchers, and regulatory bodies will be essential to standardize these detection methods and ensure the long-term integrity of critical infrastructure. Establishing this hardware-level trust is a prerequisite for the next layer of defense: ensuring that the intelligent software monitoring these systems is interpretable and trustworthy to human operators, a challenge addressed in the following section on Explainable AI and Human-in-the-Loop frameworks.

6.7 Explainable AI and Human-in-the-Loop for Operational Trust

6.7 Explainable AI and Human-in-the-Loop for Operational Trust

While Section 6.6 established the necessity of hardware-level integrity and multi-layered defenses against physical tampering in Cyber-Physical Systems (CPS), the efficacy of these defenses increasingly relies on the software intelligence tasked with monitoring and responding to threats. The integration of Artificial Intelligence (AI) into Intrusion Detection Systems (IDS) for Critical Infrastructure (CI) and the Industrial Internet of Things (IIoT) has significantly enhanced the capability to detect sophisticated cyber-physical threats in real time. However, the widespread adoption of deep learning-based IDS is frequently hindered by the “black-box” nature of these models, where the internal decision-making logic remains opaque to human operators. In safety-critical environments such as power grids, water treatment facilities, and manufacturing plants—where the consequences of false positives (unnecessary shutdowns) or false negatives (catastrophic physical damage) are severe—Cyber Security Operation Center (CSoC) analysts often hesitate to rely on AI-generated alerts that lack justification. This opacity creates a significant barrier to effective human-AI collaboration, necessitating the deployment of Explainable AI (XAI) within a Human-in-the-Loop (HITL) framework to bridge the gap between high-accuracy predictions and actionable, trustworthy decision-making.

The primary function of XAI in this context is to transform abstract model outputs into interpretable insights that align with the cognitive workflows of human analysts. Traditional IDS utilizing deep learning techniques often fail to provide the rationale behind a detection, leaving analysts unable to verify the legitimacy of a threat or refine their response strategies. This challenge is particularly acute in IIoT security, where numerical data and complex temporal dependencies dominate, making it difficult to design explanations that are both fast and accurate. Recent advancements propose universal XAI models, such as Transparency Relying Upon Statistical Theory (TRUST), which are specifically designed for numerical applications common in IIoT. By modeling the statistical behavior of AI outputs and utilizing factor analysis to identify influential latent variables, such approaches can provide model-agnostic explanations with high success rates, outperforming traditional methods like LIME in terms of speed and performance [356]. These interpretable alerts allow operators to understand not just *that* an anomaly was detected, but *why* specific sensor readings or network flows triggered the alarm, thereby facilitating rapid and informed intervention.

Furthermore, the integration of XAI must move beyond simple feature attribution to support diverse stakeholder needs within the operational chain. Research indicates that different stakeholders, ranging from development teams to domain experts and decision-makers, possess distinct explanation requirements. A one-size-fits-all approach to explainability often fails to meet the specific contextual needs of these groups, potentially leading to mistrust or misinterpretation of the AI’s capabilities [357]. For instance, while a data scientist may require detailed gradient-based visualizations to debug a model, a plant operator needs concise, action-oriented insights that highlight the physical implications of a cyber intrusion. Addressing this disparity requires the design of XAI systems that can tailor the granularity and format of explanations to the specific role and expertise of the human user, ensuring that the provided information is both understandable and actionable.

To further enhance operational trust and reduce the cognitive load on analysts, modern architectures are increasingly incorporating Large Language Models (LLMs) alongside traditional XAI techniques to create more intuitive human-machine interfaces. Systems such as HuntGPT demonstrate the potential of integrating Random Forest classifiers with XAI frameworks like SHAP and

LIME, coupled with LLMs to translate technical anomaly detections into natural language narratives. This synthesis allows incident responders to interact with the IDS through conversational agents, receiving robust and explainable alerts in a format that significantly enhances user understanding and interactive experience [358]. By converting complex statistical anomalies into coherent stories, these systems accelerate the mean time to respond (MTTR), which is critical in preventing the escalation of attacks in critical infrastructure.

However, the reliance on post-hoc explanation methods introduces its own set of challenges regarding faithfulness and reliability. There is a growing concern that some XAI techniques may “paint the black box white,” offering a superficial layer of transparency that does not truly reflect the model’s internal logic, a phenomenon known as the “white-box paradox.” Studies involving medical and industrial diagnostics have shown that users may incorrectly defer to automated decisions if the explanations appear plausible but are actually misleading, leading to technology dominance where human oversight is diminished rather than enhanced [359]. Therefore, a robust HITL framework must include mechanisms for validating the quality of explanations themselves. This involves employing rigorous evaluation metrics and potentially utilizing backdoor triggers as ground truth to test whether saliency maps correctly identify the features responsible for a decision, ensuring that the explanations provided to operators are genuinely faithful to the model’s behavior [360].

Moreover, the design of XAI for critical infrastructure must be grounded in an understanding of how human experts naturally explain their own decisions in high-stakes visual detection tasks. Research into building damage assessment from satellite imagery has revealed that humans utilize specific strategies, such as comparing current states to baseline expectations and focusing on distinct structural features, which current XAI methods often ignore. By characterizing these human explanation strategies, future X-IDS (Explainable Intrusion Detection Systems) can be designed to mimic human reasoning patterns, thereby fostering a more natural and effective collaboration between human analysts and AI agents [361]. Additionally, generic architectures for X-IDS must explicitly consider the human-in-the-loop, defining clear roles for model users, designers, and adversaries to ensure that explanations serve their intended purpose without exposing vulnerabilities to malicious actors who might exploit explainability features for model extraction or evasion attacks [362].

In conclusion, achieving operational trust in AI-driven anomaly detection for critical infrastructure requires a holistic approach that seamlessly integrates advanced XAI techniques with human-centric design principles. It is not sufficient to merely generate explanations; these explanations must be accurate, tailored to the stakeholder, and validated against the reality of the physical processes they monitor. By moving towards systems that combine statistical rigor, natural language interaction, and a deep understanding of human cognitive processes, the industry can overcome the barriers of the black-box paradigm. This evolution enables CSoC analysts to transition from passive recipients of alerts to active partners in a collaborative defense ecosystem. As we look toward the future developments discussed in Section 6.8, this foundation of trusted human-AI collaboration becomes the prerequisite for adopting emerging decentralized paradigms like Federated Learning and Blockchain, where maintaining trust without centralized authority is paramount.

6.8 Emerging Frontiers: Federated Learning, Blockchain, and Quantum Resilience in CPS

6.8 Emerging Frontiers: Federated Learning, Blockchain, and Quantum Resilience in CPS

Building upon the foundation of trusted human-AI collaboration established in Section 6.7, where Explainable AI (XAI) and Human-in-the-Loop (HITL) frameworks were identified as prerequisites for operational trust, the next evolutionary step involves decentralizing this trust to operate without centralized authority. As Cyber-Physical Systems (CPS) and Industrial Internet of Things (IIoT) ecosystems expand, the traditional reliance on centralized cloud architectures for anomaly detection and model training has proven insufficient due to latency constraints, bandwidth limitations, and acute privacy vulnerabilities. The emerging frontier for resilient CPS defense lies in the synergistic integration of Federated Learning (FL), Blockchain technology, and post-quantum cryptographic mechanisms. This triad forms a robust architectural foundation capable of supporting privacy-preserving collaborative defense, ensuring immutable audit trails, and safeguarding against next-generation quantum threats in decentralized IoT networks, effectively scaling the principles of transparency and accountability from the individual analyst to the entire network.

Federated Learning has emerged as a cornerstone paradigm for enabling collaborative machine learning across distributed Edge and IoT devices without the need to centralize sensitive raw data. In the context of CPS, where data often includes proprietary industrial processes or confidential patient health information, FL allows local models to be trained on-device, with only model updates (gradients or weights) shared for global aggregation. However, standard FL architectures often suffer from single points of failure at the aggregation server and remain vulnerable to poisoning attacks from malicious clients. To address these structural weaknesses and extend the trust models discussed previously, recent research advocates for the integration of blockchain technology to decentralize the aggregation process and enforce trust. For instance, architectures leveraging blockchain-based FL have been proposed to create secure, transparent, and tamper-proof environments for model training in healthcare and industrial settings [363]. By utilizing smart contracts to automate the aggregation logic and validate model updates, these systems eliminate the need for a trusted central authority, thereby enhancing the reliability of intrusion detection systems in heterogeneous IoT networks [364].

The fusion of blockchain and FL, often termed “Blockchained Federated Learning” (BlockFL), introduces critical capabilities for provenance and accountability in CPS, directly addressing the need for verifiable decision-making highlighted in the context of XAI. In safety-critical domains such as autonomous driving or smart grids, the ability to audit the lineage of model updates is paramount. Blockchain provides an immutable ledger that records every transaction, including local model contributions and global aggregation events, ensuring that any attempt to tamper with the learning process is detectable. This is particularly relevant for defending against Byzantine attacks and model corruption, where malicious entities attempt to degrade the global model’s performance. Studies have demonstrated that incorporating peer-to-peer voting mechanisms and reward-and-slash incentives via smart contracts can effectively deter malicious behaviors and ensure the robustness of the collaborative learning process [365]. Furthermore, the use of consortium blockchains, such as Hyperledger Fabric, allows for permissioned access control, ensuring that only verified entities within a specific industrial consortium can participate in the training workflow, thus maintaining high standards of security and privacy [366].

Despite these advancements, the advent of quantum computing poses an existential threat to the cryptographic primitives underpinning current blockchain and FL security protocols. Traditional public-key cryptosystems, such as RSA and ECC, which secure communication channels and digital signatures in blockchain networks, are vulnerable to Shor’s algorithm on sufficiently powerful quantum computers. Consequently, future-ready CPS architectures must incorporate post-quantum cryptography (PQC) to ensure long-term resilience, preserving the integrity of the immutable audit trails established by blockchain. Recent proposals have begun to address this by integrating post-quantum secure features into blockchained FL frameworks, specifically designed for mobile edge computing environments where connected autonomous vehicles operate. These frameworks employ lattice-based or hash-based cryptographic schemes to verify client identities and secure model updates against both classical and quantum adversaries [367]. The transition to quantum-resistant algorithms is not merely an upgrade but a necessity for maintaining the integrity of immutable audit trails in CPS, where the longevity of infrastructure often exceeds the expected timeline for quantum supremacy.

Moreover, the scalability of these integrated systems remains a significant challenge, particularly when deploying on resource-constrained IoT devices. Innovative solutions such as hierarchical blockchain structures and sharding mechanisms have been developed to reduce communication overhead and computational latency. For example, two-layer blockchain-driven systems that utilize Directed Acyclic Graphs (DAGs) for the mainchain and sub-chains for local clusters have shown significant improvements in training efficiency and robustness against abnormal models [368]. Additionally, the application of Graph Neural Networks (GNNs) within these architectures offers a sophisticated method for analyzing the topological relationships between devices, enabling the detection of anomalous model updates that might indicate a coordinated attack, thus adding another layer of intelligent defense to the decentralized framework [369].

Looking toward the future, the concept of the “Internet of Federated Things” (IoFT) envisions a fully decentralized ecosystem where the cloud is substituted by the crowd, and intelligence is embedded directly at the edge. This vision relies heavily on the seamless interoperability of FL, blockchain, and quantum-resilient protocols to manage dynamic network topologies and diverse data distributions. Research indicates that decentralized approaches, such as Swarm Learning, which utilize permissioned blockchains to dynamically elect leaders and manage consensus, are particularly well-suited for handling biased or polluted datasets common in real-world CPS deployments [370]. As CPS environments become increasingly complex, the development of self-healing architectures that can autonomously detect, isolate, and recover from security breaches using these combined technologies will be essential. The convergence of these fields promises a new era of trustworthy AI, where privacy, security, and resilience are intrinsic properties of the system architecture rather than add-on features, ultimately enabling safe and reliable operation of critical infrastructure in an uncertain threat landscape [371].

7. Architectural Enablers for Trust: Blockchain, Zero Trust, and Explainable AI

7.1 Blockchain-Enabled Provenance and Immutable Audit Trails for AI Assets

7.1 Blockchain-Enabled Provenance and Immutable Audit Trails for AI Assets

The increasing complexity of the artificial intelligence software supply chain, characterized by intricate data pipelines, pre-trained model artifacts, and diverse third-party dependencies, necessitates robust mechanisms for ensuring trust and accountability. Traditional centralized databases are often insufficient for guaranteeing the integrity of AI assets due to their susceptibility to tamper-

ing, single points of failure, and a lack of transparent auditability. Distributed Ledger Technology (DLT), particularly blockchain, has emerged as a foundational architectural enabler to address these challenges by providing immutable, decentralized, and transparent records of data lineage, model training history, and component origins. By leveraging the inherent properties of immutability and cryptographic verification, blockchain frameworks create tamper-proof audit trails that are essential for securing the AI lifecycle against manipulation and fraud, establishing the factual baseline upon which further privacy-preserving verifications can be built.

A primary application of blockchain in this domain is the verification of data integrity throughout the supply chain. Data tampering represents a severe threat in industrial applications, potentially leading to inaccurate model outputs or corporate security crises. To mitigate this, automated and decentralized integrity verification models have been proposed to assist stakeholders in validating data authenticity without relying on expensive or potentially untrustworthy third-party auditors. For instance, frameworks utilizing carefully designed smart contracts and distributed file systems can store integrity verification attributes, while the blockchain enhances the authenticity of data certificates [372]. Such systems ensure that the data used to train AI models remains unaltered from its source, providing a verifiable chain of custody that is critical for regulatory compliance and trust.

Beyond data integrity, blockchain facilitates trusted data workflows across independent entities, which is particularly relevant in complex ecosystems like smart manufacturing and IoT where AI models are increasingly deployed. Architectural patterns integrating DLT can automate trust services, including digital identity and auditable transactions, creating a secure backbone for data flows. Solutions like SmartQC demonstrate how immutable ledgers can ensure data integrity while smart contracts automate business workflows, making processes more reliable and robust [373]. This capability is extended to supply chain scenarios involving electronic components, where permissioned ledgers such as IOTA enable the orchestration of product provenance stories. By using masked authenticated messaging channels, these frameworks allow supply chain partners to track the complete journey of a product while maintaining confidentiality and preventing data silos [374].

The concept of provenance is further critical in securing specialized hardware and network equipment, such as OPEN-RAN, where the disaggregated nature of the supply chain introduces significant risks. Blockchain-based approaches can secure equipment through its entire lifecycle by combining firmware authentication codes with permissioned ledgers and node validators. This creates a tamper-resistant ecosystem capable of tracking provenance from manufacturing to integration, ensuring that AI-driven network components have not been compromised [375]. Similarly, in the context of vaccine distribution and other critical logistics, blockchain systems have been implemented to assure transparent tracing and eliminate identity thefts, using smart contracts to monitor handling rules and track administration [376]. These examples illustrate the versatility of blockchain in establishing immutable records for physical and digital assets alike, a prerequisite for trustworthy AI supply chains.

While blockchain excels at recording immutable histories, the transparency of public ledgers can sometimes conflict with the need for confidentiality regarding proprietary AI models and sensitive training data. To resolve this tension, hybrid architectures combining blockchains with Trusted Execution Environments (TEEs) or off-chain storage have been developed. For example, Private Data Objects (PDOs) integrate distributed ledgers with Intel SGX to run smart contracts over private data, preserving confidentiality while maintaining execution integrity and providing an authoritative record of transactions [377]. Similarly, patterns for confidential contract enforcement propose storing execution states on-chain while keeping business logic off-chain, thereby guaran-

teering pseudonymity and data confidentiality while ensuring an audit trail can be reconstituted to identify misbehavior [378]. These hybrid approaches serve as a crucial bridge, leveraging the ledger for accountability while deferring the protection of sensitive intellectual property to secure enclaves—a synergy that will be explored in greater depth through cryptographic proofs and hardware isolation in the subsequent section.

Furthermore, the scalability and efficiency of these provenance systems are enhanced by advanced consensus mechanisms and ledger structures. Federated ledger frameworks, such as Fed-DDM, address the limitations of direct blockchain integration in large-scale IoT environments by organizing participants into permissioned domains with private intra-ledgers, federated by a public inter-ledger. This hierarchical approach ensures security and auditability for cross-domain operations without exposing sensitive privacy information, a feature highly applicable to decentralized AI marketplaces [379]. Additionally, the use of Directed Acyclic Graph (DAG) architectures offers lightweight alternatives suitable for resource-constrained devices, enabling privacy-preserving machine learning and secure data monetization in smart mobility applications [380].

In the realm of intellectual property protection for AI models, blockchain enables novel mechanisms for asserting ownership and preventing piracy. Techniques such as software watermarking for smart contracts can be adapted to embed verifiable marks into AI model artifacts, ensuring that origins can be traced even if the model is copied or modified [381]. Moreover, the ability to create individual accountability within permissioned ledgers ensures that misbehaving nodes or malicious actors in the supply chain can be identified and held responsible, providing strong disincentives against tampering with AI assets [382]. By synthesizing these technologies—immutable ledgers, smart contracts, privacy-preserving computation, and scalable consensus architectures—the AI community can establish a resilient foundation for provenance. This foundation not only secures the supply chain against adversarial threats but also fosters a culture of transparency and accountability essential for the widespread adoption of AI in critical sectors, setting the stage for advanced verification methods that can validate these records without compromising the secrecy of the underlying assets.

7.2 Privacy-Preserving Verification via Zero-Knowledge Proofs and Secure Enclaves

7.2 Privacy-Preserving Verification via Zero-Knowledge Proofs and Secure Enclaves

While the immutable ledgers and decentralized provenance systems discussed in the previous section establish a factual baseline for AI asset history, they introduce a critical paradox: the transparency required for auditability often conflicts with the necessity of protecting proprietary intellectual property and sensitive training data. Public or even permissioned blockchains record transactions and hashes openly, which can inadvertently expose metadata or allow inference attacks on model weights and datasets. To resolve this tension between verifiable accountability and data confidentiality, the AI supply chain relies on advanced cryptographic and hardware-based mechanisms, specifically Zero-Knowledge Proofs (ZKPs) and Secure Enclaves (TEEs). These technologies serve as the essential bridge, enabling stakeholders to validate the integrity, safety, and compliance of AI components without revealing the underlying secrets, thus operationalizing the “confidential contract enforcement” patterns introduced earlier.

Zero-Knowledge Proofs act as the mathematical cornerstone for this privacy-preserving verification, allowing a prover to convince a verifier of a statement’s validity without disclosing any information beyond the truth of that statement. In the context of the AI supply chain, this capability transforms how trust is established; model owners can cryptographically prove that a specific model was trained on a certified dataset or adheres to safety constraints without exposing the data or the

model weights themselves. The theoretical foundation of ZKPs relies on intractable problems, and recent advancements have expanded the scope of these proofs to include complex computational statements relevant to machine learning [383]. For instance, the application of ZKPs extends to verifying the execution of programs on potentially compromised devices, ensuring that firmware and data processing maintain integrity even when the underlying hardware cannot be fully trusted [384].

A primary application of ZKPs within this framework is the auditability of model training and inference, addressing the limitations of traditional auditing which requires full access to models and data. Protocols like ZkAudit demonstrate that it is possible to perform trustless audits where model providers publish cryptographic commitments of their datasets and weights alongside a zero-knowledge proof certifying that the published commitments correspond to a correctly trained model [385]. This approach enables third parties to verify properties such as copyright compliance, the absence of specific biased data, or the correctness of counterfactual outcomes without ever seeing the raw data or the model parameters. Furthermore, the integration of ZKPs with machine learning, often termed Zero-Knowledge Machine Learning (ZKML), facilitates the independent validation of AI-generated content. Implementations such as snarkGPT illustrate how transformers can be audited for fairness and accuracy while preserving model privacy, thus addressing critical concerns in high-stakes domains like medicine and law [386].

Beyond model auditing, ZKPs are instrumental in securing the identity and credential management aspects of the supply chain, a prerequisite for the dynamic access control models discussed in the subsequent section. Attribute-based encryption and verifiable credentials allow stakeholders to prove they possess certain qualifications or access rights without revealing their full identity or unnecessary attributes. Systems like ZKclaims utilize Succinct Non-interactive Arguments of Knowledge (SNARKs) to enable users to present attribute-based credentials in a privacy-preserving manner, ensuring that credential contents remain hidden from verifiers [387]. Similarly, frameworks such as zkFaith provide a mechanism for issuing verified zero-knowledge identities, allowing individuals to prove eligibility for services without exposing personal documents, leveraging vector commitments to update credentials efficiently [388]. These mechanisms are vital for managing access control in decentralized AI ecosystems where participants may not trust one another, laying the groundwork for the Zero Trust principles explored next.

The scalability and efficiency of these proofs are paramount for their adoption in real-time supply chain operations, particularly given the computational intensity of AI workloads. Recent developments have focused on optimizing the computational overhead associated with generating proofs for complex circuits. For example, the LUMEN protocol introduces novel algorithms to generate transparent zk-SNARKs that improve security by eliminating the need for trusted setup ceremonies while maintaining efficiency comparable to non-transparent variants [389]. Additionally, libraries like zkSk provide composable tools for defining sigma protocols, simplifying the implementation of privacy-preserving systems for developers who may not be cryptography experts [390]. Despite these advances, challenges remain regarding the computational cost of proving large-scale neural network operations, necessitating further research into multi-party proof generation schemes that can delegate computation without compromising security [391].

Complementing these cryptographic approaches, Secure Enclaves, or Trusted Execution Environments (TEEs), offer hardware-based isolation for sensitive computations, echoing the hybrid architectures mentioned in the preceding section. While ZKPs provide mathematical guarantees, TEEs rely on the physical security of the processor to protect data in use. In data on-chaining workflows for IoT and AI, TEEs can be employed to perform trustworthy pre-processing of sensor data

off-chain before it is committed to a blockchain, ensuring end-to-end integrity where traditional cryptographic signatures might fail due to intermediate manipulation [392]. The convergence of ZKPs and TEEs offers a robust defense-in-depth strategy; TEEs can handle heavy computation and data privacy, while ZKPs can verify the integrity of the TEE’s output or the correctness of the enclave’s code, creating a resilient barrier against both software and hardware-level attacks.

Furthermore, the integration of these technologies supports regulatory compliance without sacrificing privacy, a key requirement for the adoption of AI in regulated sectors. Systems like zkTax demonstrate how zero-knowledge disclosures can allow organizations to prove compliance with financial regulations or tax obligations without revealing sensitive underlying financial data [393]. In the realm of federated learning, where data remains localized, ZKPs ensure the authenticity of local model updates and prevent poisoning attacks without requiring the central server to access raw data [394]. As the AI supply chain becomes increasingly decentralized, the synergy between Zero-Knowledge Proofs and Secure Enclaves defines the standard for verifiable, private, and trustworthy ecosystems. This foundation of cryptographic and hardware-backed verification enables the transition to broader architectural paradigms, specifically the Zero Trust Architectures adapted for decentralized AI and IoT ecosystems discussed in the following section, where continuous verification replaces static trust assumptions.

7.3 Zero Trust Architectures Adapted for Decentralized AI and IoT Ecosystems

7.3 Zero Trust Architectures Adapted for Decentralized AI and IoT Ecosystems

While the cryptographic and hardware-based mechanisms discussed in the previous section provide robust methods for verifying specific computations and protecting data privacy, they operate most effectively within a broader architectural framework that fundamentally redefines how trust is established across the supply chain. The convergence of Artificial Intelligence (AI), the Internet of Things (IoT), and distributed ledger technologies has rendered traditional perimeter-based defense models obsolete, creating an environment where endpoints are heterogeneous, resource-constrained, and often untrusted. In such highly dynamic ecosystems, the assumption of implicit trust within a network boundary constitutes a critical vulnerability. Consequently, the adaptation of Zero Trust Architecture (ZTA) principles has become imperative, shifting the paradigm from static perimeter defense to the continuous verification of every access request and transaction, regardless of its origin. This shift is particularly vital given that adversaries can leverage compromised but authenticated sessions to perform malicious activities, effectively turning endpoints into the “Achilles heel” of otherwise robust security frameworks [395].

In decentralized AI and IoT ecosystems, the implementation of Zero Trust requires a seamless synergy between continuous monitoring, dynamic risk assessment, and immutable record-keeping. Traditional Role-Based Access Control (RBAC) models, which grant access based on static roles, are insufficient against insider threats and compromised credentials. Instead, a Zero Trust approach mandates the continuous evaluation of trust parameters for each resource access request, utilizing strict protocols that adapt to real-time threat intelligence [396]. This is especially challenging in IoT networks characterized by massive device proliferation and moderate security levels, where classical centralized architectures often cloak vulnerabilities that hackers exploit for spying or taking control of the network [397].

To address these challenges, modern adaptations of ZTA integrate AI-driven anomaly detection with blockchain technology to create a decentralized trust layer that complements the privacy-preserving proofs mentioned earlier. AI and machine learning algorithms play a pivotal role in this

architecture by analyzing patterns, detecting anomalies, and predicting threats in real-time, thereby enhancing the decision-making processes essential for Zero Trust enforcement [398]. For instance, in 5G and 6G networks, an Intelligent Zero Trust Architecture (i-ZTA) leverages AI to monitor the security state of network assets and evaluate the risk of individual access requests dynamically. This approach utilizes service-based architectures and open radio access networks to collect necessary data for machine learning components, ensuring security in untrusted environments [57].

However, AI models alone are susceptible to adversarial attacks and data poisoning, particularly during the training phase where observations are often assumed to be benign. To mitigate this, blockchain technology serves as a foundational enabler for trustworthy AI operations within a Zero Trust framework, acting as the immutable ledger for the trust scores generated by the system. By utilizing blockchain as an immutable database for storing user requests and detection logs, the system ensures that behavioral data cannot be altered or deleted by malicious software, thus preserving the integrity of the trust evaluation process [58]. Furthermore, blockchain facilitates collaborative intrusion detection systems where nodes share detection rules and intrusion alarms without relying on a central authority. In such decentralized Collaborative Intrusion Detection Systems (CIDS), the blockchain offloads trust computation, ensuring that the trustworthiness of each node is continuously evaluated rather than assumed [399].

The integration of these technologies also extends to lightweight access control protocols designed for resource-constrained IoT sensors, bridging the gap between heavy cryptographic proofs and practical deployment. Novel protocols empower data owners to define intricate access policies while utilizing efficient cryptographic primitives to eliminate reliance on trusted third parties. By employing mechanisms such as Merkle Trees for lightweight, ongoing trust measurement, these systems ensure efficient trust assessment by sensor coordinators while the cloud conducts thorough evaluations for network entities [400]. Additionally, the use of Zero-Knowledge Proofs (ZKPs) within these architectures allows for the verification of device integrity and data processing without revealing sensitive information, establishing a trust layer that facilitates secure, autonomous communication between devices that do not inherently trust each other [384]. This creates a logical bridge where ZTA provides the operational policy, while ZKPs and TEEs (from the previous section) provide the cryptographic and hardware enforcement mechanisms.

Despite the clear advantages, implementing Zero Trust in decentralized AI and IoT ecosystems presents significant challenges, including cultural barriers, technical complexities, and the need for seamless interoperability between diverse technologies. The transition from legacy security policies to ZT frameworks requires a comprehensive understanding of the differences in policy enforcement and the potential impacts of emerging technologies like quantum computing [398]. Moreover, while blockchain provides immutability and decentralization, it does not guarantee the trustworthiness of data at the origin. Therefore, layered architectures are proposed to evaluate the trustworthiness of sensor observations at the data layer and adapt block verification at the blockchain layer through data trust and gateway reputation modules [401]. These reputation modules serve as a direct precursor to the decentralized governance models discussed in the following section, where reputation becomes a quantifiable metric for access control and incentive alignment.

Ultimately, the convergence of Zero Trust principles, AI-driven analytics, and blockchain immutability creates a resilient architectural pattern capable of withstanding advanced persistent threats and insider attacks. By challenging conventional trust models and emphasizing continuous verification, this integrated approach empowers organizations to navigate the ever-evolving cybersecurity realm with resilience and adaptability [402]. As the complexity of threats increases, particularly in smart cities and industrial IoT sectors, the adoption of such intelligent, decentralized Zero Trust

architectures becomes not just an option, but a necessity for securing the digital future [403]. This architectural foundation sets the stage for the next evolution in supply chain security: the formalization of these dynamic trust evaluations into decentralized governance and reputation mechanisms that autonomously manage participant integrity.

7.4 Decentralized Governance and Reputation Mechanisms for Supply Chain Integrity

7.4 Decentralized Governance and Reputation Mechanisms for Supply Chain Integrity

Building upon the architectural foundation of Zero Trust and the immutable ledgers discussed in the previous section, the operationalization of trust in decentralized AI and IoT ecosystems requires a shift from static policy enforcement to dynamic, autonomous governance. While Zero Trust Architecture (ZTA) establishes the principle of continuous verification, and blockchain provides the tamper-proof for trust scores, a robust mechanism is needed to quantify these scores, align participant incentives, and autonomously manage access rights across heterogeneous networks. Traditional centralized governance models struggle to scale within consortium-based supply chains where multiple organizations collaborate without a single point of absolute trust. Consequently, emerging paradigms are shifting toward decentralized governance frameworks that leverage token-based incentives and dynamic reputation systems. These mechanisms transform the abstract concept of “trust” into a quantifiable metric, ensuring that only verified and trustworthy entities can contribute to or consume resources, thereby directly addressing the data origin trustworthiness challenges highlighted at the end of the preceding section.

At the core of this decentralized governance lies the concept of reputation as a fluid, quantifiable metric for trust. In complex supply chain environments, where participants act as validators, data providers, or service operators, establishing a robust reputation system is critical for mitigating risks such as data poisoning, malicious package injection, and unauthorized access. Research into blockchain-based Trust and Reputation Systems (TRS) demonstrates that integrating reputation scores directly into access control policies allows for dynamic, self-adaptive security postures. For instance, frameworks have been developed where trust and reputation are explicitly incorporated into attribute-based access control, enabling the system to assign varying levels of access rights based on a node’s historical behavior and current standing [404]. This approach evolves the static, binary decisions of traditional Role-Based Access Control (RBAC) into a fluid process that adapts to real-time threat intelligence, effectively isolating compromised or malicious nodes before they can inflict significant damage on the supply chain.

The efficacy of these reputation mechanisms often relies on sophisticated algorithms capable of aggregating feedback and detecting Sybil attacks, where a single adversary creates multiple fake identities to manipulate the system. Advanced reputation models, such as those utilizing feedback aggregation with decay parameters, have been proposed to bound the benefits of such attacks rather than attempting to prevent them entirely, thereby ensuring Sybil tolerance in merit-based tokenomics [405]. Furthermore, the application of network control theory to reputation dynamics suggests that firms or entities with medium reputation often serve as the most effective driver nodes for maintaining network controllability, highlighting the need for governance models that carefully balance the influence of high-reputation incumbents with emerging participants [406]. In signed social networks, quantifying the trustworthiness of community structures provides a pipeline for assessing the overall resiliency of the system against link or node failures, a capability essential for

maintaining supply chain stability during adversarial conditions [407].

Token-based incentives serve as the economic engine driving participation and honest behavior within these decentralized governance frameworks, complementing the cryptographic and architectural layers. Cryptoeconomic tokens function not merely as currency but as programmable instruments that align individual incentives with the collective security goals of the supply chain. Studies indicate that while tokens can incentivize information sharing, care must be taken to avoid crowding out intrinsic motivation; however, well-designed multi-token systems can create interaction effects that enhance both the quantity and quality of shared data [408]. In the context of Proof-of-Stake (PoS) networks, which underpin many consortium blockchains, the selection of validators is heavily influenced by reputation and cost considerations. Users tend to delegate stakes to validators who have established a reputation for reliability, reaching a Nash equilibrium that, while efficient, can unfortunately lead to token concentration [409]. To counteract centralization risks and ensure decentralization, novel consensus mechanisms like Proof-of-Reputation (PoR) have been proposed, where nodes with the highest reputation values form the consensus group, thereby securing the ledger against malicious behavior inherent in open, dynamic peer-to-peer networks [410].

Moreover, the integration of reputation into sharding architectures offers a pathway to high-throughput supply chain transactions without compromising security. Systems like RepChain utilize reputation to characterize validator heterogeneity, enabling a reputation-based sharding and leader selection scheme that enhances both throughput and security levels [411]. This is particularly relevant for supply chains requiring rapid verification of software components and AI models. Additionally, incentive mechanisms must be carefully engineered to prevent power grabs by stake-pool operators who might otherwise censor transactions to maintain control. Game-theoretic analyses reveal that anti-censorship mechanisms can incentivize a “revolution” where stakeholders withdraw support from censoring cartels, thus preserving the decentralized nature of the governance model [412].

In consortium-based networks, where participants represent different organizations, the management of access rights through decentralized identifiers and verifiable credentials further strengthens governance. By replacing traditional certificates with decentralized identifiers, systems can achieve flexible user authentication and authorization, supporting distributed policy decision points even among mutually untrusted entities [413]. This is complemented by frameworks that use smart contracts to automate the verification of usage conditions and monitor compliance, ensuring that resource governance remains transparent and enforceable across the supply chain [414]. Ultimately, the convergence of dynamic reputation systems, token-based incentives, and automated access control creates a resilient governance architecture. Such architectures not only detect and mitigate threats in real-time but also foster a culture of accountability, where every participant’s contribution to the supply chain integrity is continuously evaluated and rewarded. However, as these governance models increasingly rely on automated algorithms to assign reputation and enforce penalties, the transparency of the underlying decision-making logic becomes paramount. This necessity sets the stage for the subsequent discussion on Explainable AI (XAI), which provides the foundational tools to audit these automated governance decisions, ensuring they are responsible, understandable, and free from hidden biases.

7.5 Explainable AI (XAI) as a Foundation for Auditable and Responsible Security Decisions

7.5 Explainable AI (XAI) as a Foundation for Auditable and Responsible Security Decisions

While decentralized governance and reputation mechanisms provide the structural trust framework for supply chain integrity, as discussed in the previous section, the operational reliability of these systems ultimately depends on the transparency of the AI models driving security decisions. The integration of Artificial Intelligence (AI) into cybersecurity operations has fundamentally altered the threat landscape, offering unprecedented capabilities in detecting anomalies and automating responses. However, the widespread adoption of deep learning models, which often operate as opaque “black boxes,” introduces significant challenges for accountability, trust, and regulatory compliance. In high-stakes environments such as software supply chain security, the inability to comprehend the rationale behind an AI-driven security decision can lead to catastrophic failures, unjustified blocking of legitimate updates, or the silent acceptance of sophisticated attacks. Consequently, Explainable AI (XAI) emerges not merely as a technical enhancement but as a foundational architectural enabler required to transform security operations from inscrutable automated processes into transparent, auditable, and responsible workflows that underpin the reputation systems described earlier.

The primary role of XAI in this context is to bridge the cognitive gap between complex algorithmic outputs and human understanding, thereby facilitating effective human-in-the-loop oversight. Traditional AI systems in cybersecurity, particularly Intrusion Detection Systems (IDS) powered by deep learning, frequently lack the ability to justify their predictions, creating a barrier for security analysts who must validate threats before taking action [362]. Without clear justifications, analysts are forced to either blindly trust the system or ignore its alerts, both of which undermine the security posture. XAI techniques aim to demystify these decisions by highlighting the specific features, code patterns, or behavioral anomalies that triggered a security alert, allowing regulators and operators to verify whether the automated response was appropriate and proportional to the threat. This verification is essential for maintaining the accuracy of the reputation scores and access rights managed by decentralized governance frameworks.

However, the path to meaningful explainability is fraught with complexities regarding who the explanation is for and what constitutes a valid explanation. Research indicates that different stakeholders possess distinct explanation needs; for instance, model designers require insights into feature importance for debugging, while regulators and end-users need understandable narratives to assess fairness and liability [415]. A one-size-fits-all approach to explainability often fails to meet these diverse requirements, potentially leading to a false sense of security or “painting the black box white,” where explanations appear transparent but fail to accurately reflect the model’s internal logic [359]. Therefore, architectural frameworks for secure supply chains must incorporate user-centric design methodologies that tailor explanations to the specific operational context and the expertise level of the recipient, whether they are automated auditing bots, human security operators, or external compliance officers [416]. Addressing this heterogeneity is a prerequisite for the granular, stakeholder-specific explainability strategies detailed in the subsequent section.

A critical aspect of using XAI for auditable security decisions is the tension between transparency and security. While explaining model decisions builds trust, it can inadvertently expose the model’s internal mechanics to adversaries, enabling them to craft evasion attacks or reverse-engineer proprietary algorithms. This phenomenon, known as the “explanation-guided attack,” suggests that

providing detailed feature attributions (such as SHAP values) can help attackers identify which features to manipulate to bypass detection [417]. To address this paradox, responsible security architectures must implement mechanisms that provide sufficient explainability for auditing without compromising the model’s robustness. Techniques such as local differential privacy in explanations or the use of counterfactuals that obscure sensitive internal weights while still offering actionable insights are essential components of a resilient system [418].

Furthermore, the reliability of XAI methods themselves is a prerequisite for their use in regulatory validation. Current literature highlights a “disagreement problem” where different XAI techniques produce conflicting explanations for the same model prediction, leaving auditors unsure of which interpretation to trust [419]. If the explanation tool is itself a black box or produces unstable results, it cannot serve as a reliable basis for accountability. Consequently, the deployment of XAI in supply chain security must be accompanied by rigorous evaluation metrics that assess faithfulness, stability, and completeness. Moving beyond post-hoc explanations, there is a growing argument for developing intrinsically interpretable models or “glass-box” architectures that provide transparent decision logic by design, thereby eliminating the approximation errors inherent in post-hoc methods [420].

From a governance perspective, XAI serves as the technical backbone for enforcing algorithmic accountability and meeting regulatory mandates such as the EU AI Act. Regulations increasingly demand that high-risk AI systems provide transparency regarding their decision-making processes to ensure fundamental rights and safety [421]. In the context of software supply chains, this implies that automated systems rejecting a software package or flagging a vulnerability must generate an immutable, explainable record of the decision. This record allows for ex-post challenges and investigations, ensuring that automated security decisions can be contested and corrected if found to be biased or erroneous [422]. By embedding XAI into the core of security architectures, organizations can transition from reactive, opaque defense mechanisms to proactive, transparent systems where every automated action is traceable, justifiable, and aligned with ethical and legal standards.

Ultimately, the successful integration of XAI into security operations requires a shift in paradigm from viewing explainability as an optional add-on to recognizing it as a critical security control. It enables the verification of AI behavior, supports the debugging of flawed models, and fosters trust among all stakeholders involved in the software supply chain. As the complexity of AI-driven threats evolves, the ability to provide clear, accurate, and secure explanations will define the boundary between resilient, accountable systems and fragile, untrustworthy automation. Future research must continue to refine these techniques, ensuring they are robust against adversarial manipulation while remaining accessible to the diverse array of humans and systems responsible for securing our digital infrastructure [423]. This foundation sets the stage for exploring advanced, context-aware explainability frameworks tailored to the specific cognitive and operational needs of diverse stakeholders.

7.6 User-Centric and Context-Aware Explainability for Diverse Stakeholders

7.6 User-Centric and Context-Aware Explainability for Diverse Stakeholders

Building upon the foundational role of Explainable AI (XAI) as a prerequisite for auditable security decisions discussed in the previous section, it becomes evident that a generic, one-size-fits-all approach to transparency is insufficient for the complex ecosystem of AI-driven software supply chains. While Section 7.5 established that XAI is critical for transforming opaque automated processes into responsible workflows, the effectiveness of these explanations hinges on their alignment with the

specific cognitive models, operational contexts, and decision-making needs of diverse stakeholders. A critical failure in current implementations is the assumption that technical interpretability equates to effective transparency; however, research indicates that explanations must be designed to support specific human understanding processes rather than merely exposing algorithmic internals [424]. Consequently, architectural enablers must evolve from static output generators to dynamic, user-centric frameworks that decouple stakeholder knowledge from their interpretability needs, ensuring that explanations are actionable, plausible, and ethically grounded. This shift is essential to operationalize the trust mechanisms introduced earlier and sets the stage for the converged architectural solutions detailed in the subsequent section.

A foundational step in designing such user-centric explainability is the rigorous characterization of the audience, moving beyond traditional categorizations based solely on professional roles like developers, auditors, or end-users. A more granular framework is required, one that characterizes stakeholders based on their formal, instrumental, and personal knowledge across the domains of machine learning, data, and the specific operational milieu [425]. For instance, a security engineer managing a Continuous Integration/Continuous Deployment (CI/CD) pipeline requires different explanatory granularity compared to a compliance officer assessing regulatory adherence or a software developer deciding whether to merge a dependency update. The former may need low-level feature attribution to debug a false positive in a vulnerability scanner, while the latter requires high-level justifications regarding risk propagation and legal liability. By mapping these distinct knowledge profiles to hierarchical typologies of stakeholder needs, architects can design systems that deliver explanations ranging from high-level domain goals to low-level interpretability tasks, thereby bridging the gap between the technical outputs described in Section 7.5 and actual human comprehension.

Central to this architectural approach is the concept of “Explainability-by-Design,” which advocates for the proactive integration of explanation capabilities during the system design phase rather than as a post-hoc addition. This methodology involves a structured workflow comprising explanation requirement analysis, technical design, and validation, ensuring that the resulting explanations meet specified regulatory and business requirements [426]. In the context of supply chain security, this implies that AI models used for anomaly detection or malware classification must be instrumented to generate provenance traces and decision logs that can be queried to construct personalized explanations. Furthermore, the granularity of these explanations must be adaptable. Research into multi-granular summarization and text encoding suggests that providing explanations at varying levels of semantic coverage—from concise summaries to detailed, evidence-rich narratives—allows stakeholders to drill down into details only when necessary [427; 428]. This progressive disclosure prevents information overload and supports the formation of accurate mental models of system behavior, which is crucial for maintaining trust during high-stakes security incidents [429].

The ethical dimensions of context-aware explainability cannot be overstated, particularly when AI decisions impact human livelihoods or critical infrastructure. Explanations serve not only to inform but also to empower stakeholders to contest undesirable outcomes or understand the conditions required for a different result. Counterfactual explanations, which describe the smallest changes to input factors that would alter a decision, have emerged as a powerful tool for this purpose [430]. However, addressing the reliability concerns raised in the previous section regarding the “disagreement problem,” it is paramount that these explanations remain stable; if counterfactuals vary significantly across different algorithms for the same instance, they can lead to confusion and potential “fairwashing,” where malicious actors exploit explanation diversity to mask unfair model behaviors [431; 432]. Therefore, secure supply chain architectures must incorporate mechanisms

to ensure the robustness, actionability, and plausibility of counterfactuals, potentially leveraging convex optimization techniques to enforce feature dependencies and realistic constraints [433].

Moreover, the tension between transparency and other values such as privacy and security requires careful calibration, echoing the security paradoxes discussed previously. While transparency is often viewed as an end in itself by the technical community, regulatory frameworks like the EU AI Act view it as a means to support broader values like accountability and human rights [421]. Consequently, explainability modules must be flexible enough to adapt to varying legal and operational contexts. For example, in highly regulated sectors like healthcare or finance, explanations must satisfy strict auditability requirements, potentially utilizing agent-based approaches to systematically specify and verify transparency goals [434]. Conversely, in fast-paced development environments, explanations might focus on rapid debugging and iterative improvement. The challenge lies in defining the “right” level of explainability by weighing the social benefits against the costs of implementation, including computational overhead and the risk of exposing sensitive intellectual property [435].

Finally, the effectiveness of any explainability strategy depends on continuous evaluation and adaptation to user feedback. Studies reveal that current deployments often fail to serve external stakeholders, focusing instead on internal debugging by engineers [436]. To rectify this, future architectures must incorporate human-in-the-loop mechanisms that allow diverse stakeholders to interact with explanations, ask questions, and provide feedback on their utility. This iterative process ensures that transparency measures evolve alongside the threat landscape and the changing needs of the ecosystem. By adopting a multidisciplinary approach that combines technical rigor with insights from human-computer interaction, law, and ethics, the AI software supply chain can move towards a future where transparency is not merely a compliance checkbox but a robust foundation for trust, accountability, and resilient security operations. These user-centric and context-aware frameworks provide the necessary functional specifications for the next evolutionary step: the synthesis of these interpretability mechanisms with blockchain and Zero Trust architectures to create fully converged, resilient systems.

7.7 Converged Architectures: Integrating Blockchain, XAI, and Zero Trust for Resilient Systems

7.7 Converged Architectures: Integrating Blockchain, XAI, and Zero Trust for Resilient Systems

Building upon the user-centric and context-aware explainability frameworks discussed previously, the next evolutionary step in securing AI software supply chains lies in the synthesis of these interpretability mechanisms with robust architectural primitives. The escalating complexity of modern ecosystems necessitates a paradigm shift from isolated security controls to converged architectural frameworks that synergize the strengths of Blockchain, Explainable AI (XAI), and Zero Trust architectures. While the preceding section established the need for tailored explanations for diverse stakeholders, this section addresses how such explanations can be anchored in immutable data and enforced through dynamic verification. Individually, these technologies offer distinct advantages—immutability, transparency, and continuous verification, respectively—but their integration creates a resilient, self-healing ecosystem capable of withstanding sophisticated adversarial threats. This convergence resolves the fundamental limitations of standalone solutions: blockchain ensures data integrity but lacks inherent interpretability; XAI provides critical insights but relies on the assumption of trusted data sources; and Zero Trust enforces strict access controls but requires dynamic, context-aware policy generation. By synthesizing these domains, we construct supply chains that

are not only secure by design but also accountable and adaptable in real-time, directly addressing the demand for actionable and plausible transparency.

At the core of this converged architecture lies the use of Distributed Ledger Technology (DLT) to establish an immutable backbone for provenance and auditability, serving as the foundational trust anchor. Blockchain records every transaction, model update, and component interaction in a tamper-proof ledger, mitigating risks associated with centralized points of failure and data tampering prevalent in traditional models. For instance, frameworks like Trinity demonstrate how combining blockchain with publish-subscribe messaging models can guarantee persistence and ordering across trust boundaries, effectively eliminating the risk of a central broker manipulating data flows [437]. Similarly, in the context of IoT and cyber-physical systems, blockchain-enabled architectures such as DeTRM provide decentralized trust and reputation management, correlating empirical sensor data to assess authenticity and quantify trustworthiness through smart contracts [438]. These systems ensure that the data feeding into AI models and the components traversing the supply chain are verifiable and untampered, providing the reliable foundation necessary for the generation of the trustworthy explanations detailed in the previous section.

However, immutability alone does not guarantee understanding or appropriate response to anomalies, reinforcing the necessity of the XAI layer within this convergence. In this integrated architecture, XAI acts as the interpretive interface that transforms opaque blockchain records and complex network telemetry into the actionable intelligence required by the diverse stakeholders identified earlier. The integration of XAI allows stakeholders to audit the decision-making processes of autonomous security systems, ensuring that alerts and remediation actions are grounded in logical reasoning rather than black-box predictions. This transparency is vital for maintaining operational trust, particularly in safety-critical domains like healthcare and manufacturing. For example, in smart manufacturing environments, layered architectures leveraging Digital Twins and metaverse applications enable human operators to inspect and monitor processes, bridging the gap between raw data and semantic understanding [439]. By embedding XAI into these monitoring systems, organizations can trace the lineage of a security alert back to specific supply chain events recorded on the blockchain, thereby facilitating rapid root cause analysis and reducing the time to recovery in a manner consistent with the “Explainability-by-Design” principles.

Complementing these layers is the Zero Trust architecture, which operates on the principle of “never trust, always verify,” providing the enforcement mechanism for the insights generated by XAI and the data secured by blockchain. In a converged system, Zero Trust policies are dynamically enforced based on real-time integrity scores derived from the blockchain and the contextual insights provided by XAI. Unlike static perimeter defenses, Zero Trust adapts to the fluid nature of modern supply chains, where devices and services frequently join and leave the network. The integration of Self-Sovereign Identity (SSI) with blockchain further strengthens this model by providing decentralized, verifiable identities for all entities. Systems like NSSIA illustrate how accountability mechanisms can be embedded within SSI schemes using consortium blockchains and threshold algorithms, ensuring that malicious actors can be identified and sanctioned without compromising user privacy [440]. Furthermore, the deployment of Trusted Execution Environments (TEEs) alongside Zero Trust principles, as seen in secure MQTT brokers like MQT-TZ, ensures that even if the underlying infrastructure is compromised, the processing of sensitive data remains secure and verifiable [441].

The true power of this triad emerges in the creation of self-healing and accountable supply chains, where the loop between detection, explanation, and enforcement is fully automated. When an anomaly is detected by an XAI-driven monitoring module, the system can automatically query

the blockchain to verify the provenance of the affected component. If the component is found to be compromised or its history inconsistent with the immutable ledger, the Zero Trust controller can instantly isolate the entity, revoke its credentials, and trigger a remediation workflow. This automated response loop is exemplified by architectures like SmartQC, which utilize DLT to automate trusted data workflows and business logic, ensuring that processes remain robust against interruptions [373]. Moreover, the concept of accountability gadgets in Byzantine fault-tolerant systems demonstrates how protocols can be designed to provide undeniable evidence of misbehavior, enabling seamless reconfiguration and the eviction of faulty nodes [442]. This seamless interaction ensures that the counterfactual explanations and audit trails discussed previously are not merely retrospective reports but active components of a defensive strategy.

Looking toward the future, the convergence of these technologies paves the way for federated and hierarchical models that scale across global ecosystems. Frameworks such as Fed-DDM propose federated ledgers that combine private intra-ledgers for efficiency with public inter-ledgers for cross-domain security, creating a scalable infrastructure for data marketplaces [379]. In such environments, the synergy between blockchain’s immutable audit trails, XAI’s transparent reasoning, and Zero Trust’s rigorous enforcement creates a resilient fabric that can withstand both internal failures and external attacks. As supply chains become increasingly autonomous and interconnected, the adoption of these converged architectures will be paramount in ensuring the integrity, transparency, and sustainability of the global digital economy. By moving beyond siloed defenses to integrated, intelligent systems, we can realize a vision of supply chains that are not merely secure, but inherently trustworthy and self-sustaining, effectively operationalizing the user-centric explainability goals into a robust, systemic reality.

8. Operationalizing Security: MLSecOps, Governance, and Automated Remediation

8.1 Implementing MLSecOps: Integrating Security into the AI Development Lifecycle

8.1 Implementing MLSecOps: Integrating Security into the AI Development Lifecycle

The operational integration of Machine Learning Security Operations (MLSecOps) represents a paradigm shift from traditional DevSecOps, necessitating a holistic reimagining of the software development lifecycle to address the unique vulnerabilities inherent in data-driven systems. Unlike conventional software, where the codebase is deterministic and static, machine learning (ML) systems introduce dynamic variables including training data distributions, model parameters, and probabilistic inference outcomes. Consequently, securing the AI software supply chain requires embedding security controls not merely at the deployment stage but throughout the entire continuum of design, data preparation, model training, and continuous monitoring. The rapid adoption of ML in safety- and business-critical domains has exposed significant gaps in current practices, where the rush to deploy often leads to the neglect of latent vulnerabilities and privacy issues that are difficult to identify post-deployment [443]. To mitigate these risks, organizations must adopt a structured MLSecOps framework that treats security as a continuous, automated process rather than a periodic audit, laying the foundational governance required before automated vulnerability analysis and patch generation can be effectively deployed.

The foundation of effective MLSecOps lies in the seamless integration of security practices into the Continuous Integration and Continuous Delivery (CI/CD) pipelines specific to machine learning,

often referred to as MLOps. While MLOps provides the architectural backbone for automating model deployment and operations, it frequently lacks native security enforcement mechanisms. Research indicates that while practitioners recognize the benefits of MLOps for larger applications requiring continuous deployment, the initial investment in establishing these automated, secure pipelines can be perceived as a barrier for smaller or initial project versions [444]. Therefore, implementing MLSecOps involves extending standard MLOps workflows to include rigorous threat modeling, data validation, and model attestation steps. This extension is critical because the ML lifecycle is inherently iterative and interdependent; changes in data can necessitate model retraining, which in turn requires re-validation of security properties. An experimental analysis of real-world projects reveals that the MLOps workflow is highly dynamic, with frequent revisits to earlier phases such as data preparation and model development, demanding that security checks be equally agile and non-disruptive [445]. By establishing these secure pipelines, organizations create the necessary infrastructure to support the advanced automated analysis techniques discussed in subsequent stages.

A primary focus of MLSecOps is the protection of data assets and model artifacts against emerging threats such as data poisoning, model extraction, and adversarial evasion. In the design phase, security teams must collaborate with data scientists to define threat models that account for the specific attack surfaces of ML components, moving beyond traditional software vulnerabilities to include adversarial examples and logic-bomb attacks embedded in modular learning components [39]. During the data preparation stage, automated pipelines must incorporate integrity checks to detect poisoned datasets or distribution shifts that could compromise model robustness. Furthermore, as ML models are increasingly deployed on edge devices and mobile applications, the risk of intellectual property theft becomes paramount. Empirical studies have shown that a significant percentage of mobile apps fail to adequately protect their on-device models, leaving them vulnerable to extraction via unsophisticated dynamic analysis [446]. MLSecOps frameworks must therefore integrate techniques such as model encryption, obfuscation, and the use of Trusted Execution Environments (TEEs) to guard offline deployments and ensure that proprietary models remain secure even in untrusted environments [447]. These defensive measures at the artifact level complement the vulnerability detection strategies that follow, ensuring that models are both inherently protected and continuously monitored.

The deployment and monitoring phases present unique challenges where traditional security metrics often fall short, creating a data-rich environment essential for the automated analysis tools described later. In production environments, the absence of ground truth labels makes it difficult to calculate standard performance metrics like accuracy, complicating the detection of security incidents or model degradation. To address this, MLSecOps pipelines should employ specialized monitoring tools, such as Model Performance Predictors (MPP), which utilize ensemble metrics to estimate prediction quality and trigger alerts for potential anomalies or attacks [448]. Additionally, the interaction between safety and security is a central concern; security violations can directly lead to compromised safety in critical systems. Active monitoring strategies that utilize statistical difference measures, such as the Kolmogorov-Smirnov test, can detect shifts in the operational context of ML components, providing a basis for identifying whether an application is under attack or experiencing data drift [449]. The insights generated by these monitoring systems serve as critical inputs for the intelligent vulnerability prioritization and patch generation mechanisms that define the next evolution of supply chain security.

Furthermore, the human element remains a critical component of successful MLSecOps implementation, acting as the bridge between theoretical frameworks and the automated tools required for

scale. Interviews with machine learning engineers reveal that operationalizing ML involves balancing velocity, visibility, and versioning, often requiring extensive collaboration across data science, operations, and security teams [450]. However, industry practitioners often lack the tactical and strategic tools necessary to protect, detect, and respond to attacks on their ML systems, highlighting a gap between academic research and industrial practice [451]. Bridging this gap requires the development of maturity models, such as the Machine Learning Security Maturity Model (MLSMM), which organize security practices along the ML development lifecycle and provide clear benchmarks for improvement [452]. By integrating these maturity models into organizational governance, enterprises can systematically advance their security posture from ad-hoc measures to optimized, automated defenses, thereby enabling the transition to the sophisticated AI-driven remediation workflows detailed in the following section.

Finally, the scalability of MLSecOps depends on the ability to automate security testing and remediation, setting the stage for the intelligent patch generation capabilities explored next. Just as traditional software benefits from automated static and dynamic analysis, ML systems require automated adversarial training and robustness verification. Techniques such as continual learning plugins can facilitate end-to-end support for model updating and validation without disrupting serving engines, ensuring that security patches and model updates are deployed efficiently [453]. Moreover, in specialized domains like biotechnology and healthcare, the integration of MLSecOps is not just a technical necessity but a regulatory imperative, ensuring that innovative solutions comply with strict frameworks while maintaining ethical responsibilities [454]. Ultimately, the successful implementation of MLSecOps transforms the AI development lifecycle into a resilient, self-healing system capable of withstanding the evolving landscape of adversarial threats, providing the robust operational context necessary for the automated vulnerability analysis and intelligent patch generation strategies that follow.

8.2 Automated Vulnerability Analysis and Intelligent Patch Generation

8.2 Automated Vulnerability Analysis and Intelligent Patch Generation

Building upon the robust operational foundations and secure pipelines established by MLSecOps in the preceding section, the sheer scale of modern AI deployment necessitates a transition from manual oversight to automated intelligence in vulnerability management. The exponential growth in disclosed software vulnerabilities has rendered traditional, manual processes increasingly unsustainable for Security Operations Centers (CSOCs), which are frequently overwhelmed by the volume of Common Vulnerabilities and Exposures (CVEs). This bottleneck leads to critical delays in patching, leaving systems exposed to exploitation. To address this, the field is shifting toward automated vulnerability analysis and intelligent patch generation, leveraging advanced artificial intelligence techniques to prioritize threats based on real-world impact and to autonomously generate remediation code. This transition represents a fundamental evolution from the reactive, static scoring systems discussed in traditional contexts to dynamic, AI-driven decision-making frameworks that integrate semantic understanding with strategic reasoning, serving as the technical engine that powers the resilient workflows initiated by MLSecOps.

A primary challenge in this automated landscape is the precise prioritization of threats. The industry standard, the Common Vulnerability Scoring System (CVSS), often provides low-level, product-oriented descriptions that lack the essential semantic context required to estimate actual threat impact or identify specific countermeasures within complex AI supply chains. Manual evaluation of these records to map them to Common Weakness Enumerations (CWE) is time-consuming

and prone to human error, creating a gap that automated tools must fill. Recent advancements have introduced predictive models capable of automating this analysis with high precision. For instance, novel tools have been developed to accurately estimate CVSS vectors and seamlessly automate the classification of CVEs into appropriate CWE hierarchy classes, surpassing the capabilities of general-purpose large language models in this specific domain [455]. Furthermore, the mapping of CVE records to MITRE CWE weaknesses has been successfully approached as a ranking task using fine-tuned deep learning models such as Sentence-BERT and rankT5, which demonstrate sizable performance gains over traditional retrieval methods by leveraging superior semantic understanding [456]. Other hierarchical neural network approaches, such as ThreatZoom, have achieved accuracy rates of up to 94% in coarse-grain classification by adaptively adjusting weights based on text analytic scores, effectively addressing the backlog of unclassified vulnerabilities [457].

Beyond simple classification, intelligent analysis requires linking vulnerabilities to potential attack techniques and threat actors to understand the operational risk, a necessity highlighted by the dynamic threat models introduced in MLSecOps. Advanced frameworks now utilize multi-modal representation learning, combining Knowledge Graphs with Large Language Models, to uncover hidden links between unseen security entities. These systems can predict links between CVEs, CWEs, and Common Platform Enumerations (CPEs) with significantly higher accuracy than previous methods, enabling the discovery of affected products that might otherwise remain unpatched within critical compliance windows [458]. Additionally, domain-specific language models have been engineered to extract semantic information from CVE descriptions to infer plausible Tactics, Techniques, and Procedures (TTPs) within the MITRE ATT&CK framework. Tools like TTPpredictor have demonstrated the ability to correlate vulnerabilities with specific attack actions with accuracy rates approaching 98%, providing security practitioners with actionable intelligence for proactive defense [459]. Similarly, systems like Crimson enhance the strategic reasoning of LLMs through Retrieval-Aware Training, converting raw vulnerability data into structured insights that rival the performance of much larger models in strategic cybersecurity tasks [460].

Once threats are prioritized, the next frontier is the automated generation and validation of patches, fulfilling the “self-healing” promise of the MLSecOps lifecycle. Traditional approaches are often deterministic and fail to account for future uncertainties or the dynamic nature of resource constraints inherent in continuous AI development. To overcome this, researchers have proposed deep reinforcement learning (DRL) frameworks for cyber vulnerability management. These frameworks, such as Deep VULMAN, utilize DRL agents to make sequential decisions regarding resource allocation and vulnerability mitigation, outperforming static methods by adapting to fluctuations in vulnerability arrivals and optimizing the selection of fixes under uncertainty [461]. In the realm of code repair, sequence-to-sequence learning models have been trained on vast software development histories from repositories like GitHub to automatically generate patches for C vulnerabilities. These systems leverage the rich context of historical code changes to produce fixes that are evaluated against real-world CVE databases, demonstrating the feasibility of fully automated repair pipelines [462].

Furthermore, the application of Chain-of-Thought (CoT) prompting in Large Language Models has revolutionized the discovery and fixing of software vulnerabilities, enhancing the granularity of detection required for probabilistic AI systems. By guiding LLMs through a structured reasoning process, unified prompting approaches have achieved substantial improvements in vulnerability identification, discovery, and patching tasks compared to standard baselines. This methodology allows models to not only detect vulnerabilities but also provide fine-grained interpretations and generate valid patches, bridging the gap between detection and remediation [139]. Complementary research has also explored “ahead-of-threat” virtual patching, where deep neural networks are

trained to predict malicious inputs for vulnerabilities that have not yet been discovered, effectively putting defenders steps ahead of attackers by generating predictive models based on historical testing data [463].

Finally, the integration of these technologies into a cohesive operational workflow is essential to realize the full potential of the secure pipelines described previously. Vulnerability prioritization strategies inspired by offensive security practices, such as penetration testing, offer a more realistic assessment of risk than traditional scoring systems. By automating the end-to-end process of evaluating vulnerabilities through the lens of an attacker, organizations can focus their limited resources on the most critical exposures [464]. As the landscape evolves, the synthesis of automated CVE analysis, semantic mapping, reinforcement learning for resource optimization, and generative AI for patch creation forms the backbone of a resilient, next-generation software supply chain security posture. However, the efficacy of these automated tools relies not just on technical precision but on how the resulting findings are communicated and managed. This necessity drives the evolution from technical automation to procedural governance, setting the stage for the specialized disclosure protocols and Coordinated Flaw Disclosure (CFD) frameworks detailed in the following section.

8.3 Evolving Coordinated Disclosure: From CVD to Coordinated Flaw Disclosure for AI

8.3 Evolving Coordinated Disclosure: From CVD to Coordinated Flaw Disclosure for AI

While Section 8.2 detailed the technological shifts toward automated vulnerability analysis and intelligent patch generation, the operational efficacy of these tools relies fundamentally on robust disclosure protocols. The establishment of such protocols has long been a cornerstone of traditional cybersecurity, with Coordinated Vulnerability Disclosure (CVD) serving as the gold standard for managing the discovery, reporting, and remediation of software flaws. However, the advent of artificial intelligence (AI) and machine learning (ML) systems has introduced a paradigm shift in the nature of system failures, rendering many aspects of conventional CVD inadequate. The fundamental distinction lies in the deterministic nature of traditional software versus the probabilistic and non-deterministic behavior of AI models. While traditional vulnerabilities often stem from clear coding errors or logic flaws that can be uniquely identified and patched—tasks increasingly addressed by the AI-driven methods discussed previously—AI systems suffer from “algorithmic harms” and model weaknesses that are context-dependent, emergent, and often resistant to binary fixes. Consequently, there is an urgent imperative to evolve from standard CVD practices toward a specialized Coordinated Flaw Disclosure (CFD) framework tailored specifically to the intricacies of the AI lifecycle.

The current landscape of harm reporting in the field of AI operates largely on an ad hoc basis, lacking the structured processes that define mature security ecosystems. In contrast, the CVD ethos plays a pivotal role in software security by fostering transparency and institutional support for researchers acting in good faith. Yet, algorithmic flaws in ML models present distinct challenges compared to traditional software vulnerabilities, warranting a specialized approach that accounts for the unique properties of data-driven systems. A dedicated CFD framework is necessary to address the gap between the academic research community, where algorithmic flaws are well-known, and the operational communities that deploy and manage these systems. Without a clear line of communication, critical vulnerabilities in model behavior remain unaddressed in production envi-

ronments. By hypothetically assigning Common Vulnerabilities and Exposures (CVE) identifiers to ML algorithms and model objects, stakeholders could establish useful lines of communication, introducing the research community to operational security concepts that are currently missing [465]. This conceptual bridging is essential before the industry can fully adopt the advanced metadata standards and AI Bill of Materials (AIBOM) discussed in the subsequent section.

A primary limitation of traditional CVD is its inability to adequately handle the non-deterministic failures inherent in AI. AI systems are susceptible to a complex array of failure modes that go beyond simple code execution errors. These include specification gaming, reward hacking, fragility to distributional shifts, and various multi-agent failure modes such as accidental steering, coordination failures, and adversarial misalignment. These issues are often more complex and less understood than single-agent failures, yet they occur frequently and largely unnoticed in deployed systems [466]. Furthermore, the risk associated with AI is not static; it involves reasoning under uncertainty, where the lack of performance guarantees and the computational explosiveness of deep uncertainty constraints create structural risks that traditional disclosure models do not capture [467]. A CFD framework must therefore accommodate the reporting of these nuanced failures, distinguishing between transient errors, systemic biases, and fundamental model incapacity.

The proposed CFD framework must also integrate mechanisms for proactive failure detection and risk quantification that are specific to regression and classification tasks in AI, complementing the automated analysis techniques outlined earlier. For instance, failure detection in safety-critical AI tasks requires metrics that go beyond simple accuracy, such as the Risk-coverage curve, to determine optimal coverage rates and model trustworthiness. Metrics like the Excess Area Under the Optimal RC Curve (E-AUoptRC) and the Trust Index (TI) provide a more meaningful evaluation of model reliability than traditional methods, highlighting that high overall accuracy does not necessarily equate to high trustworthiness [468]. Similarly, frameworks like PAGER (Principled Analysis of Generalization Errors in Regressors) demonstrate the need to systematically characterize failure modes in regression tasks by unifying epistemic uncertainties with non-conformity scores, a level of granularity that standard vulnerability reports rarely achieve [469]. A robust CFD process would mandate the inclusion of such detailed failure characterizations in disclosure reports, enabling developers to understand not just that a model failed, but why and under what conditions.

Moreover, the scope of CFD must extend to the collaborative dynamics between humans and AI, recognizing that failures often arise in the interaction layer. Attacks such as Dual Denial of Decision (DDoD) aim to deplete both computational and human resources, impairing the decision-making capabilities of hybrid teams [470]. Traditional CVD focuses on technical exploits, whereas CFD must address these socio-technical vulnerabilities. Additionally, the framework should facilitate the sharing of undisclosed vulnerabilities in a trusted manner, potentially leveraging blockchain-assisted mechanisms to protect sensitive information while ensuring that security workers can collaborate on patches without fear of leakage [471]. Such trusted sharing mechanisms lay the groundwork for the blockchain-empowered architectures and provenance tracking essential to the AIBOM standards explored next.

Ultimately, the transition from CVD to CFD represents a critical evolution in operationalizing AI security. It requires a shift from viewing vulnerabilities as static code defects to understanding them as dynamic algorithmic flaws that require continuous monitoring, specialized metrics, and cross-disciplinary collaboration. By adopting a CFD framework, the industry can enhance public trust through transparent processes that carefully balance the interests of organizations and the community, ensuring that the rapid deployment of AI systems does not outpace our ability to manage their inherent risks [61]. This evolution is essential for creating a resilient AI supply chain

where flaws are identified, disclosed, and remediated with the same rigor applied to traditional software, but with the nuance required by the complexity of intelligent systems. As the industry moves toward formalizing these disclosure practices, the need for standardized metadata and governance frameworks becomes paramount, setting the stage for the comprehensive AIBOM ecosystems discussed in the following section.

8.4 Governance Frameworks: Metadata Management, SBOMs, and the AIBOM Standard

8.4 Governance Frameworks: Metadata Management, SBOMs, and the AIBOM Standard

Building upon the specialized disclosure protocols outlined in the preceding section, the operationalization of Coordinated Flaw Disclosure (CFD) necessitates a robust structural foundation capable of capturing the complex lineage of AI artifacts. While CFD provides the procedural mechanism for reporting algorithmic harms, its efficacy relies fundamentally on comprehensive governance frameworks that extend traditional component tracking to encompass the unique metadata of machine learning systems. At the heart of this transformation lies the evolution of the Software Bill of Materials (SBOM) into an AI-specific standard, often referred to as the AI Bill of Materials (AIBOM). Although SBOMs have emerged as pivotal tools for managing software dependencies, vulnerabilities, and licenses, their current implementation faces significant hurdles that limit their effectiveness in securing modern, AI-driven infrastructures. Recent empirical studies indicate that despite heightened awareness and the development of various formats, SBOMs remain an early-stage technology with adoption barriers stemming from content deficiencies, tool limitations, and maintenance challenges [53]. Practitioners across diverse domains, including AI/ML and cyber-physical systems, have identified substantial gaps in how SBOMs are created, verified, and utilized, underscoring the urgent need for architectural enhancements that can accommodate the dynamic nature of AI assets required for meaningful flaw disclosure.

The transition from a standard SBOM to an AIBOM is driven by the fundamental differences between deterministic software components and probabilistic AI models, a distinction central to the shift from CVD to CFD discussed previously. Traditional SBOMs primarily inventory code libraries and their versions, yet they fail to capture the nuanced lineage of AI systems, which includes training datasets, hyperparameters, model weights, and the specific preprocessing pipelines used during development. To address this, researchers have proposed blockchain-empowered architectures that extend the remit of SBOMs to encompass AI systems, thereby coining the term AIBOM [51]. This expanded framework is essential for safeguarding critical infrastructure where the integrity of AI components is paramount. By leveraging verifiable credentials and selective disclosure mechanisms, such architectures aim to mitigate data tampering risks and overcome vendor reluctance to disclose sensitive intellectual property, thus fostering the transparent and resilient supply chain ecosystem necessary for the trusted sharing of vulnerabilities mentioned in the context of CFD.

Effective governance frameworks must also address the architectural requirements for managing the vast and heterogeneous metadata associated with AI models to support the detailed failure characterizations demanded by CFD. The current landscape of ML asset management is fraught with challenges related to model versioning, data traceability, and collaboration, which are crucial for maintaining consistent performance and streamlined development lifecycles [472]. Without standardized metadata representations, the search, reuse, comparison, and composition of pre-trained models are severely hindered, making it difficult to pinpoint the root causes of the non-

deterministic failures described earlier. To remedy this, there is a growing advocacy for standardized ML model metadata representations that support queryable “model zoos,” enabling practitioners to efficiently audit and report on model properties [473]. Such standardization is a prerequisite for the successful deployment of AIBOMs, as it ensures that the metadata accompanying AI artifacts is machine-readable, interoperable, and rich enough to support the rigorous security audits and risk quantification metrics essential for modern disclosure practices.

Furthermore, the integration of AIBOMs into governance frameworks requires addressing the semantic heterogeneity inherent in complex systems, particularly in sectors like smart cities and industrial IoT where AI interoperates with diverse sensor networks and building management systems. The lack of standardized practices in developing and deploying AI solutions has led to fragmented ecosystems, presenting challenges in interoperability and ethical governance [474]. Just as efforts are underway to establish industry standards for AI in education to ensure inclusivity and fairness, similar multi-tiered frameworks are necessary for the broader AI supply chain. These frameworks must facilitate the iterative development of standards, incorporating feedback loops from real-world applications to refine metadata schemas and governance protocols over time, thereby creating the consistent data environment needed to bridge the gap between technical vulnerability reports and the regulatory compliance discussions that follow.

The practical implementation of these governance frameworks also demands advanced tooling capable of automating the generation and analysis of AIBOMs, complementing the automated vulnerability analysis techniques discussed in prior sections. Current approaches to SBOM generation often rely on case-based reasoning that inconsistently identifies embedded components, particularly in binary files. Innovative methods utilizing zero-shot semantic similarity and transformer models have shown promise in automating this process, interpreting component identification as a semantic similarity task to relate product names to version strings with high accuracy [56]. Adapting such techniques for AI components could enable the automatic extraction of model lineage and dependency information, significantly reducing the manual burden on developers and improving the reliability of the resulting AIBOMs. Additionally, the use of ontologies, such as the Artificial Intelligence Ontology (AIO) constructed with LLM assistance, can provide a standardized hierarchy of AI concepts and methodologies, further enhancing the precision and utility of metadata within governance systems [475].

Ultimately, the establishment of robust governance frameworks centered on AIBOMs serves as the critical bridge between the operational disclosure practices of Section 8.3 and the regulatory and economic imperatives of Section 8.5. It is not merely a technical exercise but a socio-technical imperative that aligns technical standards with the accountability structures required for legal compliance. As organizations increasingly rely on automated decision-making systems, the ability to trace the origins and transformations of AI assets becomes critical for verifying claims about model behavior and responding to potential inquiries [476]. By integrating blockchain provenance, standardized metadata, and automated compliance auditing, these frameworks transform the supply chain from an opaque collection of dependencies into a transparent, accountable, and secure foundation. The convergence of these elements—enhanced SBOMs, rigorous metadata management, and the formalization of AIBOM standards—provides the necessary data richness to support the dynamic, data-driven governance mechanisms and economic risk mitigation strategies explored in the subsequent section, ensuring that all actors in the algorithmic supply chain can be held accountable with clarity and precision.

8.5 Regulatory Compliance, Accountability, and Economic Risk Mitigation

8.5 Regulatory Compliance, Accountability, and Economic Risk Mitigation

While the governance frameworks and AIBOM standards discussed in the previous section provide the necessary structural foundation for metadata management and component tracking, their efficacy ultimately hinges on robust regulatory compliance, clear accountability structures, and effective economic risk mitigation strategies. The operationalization of security within the AI software supply chain extends far beyond technical controls; it necessitates a socio-legal paradigm capable of assigning responsibility across distributed algorithmic ecosystems. As AI systems become increasingly embedded in critical infrastructure, the complexity of systems produced by “many hands”—involving data providers, model developers, deployers, and third-party vendors—creates significant visibility gaps that complicate traditional governance models. Effective accountability requires moving beyond simple liability assignment to understanding the intricate flows of data and decision-making that characterize modern algorithmic ecosystems. Scholars argue that algorithmic accountability discussions must explicitly consider the supply chain context, recognizing that the working together of diverse actors drives system outcomes and produces specific risks that cannot be attributed to a single entity in isolation [477]. This distributed nature of responsibility challenges existing legal frameworks, which often struggle to address the “accountability horizon” where the chain of causality becomes obscured by intermediate service models and cross-border regulatory arbitrage [477].

Meeting regulatory mandates in this complex environment demands a shift from static compliance checklists to dynamic, data-driven governance mechanisms that leverage the rich metadata enabled by AIBOMs. Boards of Directors and senior leadership are increasingly held accountable for cyber risk management, yet they frequently lack the adequate metrics and models necessary to make informed strategic decisions. A systematic review of cybersecurity governance highlights a critical gap: while sophisticated technical tools exist, there is a severe shortage of information presented in a language understandable to non-technical decision-makers, hindering effective oversight [478]. To bridge this gap, organizations must adopt interdisciplinary approaches that integrate technical risk assessments with socio-economic modeling. For instance, the application of structured expert judgment methods can facilitate risk assessment even when empirical data is scarce, enabling managers to set risk alarms, negotiate service level agreements, and identify necessary insurance coverage [479]. Furthermore, the implementation of automated compliance blueprints using Artificial Intelligence can help highly regulated industries map technical specifications to regulatory controls, ensuring continuous adherence to evolving standards from the outset of system design [480]. These automated mechanisms serve as a crucial precursor to the continuous, blockchain-enabled verification systems detailed in the following section.

Economic risk mitigation, particularly through cyber insurance, has emerged as a vital complement to technical defenses and regulatory compliance, offering a mechanism to transfer residual risks that cannot be entirely eliminated. However, the cyber insurance market faces significant hurdles, primarily driven by information asymmetry between insurers and the insured, as well as the correlated nature of cyber risks in interconnected networks. To address these challenges, researchers have proposed various game-theoretic and economic models to design incentive-compatible contracts that align the interests of all parties. For example, dynamic principal-agent models incorporating Markov decision processes have been developed to capture the interaction between user protection decisions and insurer pricing, revealing principles such as risk compensation and the Peltzman effect where insurance coverage might inadvertently reduce self-defense efforts [481].

Similarly, frameworks like FlipIn utilize bi-level game theory to design welfare-maximizing contracts for IoT ecosystems, accounting for the strategic interactions between Advanced Persistent Threat (APT) attackers, defenders, and insurers to optimize network-wide security outcomes [482].

The integration of cyber insurance into the broader risk management strategy also requires innovative approaches to pricing and premium calculation that reflect the specific maturity and topology of an organization’s security posture. Traditional actuarial methods often fail due to the lack of historical loss data in the cyber domain; consequently, new models propose using security maturity levels and entity relationship diagrams to derive more accurate premiums [483]. Moreover, the concept of mutual insurance models, such as those based on the Shapley value, offers a fair distribution of indemnity among participants in critical infrastructure sectors like power systems, ensuring that premiums reflect each entity’s contribution to systemic risk [484]. In the context of IoT and supply chains, accountability frameworks are being extended to include contract designs and cyber insurance as economic solutions that encourage truth-telling regarding security postures, thereby facilitating reliable investigations and risk mitigation [485].

Furthermore, the role of insurers is evolving from passive risk bearers to active regulators of cyber-security practices, creating a feedback loop that reinforces the technical controls established earlier. Post-incident audit strategies, modeled as Bayesian games, allow insurers to verify self-reported security levels and deter fraudulent claims, thus enforcing higher security standards across the supply chain [486]. The effectiveness of these economic instruments is further enhanced when combined with technological interventions, such as the adoption of secure operating systems, which can be incentivized through differentiated pricing markets linked to insurance data [487]. Ultimately, achieving resilient AI supply chains requires a converged approach where regulatory compliance, clear accountability mechanisms, and sophisticated economic risk transfer models work in tandem with the immutable provenance records discussed subsequently. This holistic strategy not only mitigates financial losses but also fosters a culture of shared responsibility, ensuring that all actors in the algorithmic supply chain are motivated to maintain high standards of security and integrity. As the landscape evolves, future research must continue to refine these models to address emerging threats, such as those posed by generative AI and quantum computing, ensuring that economic and regulatory frameworks remain robust enough to support the sustainable growth of the digital economy.

8.6 Operationalizing Trust: Blockchain Provenance and Automated Compliance Auditing

8.6 Operationalizing Trust: Blockchain Provenance and Automated Compliance Auditing

Building upon the regulatory frameworks, accountability structures, and economic risk models discussed in the preceding section, the practical realization of a secure AI software supply chain requires a paradigm shift from periodic, manual security assessments to continuous, automated verification mechanisms. While the previous section outlined the “why” and “what” of governance and risk transfer, this section addresses the “how” by detailing the technological infrastructure necessary to operationalize trust. As the complexity of AI dependencies grows, relying solely on human expertise for compliance auditing becomes unsustainable due to the sheer volume of data and the velocity at which vulnerabilities emerge. To bridge the gap between high-level policy and ground-level execution, the integration of blockchain-enabled traceability with expert-system frameworks offers a robust solution. This convergence creates an environment where trust is not

merely assumed based on contractual agreements or insurance policies but is mathematically and cryptographically verified through immutable audit trails and autonomous decision-making agents.

At the core of operationalizing trust is the establishment of immutable provenance records, which serve as the technical backbone for the accountability and AIBOM standards mentioned earlier. Blockchain technology provides a decentralized ledger capable of recording every stage of the AI lifecycle, from data collection and model training to deployment and inference. By leveraging blockchain, organizations can create tamper-proof audit trails that document the origin and transformation of software artifacts and data assets, directly addressing the visibility gaps and “many hands” problems identified in regulatory discussions. This capability is critical for verifying the authenticity of components and detecting unauthorized modifications that could undermine liability assignments. For instance, frameworks designed for supply chain management utilize blockchain to track interactions among participants, dynamically assigning trust and reputation scores based on verified events [488]. Such systems ensure that the history of an AI model or dataset is transparent and (immutable), allowing auditors to trace any anomaly back to its source with high fidelity. Furthermore, the application of blockchain in scientific data provenance demonstrates how immutable records can guarantee the trustworthiness of data processing steps, preventing fabrication or falsification of results [489].

However, the mere existence of an immutable ledger is insufficient without the intelligence to interpret the stored data and enforce the compliance policies and economic incentives described previously. This is where expert-system frameworks and automated auditing tools become indispensable. Traditional security compliance methods, such as vulnerability assessment and penetration testing, are often repetitive and resource-intensive, struggling to keep pace with the dynamic nature of algorithmic ecosystems. To overcome these inefficiencies, novel frameworks have been developed that enable the extraction, generalization, and re-utilization of security expertise. The Expert-System Automated Security Compliance Framework (ESASCF), for example, automates the security compliance process by capturing human expert knowledge and applying it autonomously to recurring scenarios [490]. By integrating such expert systems with blockchain ledgers, organizations can achieve a “set-and-forget” compliance posture where the system continuously monitors the supply chain, validates artifacts against stored policies, and flags deviations without constant human intervention. This synergy not only cuts the time required for security audits but also mitigates the risk of false negatives arising from human error or fatigue, thereby supporting the data-driven governance mechanisms required by board-level decision-makers.

The automation of compliance auditing also extends to the verification of smart contracts and decentralized applications that underpin many modern supply chain infrastructures, including those facilitating the cyber-insurance models discussed earlier. Given the susceptibility of smart contracts to vulnerabilities like Denial of Service (DoS) or reentrancy attacks, automated tools are essential for ensuring their security before and after deployment. Frameworks combining static and dynamic analysis, such as SmartScan, can detect specific vulnerability patterns with high precision, thereby enhancing the reliability of the underlying blockchain infrastructure [491]. Moreover, the integration of Large Language Models (LLMs) with fine-tuning techniques has shown promise in auditing smart contracts by generating justifications for detected vulnerabilities, although challenges in recall remain [492]. When these automated auditing capabilities are embedded within a blockchain-based governance structure, they form a self-regulating loop where non-compliant components are automatically identified and potentially isolated, reinforcing the incentive-compatible contracts proposed in economic risk mitigation strategies.

To further enhance trust, privacy-preserving mechanisms must be integrated into these automated

auditing workflows to balance transparency with confidentiality. In many regulatory environments, sharing detailed logs for auditing purposes may conflict with data privacy requirements or expose sensitive trade secrets, creating tension between compliance and competitiveness. Solutions like AGAPECert address this by utilizing trusted execution environments and blockchain to perform auditable computations on private data without revealing the underlying information [493]. This allows for real-time aggregate certification status reporting, ensuring that compliance is maintained without compromising sensitive trade secrets or personal data. Similarly, frameworks employing Attribute-Based Encryption enable fine-grained access control over process data stored on blockchains, ensuring that only authorized entities can verify specific aspects of the supply chain [494].

The deployment of these technologies also facilitates the creation of comprehensive assurance cases for business-critical applications, providing the tangible evidence needed to satisfy regulatory mandates and insurance underwriters. By adopting engineering perspectives rooted in safety-critical systems, such as the Claim-Argument-Evidence (CAE) framework, organizations can systematically justify the dependability and security of their blockchain-based AI systems [495]. This structured approach, supported by automated evidence collection from blockchain ledgers, provides regulators and stakeholders with convincing arguments regarding the system’s resilience. Additionally, the use of digital twins in conjunction with blockchain allows for the simulation of attack scenarios and the validation of defense mechanisms in a risk-free environment, further strengthening the overall security posture [496].

In conclusion, operationalizing trust in the AI software supply chain requires a holistic architecture that marries the immutability of blockchain with the analytical power of automated expert systems. This combination enables continuous, real-time compliance auditing that is both efficient and rigorous, effectively translating the socio-legal and economic strategies of the previous section into actionable technical controls. By automating the detection of vulnerabilities, the verification of provenance, and the enforcement of security policies, organizations can significantly reduce the manual burden on security teams while enhancing the overall resilience of their AI ecosystems. As the field evolves, the integration of incentive mechanisms and decentralized governance will further democratize security, ensuring that trust is a verifiable property of the system rather than a hopeful assumption [497].

9. Open Challenges, Ethical Considerations, and Future Research Directions

9.1 Benchmark Standardization and the Scarcity of Representative Datasets

9.1 Benchmark Standardization and the Scarcity of Representative Datasets

The rapid proliferation of artificial intelligence systems across critical domains has brought the issue of dataset quality and standardization to the forefront of security and ethical research. A fundamental challenge hindering the development of robust, fair, and secure AI software supply chains is the critical lack of standardized benchmarks and the scarcity of representative datasets. Current evaluation practices often rely on a narrow set of legacy datasets that fail to capture the complexity, diversity, and evolving nature of real-world data distributions. This reliance creates a fragile foundation for vulnerability detection and fairness mitigation, as models trained and tested on non-representative data are prone to failure when deployed in heterogeneous environments. The community suffers from a collective “data documentation debt,” characterized by opacity

regarding data origins and sparsity of available information, which severely limits the ability to assess the true reliability of algorithmic fairness findings [498]. Without rigorous documentation and standardization, it becomes nearly impossible to distinguish between genuine model vulnerabilities and artifacts stemming from biased or poorly curated training data.

One of the most pressing issues is the prevalence of “lazy data practices” that inadvertently harm fairness research and obscure potential security risks. Systematic analyses have revealed that common preprocessing steps often lead to the exclusion of minority groups and a lack of representation for certain protected attributes, thereby threatening the generalization of fairness research [499]. When datasets are not representative, the resulting models may appear secure and fair under benchmark conditions but fail catastrophically when encountering underrepresented populations or novel attack vectors in the wild. This problem is exacerbated by the fact that data collection is frequently treated as a one-off practice, where datasets collected for specific purposes are reused for entirely different problems without adequate scrutiny of their suitability or temporal stability [500]. The reuse of such datasets without iterative quality analysis can lead to the amplification of biases and the introduction of subtle vulnerabilities that standard detection methodologies miss.

The scarcity of diverse datasets is particularly acute in high-stakes domains such as healthcare and finance, where the cost of error is substantial. In medical imaging, for instance, the lack of consensus on fairness criteria and the heterogeneity of existing datasets make it difficult to compare the effectiveness of bias mitigation algorithms, leading to inconclusive results regarding model robustness [501]. Similarly, in fraud detection, unfairness often stems from complex interactions between model architectures and specific data biases, yet existing benchmarks rarely provide the granular data necessary to disentangle these factors [502]. The reliance on a few popular datasets, such as Adult, COMPAS, and German Credit, has created a bottleneck where progress is measured against saturated benchmarks that no longer reflect the nuances of modern algorithmic challenges [498]. This saturation discourages the development of novel detection techniques capable of handling the dynamic nature of adversarial threats in the software supply chain.

Furthermore, the interplay between privacy and fairness complicates the creation of representative benchmarks. Strict privacy regulations often prevent the collection and sharing of sensitive attributes, which are essential for evaluating fairness and detecting discriminatory vulnerabilities. However, recent research suggests that it is possible to develop frameworks that operate effectively even when sensitive attributes are private or unavailable, utilizing techniques such as adversarial reweighting or proxy label generation [503]. Despite these advances, the lack of standardized datasets that incorporate privacy-preserving mechanisms while maintaining utility remains a significant gap. The tension between privacy, fairness, and predictive performance creates a “three-way knot” where optimizing for one vector often penalizes the others, a dynamic that is poorly understood due to the absence of comprehensive benchmark suites designed to explore these trade-offs [504].

To address these challenges, the field must move towards the adoption of “Responsible Machine Learning Datasets.” This paradigm shift requires a systematic approach to data collection that prioritizes transparency, inclusivity, and continuous monitoring. Frameworks like FairPrep emphasize the need to promote data to a first-class citizen in the development lifecycle, ensuring that data cleaning methods and hyperparameter tuning are rigorously evaluated to prevent artifacts from skewing fairness results [505]. Moreover, the integration of fairness and privacy into the dataset design phase is crucial. Approaches that learn with impartiality to walk on the Pareto frontier of fairness, privacy, and utility offer a theoretical basis for constructing datasets that do not inherently favor one objective over another, thus providing a more balanced ground for evaluation [506].

Future research directions must focus on establishing standardized protocols for dataset curation that include detailed metadata, provenance tracking, and explicit documentation of potential biases. The FAIR guiding principles (Findable, Accessible, Interoperable, Reusable) should be extended to encompass ethical dimensions, ensuring that datasets are not only technically sound but also socially responsible [507]. Additionally, the development of synthetic data generation techniques and privacy-preserving collaboration frameworks, such as those based on matrix manifolds, can help alleviate data scarcity while protecting user privacy [508]. By fostering a culture of responsible data practices and investing in the creation of diverse, well-documented benchmarks, the AI community can build more resilient software supply chains capable of withstanding both adversarial attacks and inherent societal biases. Only through such holistic standardization can we ensure that vulnerability detection tools and fairness interventions are truly effective across the broad spectrum of real-world applications, thereby laying the necessary empirical groundwork for the ethical autonomous defense strategies discussed in the subsequent section.

9.2 Ethical Frameworks for Autonomous Defense and Human-AI Teaming

9.2 Ethical Frameworks for Autonomous Defense and Human-AI Teaming

Building upon the foundational need for standardized, representative datasets established in the previous section, the deployment of AI within cybersecurity operations precipitates a paradigm shift from manual incident response to autonomous cyber defense. While the diverse and well-documented benchmarks discussed earlier provide the empirical ground for training robust models, the transition to autonomous agents introduces profound ethical challenges regarding accountability, moral agency, and the potential for unintended harm. As organizations increasingly rely on machine learning models to predict attack vectors and execute remediation strategies in real-time, the traditional boundaries of human responsibility become blurred. The central ethical imperative for the future of AI software supply chain security lies in establishing robust frameworks that ensure meaningful human control while leveraging the computational superiority of autonomous systems. This necessitates a move beyond simple automation toward sophisticated human-AI teaming architectures grounded in ethical reasoning and transparent collaboration, serving as the critical bridge between high-quality data inputs and the broader regulatory governance structures discussed subsequently.

A primary concern in autonomous defense is the risk of “security theater” or, conversely, the escalation of conflicts through algorithmic errors. Autonomous cyber defenses, while essential for mitigating risks that exceed human reaction times, introduce the possibility of unintended functional, safety, or moral consequences if their decision-making processes are not rigorously constrained [30]. To address this, the concept of “Violet Teaming” has emerged as a critical methodological framework. Unlike traditional red teaming, which focuses solely on adversarial vulnerability probing, or blue teaming, which emphasizes defensive fortification, violet teaming integrates these approaches with a prioritization of ethics and social benefit [509]. In the context of supply chain security, violet teaming mandates that security agents are not only tested for their ability to detect vulnerabilities but also for their capacity to operate within ethical boundaries, ensuring that defensive actions do not inadvertently cause collateral damage to critical infrastructure or violate privacy norms. This holistic approach fosters the development of reliable and responsible AI systems that are aligned with societal good from the design phase onward, laying the operational groundwork for the decentralized safety mechanisms central to dual governance models.

Central to the operationalization of these ethical frameworks is the concept of “Reflective Hybrid Intelligence.” This model posits that effective security operations cannot rely on fully autonomous agents nor purely human oversight; rather, they require a symbiotic relationship where AI systems possess self-reflective capabilities that allow them to recognize their own limitations and defer to human judgment when necessary [510]. In a hybrid team, the AI agent acts not merely as a tool but as a teammate capable of explaining its rationale, thereby enabling human operators to maintain meaningful control over high-stakes decisions. This is particularly vital in scenarios where sensing failures or ambiguous data—often stemming from the dataset limitations highlighted previously—could lead to catastrophic errors. Research indicates that hybrid systems employing reinforcement learning managers can dynamically delegate tasks between human and autonomous agents based on contextual confidence levels, significantly outperforming isolated agents [511]. By continuously evaluating the likelihood of failure for both human and machine counterparts, such systems ensure that control is assigned to the entity best suited for the specific environmental context, thereby enhancing overall team resilience.

However, the efficacy of human-AI teaming is contingent upon trust, which is often eroded by the “black box” nature of deep learning models. To bridge this gap, ethical frameworks must prioritize transparency and explainability. Studies have shown that providing humans with option-centric rationale displays—where the AI explicates the reasoning behind its proposed actions—significantly calibrates trust and improves task performance [512]. Furthermore, the architecture of these systems must support accountability mechanisms that prevent the shifting of moral blame to machines. There is a documented psychological tendency for human decision-makers to offload moral responsibility when delegating to artificial agents, a phenomenon known as “hiding behind machines” [513]. Counteracting this requires designing systems where human responsibility remains commensurate with their authority and ability to intervene, ensuring that the deployment of autonomous defense does not create a moral vacuum [514]. This focus on preserving human agency directly informs the liability and accountability structures required in the emerging dual governance landscape.

Looking forward, the development of ethical autonomous defense agents must incorporate mechanisms for learning and adapting to social norms and moral uncertainties. Approaches such as reinforcement learning under moral uncertainty allow agents to navigate situations where ethical theories conflict, preventing extreme behaviors driven by rigid adherence to a single moral framework [515]. Additionally, the integration of watchdog AI agents dedicated to lifelong functional testing can provide a continuous layer of oversight, shepherding autonomous systems throughout their operational lifecycle [516]. Ultimately, the future of secure software supply chains depends on the successful convergence of technical robustness and ethical maturity. By adopting frameworks like violet teaming and reflective hybrid intelligence, the cybersecurity community can ensure that autonomous agents serve as force multipliers for human expertise rather than uncontrollable variables. These ethical autonomous defense strategies provide the necessary normative and operational foundation for the broader “Dual Governance” models and regulatory alignment strategies detailed in the following section, ensuring that as the industry scales, it does so with both technical agility and societal responsibility.

9.3 Dual Governance Models for Generative AI and Regulatory Alignment

9.3 Dual Governance Models for Generative AI and Regulatory Alignment

While the preceding section established the necessity of ethical frameworks and reflective hybrid intelligence for autonomous defense agents, the governance of the broader Generative AI (GenAI)

ecosystem requires a parallel evolution in regulatory architecture. The rapid proliferation of GenAI has created a complex landscape characterized by a fundamental tension between top-down centralized state control and bottom-up decentralized community safety mechanisms. As GenAI systems become increasingly embedded in societal infrastructure—and by extension, the software supply chain—the inadequacy of relying solely on either approach has become evident. Centralized regulations, while possessing the force of law, often struggle with the pace of technological evolution, lacking the clarity, uniformity, and interoperability required to address cross-jurisdictional risks effectively. Conversely, decentralized protections driven by crowdsourced safety tools and community norms offer agility but frequently suffer from deficiencies in oversight adequacy and the enforcement of ethical standards. To bridge this dichotomy, the concept of a “Dual Governance” framework has emerged as a critical research direction, proposing a cooperative synergy between government mandates and community-developed safety mechanisms to manage risks such as misinformation, liability, and algorithmic harm [517].

The necessity for such a hybrid model stems from the limitations inherent in current regulatory paradigms. Existing and proposed centralized regulations by governments have faced significant criticism for potentially restricting innovation, hindering free market competition, and failing to provide sufficient clarity across different legal jurisdictions [517]. The global divide in AI regulation further complicates this issue, with some regions adopting horizontal approaches that treat AI homogeneity, while others pursue context-specific modular approaches tailored to particular use cases [518]. This fragmentation creates uncertainty for developers and deployers within the supply chain, highlighting the need for a governance structure that can harmonize diverse regulatory objectives while maintaining the flexibility to adapt to specific application contexts. A Dual Governance model addresses this by bifurcating the AI lifecycle into learning and utilization phases, assigning distinct regulatory objectives to each, and promoting the adoption of international industry standards that convert abstract principles into quantifiable metrics [518].

Central to the operationalization of the Dual Governance framework is the integration of “Violet Teaming,” a methodology that directly extends the ethical defense concepts discussed previously. This integrative approach combines adversarial vulnerability probing (red teaming) with solutions for safety and security (blue teaming), all while prioritizing ethics and social benefit. Unlike traditional red teaming which focuses primarily on identifying flaws, or blue teaming which focuses on defense, violet teaming emerges from AI safety research to manage risks proactively by design, ensuring that ethical considerations are embedded throughout the development process [509]. Within the Dual Governance model, this methodology serves as the operational engine for the decentralized component, empowering community researchers and independent auditors to identify emerging threats that centralized bodies may overlook due to resource constraints or lack of technical specificity. By formalizing the role of independent evaluation and red teaming through legal and technical safe harbors, societies can indemnify public interest safety research, protecting it from legal reprisal and fostering a more inclusive environment for uncovering model risks [519].

Furthermore, the alignment of these governance layers requires a re-evaluation of liability and accountability structures, echoing the concerns regarding moral agency raised in the context of autonomous agents. Current legal frameworks often fail to address the unique challenges posed by generative models, particularly regarding the attribution of causation in cases of defamation or product liability [520]. A robust Dual Governance model must therefore differentiate between endogenous and exogenous sources of potential harm, carefully allocating liability between developers and deployers to ensure that firms are incentivized to understand their inputs and maintain the ability to retrain systems to remedy breaches [521]. This approach moves beyond the “half-hearted”

measures of current proposals that rely heavily on disclosure mechanisms, advocating instead for a comprehensive framework that includes sustainability impact assessments and provisions to support innovation while ensuring fairness [62].

The implementation of Dual Governance also necessitates the involvement of a broad spectrum of stakeholders, including governments, corporations, and citizens, operating within a multilevel governance approach grounded in dimensions of trust such as competence, integrity, and benevolence [522]. This participatory element is crucial for democratic AI governance, as frameworks like “Particip-AI” demonstrate the value of gathering diverse public opinions on AI use cases and harms, which often contrast with the business-focused priorities of current development trends [523]. By incorporating these diverse perspectives, Dual Governance can better anticipate future risks and ensure that regulatory measures reflect societal values rather than solely corporate or state interests. Moreover, the framework supports the creation of an international consortium for evaluations of societal-scale risks, facilitating coordinated efforts to mitigate catastrophic risks from advanced AI systems through shared standards and responsible scaling policies [524].

Ultimately, the transition toward Dual Governance represents a shift from reactive regulation to proactive, adaptive management, laying the normative groundwork necessary for securing the supply chain against future technological shifts. It acknowledges that while centralized regulation provides the necessary legal backbone and enforcement mechanisms, decentralized community safety offers the agility and technical depth required to keep pace with rapid AI advancements. By synthesizing these approaches, stakeholders can create a resilient ecosystem where innovation is promoted alongside rigorous safety standards. This includes establishing clear protocols for coordinated flaw disclosure tailored to algorithmic harms, moving beyond traditional software vulnerability disclosure to address the non-deterministic nature of AI failures [61]. As the field evolves, the success of Dual Governance will depend on the continuous refinement of these collaborative mechanisms, ensuring that both state actors and community stakeholders remain aligned in their pursuit of trustworthy, equitable, and safe Generative AI systems. This alignment is particularly critical as the industry pivots to address the next horizon of threats: the convergence of AI with quantum computing, which demands not only regulatory agility but also fundamental cryptographic resilience.

9.4 Resilience Against Quantum Threats and Next-Generation Cryptography

9.4 Resilience Against Quantum Threats and Next-Generation Cryptography

Building upon the Dual Governance frameworks established in the preceding section, which emphasize the need for adaptive and proactive management of AI risks, the conversation must now pivot to the most imminent existential threat to cryptographic integrity: the convergence of Quantum Computing (QC) and Artificial Intelligence. This convergence presents a paradigm shift in the security landscape of the software supply chain, introducing unprecedented risks that necessitate an immediate and strategic transition to next-generation cryptography. As quantum hardware matures, the theoretical threat of “harvest now, decrypt later” attacks becomes a tangible reality for software artifacts, model weights, and proprietary training data currently protected by classical public-key cryptosystems. The fundamental vulnerability lies in the reliance of modern digital infrastructure on integer factorization and discrete logarithm problems, which are susceptible to Shor’s algorithm once sufficiently large, fault-tolerant quantum computers are realized [525]. This impending capability threatens to render current encryption standards obsolete, thereby compromising the integrity of the entire AI software supply chain, from code repositories to deployed

inference engines [526].

The urgency of this transition is underscored by the dual nature of quantum advancement: while it offers transformative potential for optimizing supply chain logistics and enhancing machine learning models, it simultaneously empowers adversaries with the ability to break foundational security protocols [527]. The integration of AI into cyber-physical systems and critical infrastructure further exacerbates these risks, as compromised cryptographic primitives could allow attackers to manipulate autonomous agents or poison data pipelines with undetectable backdoors [528]. Consequently, resilience against quantum threats is not merely a cryptographic upgrade but a holistic requirement for maintaining trust in AI-driven ecosystems, serving as a necessary precursor to the sustainable architectures discussed subsequently. Without addressing these cryptographic fragilities, the energy-intensive security measures of the future would be built upon foundations capable of being silently dismantled.

To address these challenges, the global security community, led by the National Institute of Standards and Technology (NIST), has embarked on a rigorous standardization process for Post-Quantum Cryptography (PQC). This initiative aims to replace vulnerable algorithms with quantum-resistant alternatives based on lattice-based, code-based, hash-based, and multivariate cryptographic problems [526]. The selection of algorithms such as CRYSTALS-Kyber for key encapsulation and CRYSTALS-Dilithium for digital signatures marks a critical milestone in this roadmap [529]. However, the migration to these new standards is fraught with complexity, particularly within the AI supply chain where dependencies are deep and heterogeneous. The transition requires more than simple algorithm swapping; it demands a re-engineering of cryptographic agility to ensure systems can adapt to future cryptanalytic breakthroughs without catastrophic failure [525].

A significant hurdle in adopting PQC for AI systems is the performance overhead associated with larger key sizes and signature lengths, which can strain resource-constrained environments such as IoT edge devices and embedded controllers common in industrial AI deployments [530]. Research indicates that while lattice-based schemes offer robust security, their implementation on low-power microcontrollers requires careful hardware-software co-design to meet strict energy and latency constraints [531]. Furthermore, the computational cost of PQC algorithms can impact the throughput of high-frequency trading models or real-time anomaly detection systems, necessitating optimized implementations and hardware accelerators [532]. Recent studies have demonstrated that specialized ASICs and FPGA implementations can significantly reduce the power consumption and area footprint of PQC operations, making them viable for widespread deployment in the AI supply chain [533]. Notably, this intersection of cryptographic hardness and energy efficiency foreshadows the critical sustainability challenges detailed in the following section, where the ecological cost of such computationally heavy defenses becomes a primary concern.

Beyond algorithmic replacement, the supply chain must address implementation-specific vulnerabilities, such as side-channel attacks, which remain a potent threat even for quantum-resistant algorithms [534]. The physical realization of cryptographic primitives on quantum and classical hardware introduces new attack surfaces, including timing side-channels in cloud-based quantum services that could leak sensitive algorithmic details [134]. Therefore, a comprehensive defense strategy must incorporate formal verification and constant-time implementations to mitigate these risks. Additionally, the concept of “cryptographic agility” must be embedded into the AI development lifecycle, allowing for seamless updates to cryptographic modules as new vulnerabilities are discovered or standards evolve [535].

The roadmap for achieving resilience involves a multi-phased approach analogous to the Y2K remediation effort, requiring systematic inventory, risk assessment, testing, and deployment of quantum-safe solutions across all layers of the software stack [536]. Organizations must begin by identifying assets with long shelf-lives, such as encrypted datasets and signed model artifacts, which are immediately vulnerable to harvest-now-decrypt-later attacks [537]. This process entails upgrading communication protocols like TLS and SSH to support hybrid modes that combine classical and post-quantum algorithms, ensuring backward compatibility while establishing forward secrecy [529]. Moreover, the integration of PQC into blockchain networks is essential for securing decentralized AI marketplaces and ensuring the immutability of supply chain provenance records [538].

Looking ahead, the synergy between AI and quantum computing may also yield defensive advantages. Quantum Machine Learning (QML) holds the potential to enhance adversarial robustness and detect sophisticated attacks that evade classical detectors, although current Noisy Intermediate-Scale Quantum (NISQ) devices face limitations in accuracy and scalability [539]. Experimental demonstrations have shown that quantum classifiers can be trained to resist adversarial perturbations, suggesting a future where quantum-enhanced defenses protect AI models against quantum-enabled adversaries [540]. However, realizing this potential requires overcoming significant hardware constraints and developing error-mitigation techniques suitable for near-term devices [541].

In conclusion, building resilience against quantum threats in the AI software supply chain is a critical imperative that demands proactive collaboration between cryptographers, AI researchers, and industry practitioners. The adoption of NIST-approved algorithms is just the first step; a sustained effort is required to optimize these solutions for diverse computing environments, harden implementations against side-channel attacks, and foster a culture of cryptographic agility. By navigating this transition strategically, the global community can secure the foundations of the AI era against the looming quantum horizon. Yet, as we fortify these digital defenses, we must remain cognizant that the computational intensity required for such security measures introduces a new set of challenges regarding environmental viability. Thus, securing the cryptographic layer is inextricably linked to the next pillar of our discourse: ensuring that these advanced security architectures do not come at an unacceptable cost to planetary health [542].

9.5 Sustainable AI and the Environmental Impact of Secure Supply Chains

9.5 Sustainable AI and the Environmental Impact of Secure Supply Chains

While the preceding section detailed the cryptographic hardening required to withstand quantum threats, and the following section explores autonomous self-healing mechanisms, a critical third pillar underpins the long-term viability of these advanced security architectures: environmental sustainability. The rapid proliferation of artificial intelligence within software supply chains has introduced a dimension to security discourse that is often overlooked yet increasingly urgent: the ecological cost of maintaining robust, secure AI operations. As organizations integrate complex models, large language models (LLMs), and automated vulnerability detection systems to defend against evolving threats, the associated carbon footprint and resource consumption have escalated to unprecedented levels. The Information and Communication Technology (ICT) sector already contributes significantly to global greenhouse gas emissions, surpassing the aviation industry, yet current regulatory frameworks and security protocols frequently neglect the ecological implications of AI deployment [543]. Consequently, the future of AI software supply-chain security must be inextricably linked with sustainability, necessitating a paradigm shift toward “Sustainable AI Regulation” that mandates carbon footprint transparency and embeds energy-efficient principles

directly into security architectures.

The environmental burden of AI is multifaceted, spanning the entire lifecycle from data collection and model training to inference and continuous monitoring. In the context of supply-chain security, where real-time anomaly detection, adversarial training, and automated patch generation are becoming standard defenses, the computational demands are immense. Training large-scale generative models, which are increasingly employed to identify novel vulnerabilities or generate secure code, consumes vast amounts of energy and water. For instance, the training of a single large model can result in the evaporation of hundreds of thousands of liters of freshwater, a statistic that remains largely opaque in current security vendor disclosures [544]. Furthermore, the operational phase of these models, particularly when deployed for continuous supply-chain monitoring across global infrastructure to detect sophisticated attacks, contributes to a super-linear growth in energy consumption that threatens to undermine global climate goals [545].

A primary challenge in addressing this issue is the “abstraction gap” between high-level ethical principles and the practical implementation of secure systems. While many organizations espouse commitments to responsible AI, there is a distinct lack of concrete metrics and enforcement mechanisms to quantify the environmental impact of security operations [546]. Current security frameworks often prioritize performance and detection accuracy at the expense of efficiency, leading to a rebound effect where improvements in model capabilities drive increased complexity and resource usage [547]. This trend is exacerbated by the competitive nature of the AI industry, where the race for domain supremacy encourages the development of ever-larger models without adequate consideration for their ecological toll [548]. Such inefficiencies pose a direct risk to the resilience of the supply chain, as -intensive security measures may become unsustainable or economically unviable in the long term.

To mitigate these risks, the concept of “Sustainable AI Regulation” must be operationalized within the security domain. This involves moving beyond voluntary guidelines to enforceable standards that require transparency regarding the carbon and water footprints of AI systems used in supply-chain defense. Regulatory bodies and industry consortia must mandate the disclosure of environmental metrics alongside traditional security performance indicators, similar to the proposed requirements in the EU AI Act [543]. Such transparency would enable stakeholders to make informed decisions, favoring security solutions that balance robustness with energy efficiency. Moreover, the development of standardized tools for measuring the “greenness” of AI models is essential. Frameworks like the Sustainability Criteria and Indicators for Artificial Intelligence Systems (SCAIS) provide a foundational structure for evaluating AI systems across environmental, social, and economic dimensions, offering a pathway to integrate these criteria into security procurement and deployment processes [549].

Technologically, the integration of sustainability into security architectures requires innovative approaches to model design and deployment that complement the cryptographic and self-healing strategies discussed elsewhere in this survey. Techniques such as compute-efficient machine learning, which utilizes compressed network architectures with marginal accuracy trade-offs, offer a viable strategy for reducing the energy footprint of vulnerability detection systems [550]. Additionally, the adoption of federated learning can distribute computational loads across edge devices, leveraging idle capacity to perform security tasks without concentrating energy consumption in massive data centers, thereby aligning with the decentralized nature of modern supply chains [550]. The use of specialized hardware, such as Field Programmable Gate Arrays (FPGAs), which have been shown to possess a lower carbon footprint than Application-Specific Integrated Circuits (ASICs) for certain applications, further presents an opportunity to optimize the hardware layer of secure

AI supply chains [551].

Looking forward, the research community must address the scarcity of “Green AI” tools specifically tailored for security applications. While academic research has demonstrated significant energy savings through hyperparameter tuning and algorithmic optimization, these findings have not yet been fully ported to industrial security practices [552]. Future directions should focus on developing “Sustainability-by-Design” principles for MLSecOps, ensuring that energy efficiency is a core requirement rather than an afterthought in the development of secure AI systems [543]. This includes exploring continual learning approaches that allow models to adapt to new threats without the need for energy-intensive retraining from scratch, a capability that parallels the adaptive fault detection mechanisms discussed in subsequent sections [553]. Furthermore, the implementation of geographical load balancing strategies can help mitigate environmental inequity by routing inference requests to regions with cleaner energy grids, thereby reducing the regional disparity in carbon and water footprints [554].

Ultimately, achieving a sustainable AI software supply chain requires a holistic convergence of regulatory pressure, technological innovation, and ethical governance. By embedding environmental considerations into the fabric of security architectures, the industry can ensure that the defenses protecting our digital infrastructure—whether against quantum decryption or physical system faults—do not come at the unacceptable cost of planetary health. As noted in recent analyses, addressing the dual challenges of digital transformation and climate change mitigation is not merely an optional add-on but a fundamental necessity for the long-term viability of AI technologies [543]. Only by securing the environmental foundation of our computational ecosystems can we ensure the resilience of the AI-driven supply chains of the future.

9.6 Self-Healing Architectures and Adaptive Fault Detection in Cyber-Physical Systems

9.6 Self-Healing Architectures and Adaptive Fault Detection in Cyber-Physical Systems

While Section 9.5 highlighted the urgent need to align security architectures with environmental sustainability, the operational realization of such resilient systems demands a parallel evolution in autonomy. Moving beyond the resource constraints and efficiency metrics discussed previously, this section addresses the dynamic mechanisms required to maintain system integrity in the face of inevitable faults and attacks. The evolution of cyber-physical systems (CPS) from static, rule-based infrastructures to dynamic, self-healing ecosystems represents a paradigm shift essential for securing the modern AI software supply chain. As supply chains become increasingly decentralized and integrated with physical operations, the ability to autonomously detect, diagnose, and recover from faults without human intervention is no longer optional but a critical requirement for resilience. This transition relies heavily on the convergence of Digital Twins (DTs), adaptive knowledge bases, and advanced machine learning techniques that enable real-time anomaly detection and autonomous recovery mechanisms. The foundational concept involves creating a symbiotic relationship between the physical asset and its virtual counterpart, where the DT serves not merely as a passive mirror but as an active agent in the security and maintenance lifecycle.

Central to this architectural evolution is the deployment of Digital Twins as dynamic defense mechanisms and attack simulators. These virtual replicas allow for the continuous assimilation of data to predict behaviors and optimize system performance, effectively shifting security operations from reactive to proactive stances. For instance, recent advancements demonstrate how DTs can be

utilized to virtually replicate complex physical systems, enabling the optimization of performance by controlling system actions through continual data assimilation [555]. Furthermore, the integration of DTs facilitates a “Shift Left and Shift Right” cybersecurity operations paradigm, where security is embedded during the design phase and continuously monitored at runtime. This approach ensures that threat insights are derived from high-reliability data feeds, bridging the gap between design-time vulnerability assessment and runtime intrusion detection [320].

A critical component of self-healing architectures is the utilization of adaptive knowledge bases that model system variables and their interdependencies under conditions of uncertainty. In complex CPS environments, faults are often masked by noise, asynchronous measurements, and time delays. To address this, researchers have developed frameworks that encode system relations in probabilistic logic languages, such as Prolog/ProbLog, to generate adaptive runtime monitors. These monitors exploit implicit redundancy across space and time to compare uncertain signals, thereby triggering recovery processes only when genuine faults are distinguished from environmental variances [556]. This capability is vital for supply chains where sensor failures or malicious data injections could otherwise lead to cascading systemic collapses.

The mechanism of self-healing also extends to the realm of reinforcement learning and autonomous decision-making. Traditional reinforcement learning approaches often struggle with the randomness of disturbances and the complexity of real-world scenarios, leading to inefficient training cycles. To overcome these limitations, hybrid imitation learning frameworks have been proposed, where intelligent agents learn optimal restorative policies by interacting with expert systems. This method significantly improves training efficiency and enables agents to handle complex discrete-continuous action spaces required for network reconfiguration and power dispatch in distribution systems [557]. Moreover, the challenge of environmental drift in production settings—where continual learning models may suffer from catastrophic forgetting—is being addressed through novel mechanisms like intentional forgetting. By deliberately erasing minor behaviors to prioritize key problem-solving skills, systems can achieve faster convergence and more effective adaptation to drifted environments [558].

In the context of the software supply chain specifically, the concept of the Digital Twin is expanding to include cognitive models capable of detecting disruptions in logistics and manufacturing flows. Hybrid deep learning approaches, combining autoencoders with support vector machines, are being employed within cognitive digital supply chain twins to detect disruptions in real-time. These systems not only identify the echelon where a disruption occurs but also predict the time-to-recovery, allowing decision-makers to mitigate negative impacts swiftly [559]. Similarly, in industrial IoT settings, self-labeling methods enabled by interactive causality are allowing ML models to adapt personally to data distribution shifts post-deployment, ensuring that the software components governing physical processes remain robust against evolving threats [560].

However, the realization of fully autonomous self-healing supply chains faces significant challenges, particularly regarding the evaluation of these systems. The lack of systematic methods to assess the correct behavior of self-adaptive systems in highly dynamic environments remains a gap. Frameworks like CHESS, which leverage chaos engineering to inject faults and evaluate self-healing responses, are emerging as essential tools for validating the resilience of these architectures before deployment [561]. Additionally, the dilemma between fault correction and fault tolerance in distributed systems requires careful modeling to avoid triggering new faults while attempting to heal existing ones [562].

Looking forward, while the integration of blockchain technology with Digital Twins offers a promis-

ing avenue for ensuring the integrity of the data driving these self-healing mechanisms, it also introduces the next layer of architectural complexity. By recording cross-validated sensory data on an immutable ledger, supply chain stakeholders can trust the inputs feeding the AI models responsible for fault diagnosis and recovery [496]. Furthermore, the development of trustworthy distributed solutions that combine AI with blockchain can safeguard product lifecycle data, ensuring that the self-healing actions taken by autonomous agents are based on verified and untampered information [563]. Yet, as these autonomous agents rely increasingly on external data sources for their healing logic, the provenance and immutability of that data become paramount. This necessity sets the stage for the subsequent discussion on decentralized trust mechanisms, where we explore how blockchain-enabled provenance systems provide the cryptographic foundation required to secure the very data streams that power these self-healing ecosystems. As the field matures, the focus must shift towards creating standardized benchmarks and evaluation metrics that account for the stochastic nature of self-healing processes, ensuring that future supply chains are not only automated but genuinely resilient and capable of surviving sophisticated adversarial attacks.

9.7 Decentralized Trust Mechanisms and Blockchain-Enabled Provenance

9.7 Decentralized Trust Mechanisms and Blockchain-Enabled Provenance

While the self-healing architectures discussed in the previous section provide robust mechanisms for detecting and recovering from faults within cyber-physical systems, their efficacy fundamentally relies on the integrity of the data driving these autonomous decisions. If the input data or the model components themselves are compromised, even the most sophisticated self-healing agent may optimize for incorrect objectives or execute malicious recovery policies. To address this foundational vulnerability, the AI software supply chain requires a shift from implicit trust in intermediaries to explicit, cryptographically verifiable trust embedded within the system architecture. Consequently, decentralized ledger technologies (DLT) and blockchain-enabled provenance systems have emerged as critical complements to self-healing ecosystems, offering immutable audit trails that secure the lineage of AI assets from creation to deployment.

One of the primary contributions of blockchain technology is the establishment of an unalterable historical record that tracks the provenance of data and models, a capability essential for accountability in high-stakes environments. In scenarios such as autonomous vehicle operations or industrial manufacturing, reconstructing the history of an asset is paramount for liability apportionment and forensic analysis. For instance, blockchain-based systems can store time-stamped sensor records to definitively define liability among intertwined subjects, preventing the manipulation of evidence even when deepfakes or synthetic media are involved [564]. Similarly, in smart manufacturing, DLT-based frameworks like SmartQC provide a secure backbone for trusted data workflows, ensuring data integrity through immutable ledgers and automating business logic via smart contracts [373]. These systems mitigate the “black box” nature of AI by providing a transparent, tamper-proof record of every transaction, update, and access event, thereby facilitating rigorous post-incident forensics. A distributed and stratified “black box” audit trail has also been proposed specifically for automotive software and data provenance to assure vehicular software integrity and reliability [565].

Beyond passive record-keeping, decentralized mechanisms actively enforce governance policies through smart contracts, automating compliance and access control without human intervention. This capability is particularly vital for managing intellectual property and data rights in the era of generative AI. The DECORAIT framework exemplifies this by establishing a decentralized registry

where creators can assert their right to opt-in or opt-out of AI training datasets, utilizing visual fingerprinting and DLT to trace data provenance and ensure fair remuneration [566]. Such systems democratize safety by granting stakeholders direct control over their data contributions, mitigating risks of unauthorized scraping and copyright infringement. Furthermore, smart contracts can enforce “specification is law” paradigms, permitting code upgrades or model updates only if they meet formally verified specifications, thus preventing the introduction of vulnerable components into the supply chain [567]. Additionally, graph-based provenance models implemented as smart contracts have been proposed to trace AI assets along complete value chains, solving challenges related to trust, privacy, and fair remuneration [568].

The integration of blockchain with AI also enables decentralized governance models capable of adapting to rapidly evolving threats. Decentralized Autonomous Organizations (DAOs) offer a novel structure for collective decision-making regarding AI safety standards and resource allocation. However, the effectiveness of these organizations depends on robust mechanisms to prevent governance attacks, such as Sybil identities. Research has shown that graph deep learning techniques can be effectively employed within DAO frameworks to identify spurious identities and secure the voting process, ensuring that governance decisions reflect the genuine consensus of the community [569]. Moreover, the concept of “Auto.gov” demonstrates how reinforcement learning can be integrated into on-chain governance for DeFi protocols to propose automated, data-driven parameter adjustments, enhancing resilience against market manipulation and oracle attacks—a methodology that holds significant promise for dynamic AI model governance [570]. Studies on DeFi governance have further highlighted vulnerabilities such as voting Sybil attacks and proposal front-running, emphasizing the need for such automated and secure governance mechanisms [571].

Despite these advancements, the deployment of decentralized trust mechanisms faces significant hurdles regarding scalability, privacy, and the nuanced definition of decentralization itself. A stratified analysis reveals that decentralization is not a binary state but a multi-layered property encompassing hardware, software, consensus, and governance, each with distinct vulnerabilities that must be addressed to ensure true system resilience [572]. In resource-constrained environments like IoT networks, direct integration of heavy blockchain protocols is often impractical. Solutions such as Federated Ledgers offer a hierarchical approach, combining efficient private intra-ledgers with public inter-ledgers to balance scalability, privacy, and security, making them suitable for large-scale data marketplaces and distributed AI training [379]. Additionally, the tension between transparency and privacy remains a critical area for research; while immutable ledgers provide visibility, they can inadvertently expose sensitive information. Emerging techniques involving zero-knowledge proofs and privacy-preserving audit trails are essential to reconcile these conflicting requirements, allowing for the verification of AI asset integrity without revealing proprietary model weights or sensitive training data [363].

Looking forward, the convergence of blockchain, AI, and formal verification methods promises to create self-healing supply chains where trust is algorithmic rather than institutional. The vision of “Safe Artificial General Intelligence via Distributed Ledger Technology” posits that DLT is integral to aligning AGI values with human ethics, providing the necessary infrastructure for remote shutdowns, component identification, and encrypted morality constraints [573]. As the field matures, future research must focus on standardizing interoperable provenance protocols, optimizing consensus mechanisms for low-latency AI inference, and developing legal frameworks that recognize smart contract-based governance. Process mining has also been identified as a useful framework for gaining visibility into AI compliance processes, surfacing bottlenecks, and monitoring uncertainty in regulatory compliance, which complements the immutable records provided by blockchain [574].

Ultimately, while these decentralized mechanisms provide the technical infrastructure for verified trust and automated governance, they set the stage for the next critical challenge: bridging the gap between these technical capabilities and the abstract ethical principles that guide them, a disconnect that remains a major barrier to truly trustworthy AI.

9.8 Addressing the Principle-Implementation Gap in Global AI Ethics

9.8 Addressing the Principle-Implementation Gap in Global AI Ethics

The decentralized trust mechanisms and blockchain-enabled provenance systems discussed in the previous section provide a robust technical infrastructure for verifying data lineage, enforcing governance via smart contracts, and ensuring algorithmic accountability. However, while these technologies secure the “how” of AI operations through immutable records and automated compliance, they do not inherently resolve the “why” or the “what” of ethical decision-making. The proliferation of artificial intelligence has been accompanied by an unprecedented surge in the formulation of high-level ethical guidelines from governments, international organizations, and private entities, centering on principles such as transparency, fairness, accountability, and non-maleficence. Yet, a critical disconnect remains between these abstract aspirations and their tangible operationalization within the very software supply chains and deployment pipelines that blockchain seeks to secure. This “principle-implementation gap” represents one of the most significant barriers to achieving truly trustworthy AI. While consensus appears to be emerging around a core set of values, the abstract nature, diversity, and context-dependency of these principles make them notoriously difficult to translate into concrete engineering practices, resulting in a landscape where ethical commitments often fail to manifest in actual system behavior [575]. The challenge is not merely technical but socio-technical, requiring a fundamental shift from principled declarations to actionable, measurable, and enforceable mechanisms embedded throughout the AI lifecycle.

A primary driver of this gap is the lack of proven methods to translate static principles into dynamic software engineering processes. Unlike established professions such as medicine, which possess deep historical norms, fiduciary duties, and robust legal accountability mechanisms, the AI industry lacks a unified professional history that effectively binds practitioners to ethical outcomes [576]. Consequently, AI professionals often navigate ethical dimensions “by sight,” relying on high-level statements that offer little guidance for low-level implementation strategies [577]. This disconnect is exacerbated by the fact that many existing interventions, such as checklists or toolkits, may inadvertently limit critique to the scope of the intervention itself, rendering broader systemic concerns illegitimate and failing to address the power dynamics that influence ethical decision-making within development teams [578]. To bridge this divide, the field must move beyond top-down imposition of abstract rules and instead foster approaches that generate ethical insights from the grounded experiences of practitioners working on tangible human problems [579].

One promising strategy for operationalizing ethics involves the adoption of enhanced well-being assessments as a foundational metric for evaluating AI systems. Moving beyond narrow technical metrics like accuracy or fairness scores, this approach advocates for a holistic evaluation of how autonomous and intelligent systems impact human livelihoods and societal flourishing. By adopting a lens motivated by regulatory impact assessments and the well-being movement in public policy, organizations can iteratively assess identified impacts, include emerging trade-offs, and navigate complex consequences that simple metrics might miss [580]. The IEEE 7010 standard exemplifies

this shift, providing a structured methodology for assessing the impact of AI on human well-being throughout the system lifecycle, thereby offering practitioners concrete guidance on integrating social and ethical considerations into design and procurement [581]. Furthermore, frameworks that prioritize stakeholder engagement and anticipatory reflection, such as the SUM Values and Stakeholder Engagement Process, enable project teams to steer innovation practices toward responsible outcomes that are responsive to transformative societal effects [582].

Interdisciplinary collaboration serves as another critical pillar in closing the implementation gap. Ethical AI cannot be achieved through technical solutions alone; it requires the integration of diverse perspectives from law, sociology, psychology, and domain-specific experts to contextualize principles within specific cultural and operational environments. Research indicates that ethical AI practices must be understood within their specific cultural and historical contexts, suggesting that a one-size-fits-all approach is insufficient for global deployment [583]. Effective governance models must therefore involve interdependent stakeholders—including governments, corporations, and citizens—to build trust through dimensions of competence, integrity, and benevolence [522]. Moreover, fostering a “third space” where the operational language of practitioners can merge with abstract ethical principles is essential. Methodologies centered on dialog and mutualized responsibility can help practitioners internalize ethical responsibility, bridging the gap between structural decision-making power and the sharp end of the ecosystem where code is written [584].

To further solidify the transition from theory to practice, the development of actionable policy frameworks and argument-based assurance cases is necessary. Actionable principles for AI policy require preliminary landscape assessments, multi-stakeholder participation, and mechanisms to support implementation, ensuring that ethical guidelines are not merely rhetorical but enforceable [585]. Additionally, the methodology of “ethical assurance” offers a structured means to unify practical mechanisms by building upon process-based project governance that supports inclusive deliberation while remaining grounded in social and technical realities [586]. This approach allows organizations to construct arguments demonstrating the ethical acceptability of specific AI systems in complex socio-technical contexts, thereby fostering confidence among stakeholders [587].

Ultimately, addressing the principle-implementation gap demands a cultural and procedural transformation within the AI community. It requires recognizing that principles alone cannot guarantee ethical outcomes and that a holistic consideration of ethics in development and deployment is necessary to navigate inconsistent regulatory pressures and conflicting business goals [588]. By integrating enhanced well-being assessments, fostering deep interdisciplinary collaboration, and adopting rigorous assurance frameworks, the global AI community can begin to close the chasm between aspirational ethics and the reality of deployed systems. This transition is vital not only for mitigating risks but also for ensuring that the transformative potential of AI is harness responsibly for the common good, aligning technological advancement with fundamental human values and societal needs [546]. Only through such comprehensive and grounded efforts can the promise of ethical AI be realized in practice, ensuring that the systems shaping our future are as morally robust as they are computationally powerful.

References

- [1] nuts-flow ml data pre-processing for deep learning
- [2] On Leakage in Machine Learning Pipelines
- [3] Managing ML Pipelines Feature Stores and the Coming Wave of Embedding Ecosystems

- [4] PeaTMOSS A Dataset and Initial Analysis of Pre-Trained Models in Open-Source Software
- [5] Deciphering the Impact of Pretraining Data on Large Language Models through Machine Un-learning
- [6] Demystifying Dependency Bugs in Deep Learning Stack
- [7] Who Needs MLOps What Data Scientists Seek to Accomplish and How Can MLOps Help
- [8] Characterization of Large Language Model Development in the Datacenter
- [9] Teaching Code LLMs to Use Autocompletion Tools in Repository-Level Code Generation
- [10] CodeChain Towards Modular Code Generation Through Chain of Self-revisions with Representative Sub-modules
- [11] LLMs with User-defined Prompts as Generic Data Operators for Reliable Data Processing
- [12] A review of SolarWinds attack on Orion platform using persistent threat agents and techniques for gaining unauthorized access
- [13] S3C2 Summit 2202-09 Industry Secure Supply Chain Summit
- [14] The Race to the Vulnerable Measuring the Log4j Shell Incident
- [15] What are Weak Links in the npm Supply Chain
- [16] Backstabber’s Knife Collection A Review of Open Source Software Supply Chain Attacks
- [17] AI Data poisoning attack Manipulating game AI of Go
- [18] Poisoned ChatGPT Finds Work for Idle Hands Exploring Developers’ Coding Practices with Insecure Suggestions from Poisoned AI Models
- [19] TrojanPuzzle Covertly Poisoning Code-Suggestion Models
- [20] Not what you’ve signed up for Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection
- [21] S3C2 Summit 2023-06 Government Secure Supply Chain Summit
- [22] Decoding the Threat Landscape ChatGPT, FraudGPT, and WormGPT in Social Engineering Attacks
- [23] Impacts and Risk of Generative AI Technology on Cyber Defense
- [24] LLM Agents can Autonomously Hack Websites
- [25] LLM Agents can Autonomously Exploit One-day Vulnerabilities
- [26] Here Comes The AI Worm Unleashing Zero-click Worms that Target GenAI-Powered Applications
- [27] Escalation Risks from Language Models in Military and Diplomatic Decision-Making
- [28] RANK AI-assisted End-to-End Architecture for Detecting Persistent Attacks in Enterprise Networks
- [29] ATHAFI Agile Threat Hunting And Forensic Investigation
- [30] Autonomous Cyber Defense Introduces Risk Can We Manage the Risk

- [31] Hybrid Deep Learning Model using SPCAGAN Augmentation for Insider Threat Analysis
- [32] Generative AI for Secure Physical Layer Communications A Survey
- [33] Spear or Shield Leveraging Generative AI to Tackle Security Threats of Intelligent Network Services
- [34] Anomaly Detection in Unsupervised Surveillance Setting Using Ensemble of Multimodal Data with Adversarial Defense
- [35] Automated Cyber Defence A Review
- [36] Autonomous Threat Hunting A Future Paradigm for AI-Driven Threat Intelligence
- [37] Intrinsic Geometric Vulnerability of High-Dimensional Artificial Intelligence
- [38] General stochastic separation theorems with optimal bounds
- [39] Modular Learning Component Attacks Today's Reality, Tomorrow's Challenge
- [40] FeSHI Feature Map Based Stealthy Hardware Intrinsic Attack
- [41] SoWaF Shuffling of Weights and Feature Maps A Novel Hardware Intrinsic Attack (HIA) on Convolutional Neural Network (CNN)
- [42] Relational Analysis of Sensor Attacks on Cyber-Physical Systems
- [43] The failure tolerance of mechatronic software systems to random and targeted attacks
- [44] Interplay of intra- and inter-dependence affects the robustness of network of networks
- [45] Performance of attack strategies on modular networks
- [46] Beyond Boundaries A Comprehensive Survey of Transferable Attacks on AI Systems
- [47] On the Limitation of Local Intrinsic Dimensionality for Characterizing the Subspaces of Adversarial Examples
- [48] Local Intrinsic Dimensionality Signals Adversarial Perturbations
- [49] Fixed Points in Cyber Space Rethinking Optimal Evasion Attacks in the Age of AI-NIDS
- [50] Vulnerabilities of Connectionist AI Applications Evaluation and Defence
- [51] Trust in Software Supply Chains Blockchain-Enabled SBOM and the AIBOM Future
- [52] Quantitative Toolchain Assurance
- [53] BOMs Away! Inside the Minds of Stakeholders A Comprehensive Study of Bills of Materials for Software Systems
- [54] Challenges of Producing Software Bill Of Materials for Java
- [55] On the Way to SBOMs Investigating Design Issues and Solutions in Practice
- [56] Automating SBOM Generation with Zero-Shot Semantic Similarity
- [57] Intelligent Zero Trust Architecture for 5G 6G Networks Principles, Challenges, and the Role of Machine Learning in the context of O-RAN
- [58] Blockchain-based Zero Trust on the Edge

- [59] Fast Authentication and Progressive Authorization in Large-Scale IoT How to Leverage AI for Security Enhancement
- [60] ZTCloudGuard Zero Trust Context-Aware Access Management Framework to Avoid Misuse Cases in the Era of Generative AI and Cloud-based Health Information Ecosystem
- [61] Coordinated Disclosure for AI Beyond Security Vulnerabilities
- [62] The European AI Liability Directives – Critique of a Half-Hearted Approach and Lessons for the Future
- [63] BlendSM-DDM BLockchain-ENabled Secure Microservices for Decentralized Data Marketplaces
- [64] Blockchain-based Security Framework for Critical Industry 4.0 Cyber-physical System
- [65] How Platform-User Power Relations Shape Algorithmic Accountability A Case Study of Instant Loan Platforms and Financially Stressed Users in India
- [66] A Review of Attacks Against Language-Based Package Managers
- [67] Small World with High Risks A Study of Security Threats in the npm Ecosystem
- [68] SpellBound Defending Against Package Typosquatting
- [69] An Empirical Study of Malicious Code In PyPI Ecosystem
- [70] On the Impact of Security Vulnerabilities in the npm and RubyGems Dependency Networks
- [71] Mitigating Persistence of Open-Source Vulnerabilities in Maven Ecosystem
- [72] Malicious Package Detection using Metadata Information
- [73] Supporting the Detection of Software Supply Chain Attacks through Unsupervised Signature Generation
- [74] Label-Smoothed Backdoor Attack
- [75] A Backdoor Approach with Inverted Labels Using Dirty Label-Flipping Attacks
- [76] DFB A Data-Free, Low-Budget, and High-Efficacy Clean-Label Backdoor Attack
- [77] Narcissus A Practical Clean-Label Backdoor Attack with Limited Information
- [78] Poisoning MorphNet for Clean-Label Backdoor Attack to Point Clouds
- [79] Backdoor Attacks on Self-Supervised Learning
- [80] Defending Against Patch-based Backdoor Attacks on Self-Supervised Learning
- [81] How to Craft Backdoors with Unlabeled Data Alone
- [82] Defending Against Weight-Poisoning Backdoor Attacks for Parameter-Efficient Fine-Tuning
- [83] Instructions as Backdoors Backdoor Vulnerabilities of Instruction Tuning for Large Language Models
- [84] BadEdit Backdooring large language models by model editing
- [85] Broadly Applicable Targeted Data Sample Omission Attacks

- [86] Amplifying Membership Exposure via Data Poisoning
- [87] Poisoned classifiers are not only backdoored, they are fundamentally broken
- [88] Reliable Evaluation of Adversarial Transferability
- [89] Your Attack Is Too DUMB Formalizing Attacker Scenarios for Adversarial Transferability
- [90] Adversarial Malware Binaries Evading Deep Learning for Malware Detection in Executables
- [91] Deceiving End-to-End Deep Learning Malware Detectors using Adversarial Examples
- [92] Creating Valid Adversarial Examples of Malware
- [93] MDEA Malware Detection with Evolutionary Adversarial Learning
- [94] Instance-Level Trojan Attacks on Visual Question Answering via Adversarial Learning in Neuron Activation Space
- [95] Mutual-modality Adversarial Attack with Semantic Perturbation
- [96] Word-level Textual Adversarial Attacking as Combinatorial Optimization
- [97] BeamAttack Generating High-quality Textual Adversarial Examples through Beam Search and Mixed Semantic Spaces
- [98] Multi-granular Adversarial Attacks against Black-box Neural Ranking Models
- [99] Mitigating Evasion Attacks to Deep Neural Networks via Region-based Classification
- [100] When Not to Classify Anomaly Detection of Attacks (ADA) on DNN Classifiers at Test Time
- [101] Adversarial Sample Detection for Deep Neural Network through Model Mutation Testing
- [102] Evading Defenses to Transferable Adversarial Examples by Translation-Invariant Attacks
- [103] Fuzziness-tuned Improving the Transferability of Adversarial Examples
- [104] Security Weaknesses of Copilot Generated Code in GitHub
- [105] Assessing the Security of GitHub Copilot Generated Code – A Targeted Replication Study
- [106] Just another copy and paste Comparing the security vulnerabilities of ChatGPT generated code and StackOverflow answers
- [107] You Autocomplete Me Poisoning Vulnerabilities in Neural Code Completion
- [108] InjecAgent Benchmarking Indirect Prompt Injections in Tool-Integrated Large Language Model Agents
- [109] Benchmarking and Defending Against Indirect Prompt Injection Attacks on Large Language Models
- [110] Assessing Prompt Injection Risks in 200+ Custom GPTs
- [111] On the Exploitability of Instruction Tuning
- [112] Learning to Poison Large Language Models During Instruction Tuning
- [113] Adversarial Exploitation of Policy Imitation
- [114] Teacher Model Fingerprinting Attacks Against Transfer Learning

- [115] DAWN Dynamic Adversarial Watermarking of Neural Networks
- [116] Passport-aware Normalization for Deep Model Protection
- [117] Unstoppable Attack Label-Only Model Inversion via Conditional Diffusion Model
- [118] Knowledge-Enriched Distributional Model Inversion Attacks
- [119] Scale-MIA A Scalable Model Inversion Attack against Secure Federated Learning via Latent Space Reconstruction
- [120] Boosting Model Inversion Attacks with Adversarial Examples
- [121] Understanding Membership Inferences on Well-Generalized Learning Models
- [122] Membership Inference Attack Using Self Influence Functions
- [123] Membership Inference Attacks against Language Models via Neighbourhood Comparison
- [124] Membership Leakage in Label-Only Exposures
- [125] GAN-Leaks A Taxonomy of Membership Inference Attacks against Generative Models
- [126] Privacy Risks of Securing Machine Learning Models against Adversarial Examples
- [127] Stolen Memories Leveraging Model Memorization for Calibrated White-Box Membership Inference
- [128] ML-Doctor Holistic Risk Assessment of Inference Attacks Against Machine Learning Models
- [129] Backdoor Attacks Against Deep Learning Systems in the Physical World
- [130] Synthesizing Physical Backdoor Datasets An Automated Framework Leveraging Deep Generative Models
- [131] Towards Practical Deployment-Stage Backdoor Attack on Deep Neural Networks
- [132] Trojan Taxonomy in Quantum Computing
- [133] Exploration of Quantum Computer Power Side-Channels
- [134] Quantum Leak Timing Side-Channel Attacks on Cloud-Based Quantum Services
- [135] Classification of Quantum Computer Fault Injection Attacks
- [136] REPQC Reverse Engineering and Backdooring Hardware Accelerators for Post-quantum Cryptography
- [137] Cybersecurity for Quantum Computing
- [138] Dual-Key Multimodal Backdoors for Visual Question Answering
- [139] Chain-of-Thought Prompting of Large Language Models for Discovering and Fixing Software Vulnerabilities
- [140] A Comprehensive Study of the Capabilities of Large Language Models for Vulnerability Detection
- [141] Large Language Model-Powered Smart Contract Vulnerability Detection New Perspectives
- [142] Multi-role Consensus through LLMs Discussions for Vulnerability Detection

- [143] Enhancing Chain-of-Thoughts Prompting with Iterative Bootstrapping in Large Language Models
- [144] Deductive Verification of Chain-of-Thought Reasoning
- [145] BadChain Backdoor Chain-of-Thought Prompting for Large Language Models
- [146] CyberSecEval 2 A Wide-Ranging Cybersecurity Evaluation Suite for Large Language Models
- [147] Learning to Execute Programs with Instruction Pointer Attention Graph Neural Networks
- [148] GraphSearchNet Enhancing GNNs via Capturing Global Dependencies for Semantic Code Search
- [149] Detecting Code Clones with Graph Neural Network and Flow-Augmented Abstract Syntax Tree
- [150] Implant Global and Local Hierarchy Information to Sequence based Code Representation Models
- [151] HELoC Hierarchical Contrastive Learning of Source Code Representation
- [152] GN-Transformer Fusing Sequence and Graph Representation for Improved Code Summarization
- [153] Source Code Vulnerability Detection Combining Code Language Models and Code Property Graphs
- [154] Mamba Linear-Time Sequence Modeling with Selective State Spaces
- [155] DeFuzz Deep Learning Guided Directed Fuzzing
- [156] Better Pay Attention Whilst Fuzzing
- [157] CovRL Fuzzing JavaScript Engines with Coverage-Guided Reinforcement Learning for LLM-based Mutation
- [158] MTFuzz Fuzzing with a Multi-Task Neural Network
- [159] Using Multiple Code Representations to Prioritize Static Analysis Warnings
- [160] Detecting Security-Relevant Methods using Multi-label Machine Learning
- [161] ConFL Constraint-guided Fuzzing for Machine Learning Framework
- [162] Graph-Based Fuzz Testing for Deep Learning Inference Engine
- [163] The Vulnerability Is in the Details Locating Fine-grained Information of Vulnerable Code Identified by Graph-based Detectors
- [164] A systematic review of fuzzing based on machine learning techniques
- [165] LineVD Statement-level Vulnerability Detection using Graph Neural Networks
- [166] VulSPG Vulnerability detection based on slice property graph representation learning
- [167] VELVET a noVel Ensemble Learning approach to automatically locate Vulnerable Statements
- [168] Learning to Quantize Vulnerability Patterns and Match to Locate Statement-Level Vulnerabilities

- [169] ActiveClean Generating Line-Level Vulnerability Data via Active Learning
- [170] Graph Neural Networks for Vulnerability Detection A Counterfactual Explanation
- [171] Counterfactual Explanations via Locally-guided Sequential Algorithmic Recourse
- [172] Beyond Fidelity Explaining Vulnerability Localization of Learning-based Detectors
- [173] The Effect of Data Poisoning on Counterfactual Explanations
- [174] Distinguishing Look-Alike Innocent and Vulnerable Code by Subtle Semantic Representation Learning and Explanation
- [175] Do Language Models Learn Semantics of Code A Case Study in Vulnerability Detection
- [176] On the Use of Fine-grained Vulnerable Code Statements for Software Vulnerability Assessment Models
- [177] Smart Learning to Find Dumb Contracts (Extended Version)
- [178] Eth2Vec Learning Contract-Wide Code Representations for Vulnerability Detection on Ethereum Smart Contracts
- [179] G-Scan Graph Neural Networks for Line-Level Vulnerability Identification in Smart Contracts
- [180] MANDO Multi-Level Heterogeneous Graph Embeddings for Fine-Grained Detection of Smart Contract Vulnerabilities
- [181] WANA Symbolic Execution of Wasm Bytecode for Cross-Platform Smart Contract Vulnerability Detection
- [182] Firmware Re-hosting Through Static Binary-level Porting
- [183] Software Vulnerability Prediction Knowledge Transferring Between Programming Languages
- [184] MPro Combining Static and Symbolic Analysis for Scalable Testing of Smart Contract
- [185] ConFuzzius A Data Dependency-Aware Hybrid Fuzzer for Smart Contracts
- [186] Benchmark of Data Preprocessing Methods for Imbalanced Classification
- [187] The choice of scaling technique matters for classification performance
- [188] DiverseVul A New Vulnerable Source Code Dataset for Deep Learning Based Vulnerability Detection
- [189] Data Contamination Through the Lens of Time
- [190] Vulnerability Detection with Code Language Models How Far Are We
- [191] Appropriateness of Performance Indices for Imbalanced Data Classification An Analysis
- [192] Classification Performance Metric for Imbalance Data Based on Recall and Selectivity Normalized in Class Labels
- [193] Interpretable Meta-Measure for Model Performance
- [194] Identifying the Context Shift between Test Benchmarks and Production Data
- [195] Change is Hard A Closer Look at Subpopulation Shift

- [196] Does Progress On Object Recognition Benchmarks Improve Real-World Generalization
- [197] Generalizability of Machine Learning Models Quantitative Evaluation of Three Methodological Pitfalls
- [198] Can You Rely on Your Model Evaluation Improving Model Evaluation with Synthetic Test Data
- [199] A Landscape Study of Open Source and Proprietary Tools for Software Bill of Materials (SBOM)
- [200] Evaluation of Software Product Quality Metrics
- [201] An Empirical Study on Software Bill of Materials Where We Stand and the Road Ahead
- [202] The Hitchhiker’s Guide to Malicious Third-Party Dependencies
- [203] OSS Malicious Package Analysis in the Wild
- [204] Unveiling A Hidden Risk Exposing Educational but Malicious Repositories in GitHub
- [205] Early Malware Detection and Next-Action Prediction
- [206] DONAPI Malicious NPM Packages Detector using Behavior Sequence Knowledge Mapping
- [207] Andro-profiler Detecting and Classifying Android Malware based on Behavioral Profiles
- [208] Portable, Data-Driven Malware Detection using Language Processing and Machine Learning Techniques on Behavioral Analysis Reports
- [209] Shifting the Lens Detecting Malware in npm Ecosystem with Large Language Models
- [210] Prompt Engineering-assisted Malware Dynamic Analysis Using GPT-4
- [211] Detecting Social Media Manipulation in Low-Resource Languages
- [212] Does DetectGPT Fully Utilize Perturbation Bridge Selective Perturbation to Fine-tuned Contrastive Learning Detector would be Better
- [213] Demystifying the Vulnerability Propagation and Its Evolution via Dependency Trees in the NPM Ecosystem
- [214] Characterizing Dependency Update Practice of NPM, PyPI and Cargo Packages
- [215] Cargo Ecosystem Dependency-Vulnerability Knowledge Graph Construction and Vulnerability Propagation Study
- [216] Dependency Practices for Vulnerability Mitigation
- [217] Data Considerations in Graph Representation Learning for Supply Chain Networks
- [218] HKTGNN Hierarchical Knowledge Transferable Graph Neural Network-based Supply Chain Risk Assessment
- [219] SSTKG Simple Spatio-Temporal Knowledge Graph for Interpretable and Versatile Dynamic Information Embedding
- [220] Efficient Attack Graph Analysis through Approximate Inference
- [221] Containing Malicious Package Updates in npm with a Lightweight Permission System

- [222] JITScanner Just-in-Time Executable Page Check in the Linux Operating System
- [223] Manticore A User-Friendly Symbolic Execution Framework for Binaries and Smart Contracts
- [224] PARIOT Anti-Repackaging for IoT Firmware Integrity
- [225] Securing Access to Untrusted Services From TEEs with GateKeeper
- [226] On the Security Blind Spots of Software Composition Analysis
- [227] On the Threat of npm Vulnerable Dependencies in Node.js Applications
- [228] Vulnerable Open Source Dependencies Counting Those That Matter
- [229] Do Software Security Practices Yield Fewer Vulnerabilities
- [230] The Impact of a Major Security Event on an Open Source Project The Case of OpenSSL
- [231] Lags in the Release, Adoption, and Propagation of npm Vulnerability Fixes
- [232] A Case Study on Software Vulnerability Coordination
- [233] SPI Automated Identification of Security Patches via Commits
- [234] A Practical Approach to the Automatic Classification of Security-Relevant Commits
- [235] Improving adversarial robustness of deep neural networks by using semantic information
- [236] Characterizing Adversarial Subspaces Using Local Intrinsic Dimensionality
- [237] Improved Adversarial Training Through Adaptive Instance-wise Loss Smoothing
- [238] Layer-wise Regularized Adversarial Training using Layers Sustainability Analysis (LSA) framework
- [239] AccelAT A Framework for Accelerating the Adversarial Training of Deep Neural Networks through Accuracy Gradient
- [240] How Robust Are Energy-Based Models Trained With Equilibrium Propagation
- [241] Stochastic Security Adversarial Defense Using Long-Run Dynamics of Energy-Based Models
- [242] Generating Unrestricted Adversarial Examples via Three Parameters
- [243] A Spectral Perspective towards Understanding and Improving Adversarial Robustness
- [244] SparseFed Mitigating Model Poisoning Attacks in Federated Learning with Sparsification
- [245] RS-DGC Exploring Neighborhood Statistics for Dynamic Gradient Compression on Remote Sensing Image Interpretation
- [246] Fed-LSAE Thwarting Poisoning Attacks against Federated Cyber Threat Detection System via Autoencoder-based Latent Space Inspection
- [247] Identifying Backdoor Attacks in Federated Learning via Anomaly Detection
- [248] STDLens Model Hijacking-Resilient Federated Learning for Object Detection
- [249] Avoid Adversarial Adaption in Federated Learning by Multi-Metric Investigations
- [250] Fedward Flexible Federated Backdoor Defense Framework with Non-IID Data

- [251] Security of Distributed Machine Learning A Game-Theoretic Approach to Design Secure DSVM
- [252] Mitigating Adversarial Attacks in Federated Learning with Trusted Execution Environments
- [253] Reaching Data Confidentiality and Model Accountability on the CalTrain
- [254] Defending against the Label-flipping Attack in Federated Learning
- [255] Voyager MTD-Based Aggregation Protocol for Mitigating Poisoning Attacks on DFL
- [256] Optimal Membership Inference Bounds for Adaptive Composition of Sampled Gaussian Mechanisms
- [257] Effects of Differential Privacy and Data Skewness on Membership Inference Vulnerability
- [258] MIAShield Defending Membership Inference Attacks via Preemptive Exclusion of Members
- [259] One Parameter Defense – Defending against Data Inference Attacks via Differential Privacy
- [260] EnclaveTree Privacy-preserving Data Stream Training and Inference Using TEE
- [261] No Privacy Left Outside On the (In-)Security of TEE-Shielded DNN Partition for On-Device ML
- [262] MirrorNet A TEE-Friendly Framework for Secure On-device DNN Inference
- [263] Cipherfix Mitigating Ciphertext Side-Channel Attacks in Software
- [264] Secure Aggregation is Not Private Against Membership Inference Attacks
- [265] Client-specific Property Inference against Secure Aggregation in Federated Learning
- [266] SAFE Learning Enable Backdoor Detectability In Federated Learning With Secure Aggregation
- [267] Flatee Federated Learning Across Trusted Execution Environments
- [268] Ensembler Combating model inversion attacks using model ensemble during collaborative inference
- [269] PATROL Privacy-Oriented Pruning for Collaborative Inference Against Model Inversion Attacks
- [270] Provably Minimally-Distorted Adversarial Examples
- [271] Semidefinite relaxations for certifying robustness to adversarial examples
- [272] On the tightness of linear relaxation based robustness certification methods
- [273] On the Paradox of Certified Training
- [274] Double Bubble, Toil and Trouble Enhancing Certified Robustness through Transitivity
- [275] Towards Bridging the gap between Empirical and Certified Robustness against Adversarial Examples
- [276] Is Certifying ℓ_p Robustness Still Worthwhile
- [277] GSmooth Certified Robustness against Semantic Transformations via Generalized Randomized Smoothing

- [278] TSS Transformation-Specific Smoothing for Robustness Certification
- [279] Provable Defense Against Geometric Transformations
- [280] Learning Security Classifiers with Verified Global Robustness Properties
- [281] Collective Certified Robustness against Graph Injection Attacks
- [282] Verifiable Boosted Tree Ensembles
- [283] Getting a-Round Guarantees Floating-Point Attacks on Certified Robustness
- [284] Computational Asymmetries in Robust Classification
- [285] RobustEdge Low Power Adversarial Detection for Cloud-Edge Systems
- [286] David and Goliath An Empirical Evaluation of Attacks and Defenses for QNNs at the Deep Edge
- [287] QUENN QUantization Engine for low-power Neural Networks
- [288] The Case for Hierarchical Deep Learning Inference at the Network Edge
- [289] EARLIN Early Out-of-Distribution Detection for Resource-efficient Collaborative Inference
- [290] Sustainable Edge Intelligence Through Energy-Aware Early Exiting
- [291] SplitPlace AI Augmented Splitting and Placement of Large-Scale Neural Networks in Mobile Edge Environments
- [292] Towards Energy-Efficient and Secure Edge AI A Cross-Layer Framework
- [293] RADAR Run-time Adversarial Weight Attack Detection and Accuracy Recovery
- [294] Order P4-66 Characterizing and mitigating surreptitious programmable network device exploitation
- [295] STPA-driven Multilevel Runtime Monitoring for In-time Hazard Detection
- [296] ICSSIM-A Framework for Building Industrial Control Systems Security Simulation Testbeds
- [297] Evaluation of vulnerability reproducibility in container-based Cyber Range
- [298] Federated TON_IoT Windows Datasets for Evaluating AI-based Security Applications
- [299] Scalable Cyber-Physical Testbed for Cybersecurity Evaluation of Synchrophasors in Power Systems
- [300] A Deep Learning-based Framework for Conducting Stealthy Attacks in Industrial Control Systems
- [301] Towards Situational Aware Cyber-Physical Systems A Security-Enhancing Use Case of Blockchain-based Digital Twins
- [302] EPICTWIN An Electric Power Digital Twin for Cyber Security Testing, Research and Education
- [303] SCART Simulation of Cyber Attacks for Real-Time
- [304] Assembling a Cyber Range to Evaluate Artificial Intelligence Machine Learning (AI ML) Security Tools

- [305] Assessment of Cyber-Physical Intrusion Detection and Classification for Industrial Control Systems
- [306] A Data-Centric Approach to Generate Invariants for a Smart Grid Using Machine Learning
- [307] Extracting Physical Causality from Measurements to Detect and Localize False Data Injection Attacks
- [308] On the Feasibility of Distinguishing Between Process Disturbances and Intrusions in Process Control Systems Using Multivariate Statistical Process Control
- [309] Multi-Source Data Fusion for Cyberattack Detection in Power Systems
- [310] Cyber-Attack Detection in Socio-Technical Transportation Systems Exploiting Redundancies Between Physical and Social Data
- [311] Unsupervised Spatio-Temporal State Estimation for Fine-grained Adaptive Anomaly Diagnosis of Industrial Cyber-physical Systems
- [312] Self-Supervised and Interpretable Anomaly Detection using Network Transformers
- [313] Machine Learning Assisted Bad Data Detection for High-throughput Substation Communication
- [314] Compact Data Structures for Network Telemetry
- [315] Digital Twin-based Intrusion Detection for Industrial Control Systems
- [316] Digital Twin-based Anomaly Detection with Curriculum Learning in Cyber-physical Systems
- [317] A Bayesian Framework for Digital Twin-Based Control, Monitoring, and Data Collection in Wireless Systems
- [318] Knowledge Distillation-Empowered Digital Twin for Anomaly Detection
- [319] A Cloud-Fog Computing Architecture for Real-Time Digital Twins
- [320] Digital Twins and the Future of their Use Enabling Shift Left and Shift Right Cybersecurity Operations
- [321] Resilient Output Containment Control of Heterogeneous Multiagent Systems Against Composite Attacks A Digital Twin Approach
- [322] Digital Twins for Supporting AI Research with Autonomous Vehicle Networks
- [323] Causal Falsification of Digital Twins
- [324] A White-Box Adversarial Attack Against a Digital Twin
- [325] Explainable Human-in-the-loop Dynamic Data-Driven Digital Twins
- [326] Integration of Digital Twin and Federated Learning for Securing Vehicular Internet of Things
- [327] Constrained Twin Variational Auto-Encoder for Intrusion Detection in IoT Systems
- [328] SuperNeurons Dynamic GPU Memory Management for Training Deep Neural Networks
- [329] TinyAD Memory-efficient anomaly detection for time series data in Industrial IoT
- [330] Evolutionary Bin Packing for Memory-Efficient Dataflow Inference Acceleration on FPGA

- [331] Stacked Auto Encoder Based Deep Reinforcement Learning for Online Resource Scheduling in Large-Scale MEC Networks
- [332] HeteroEdge Addressing Asymmetry in Heterogeneous Collaborative Autonomous Systems
- [333] FrankenSplit Efficient Neural Feature Compression with Shallow Variational Bottleneck Injection for Mobile Edge Computing
- [334] LC-NAS Latency Constrained Neural Architecture Search for Point Cloud Networks
- [335] Benchmarking adversarial attacks and defenses for time-series data
- [336] Robust Multivariate Time-Series Forecasting Adversarial Attacks and Defense Mechanisms
- [337] Are Adversarial Examples Created Equal A Learnable Weighted Minimax Risk for Robustness under Non-uniform Attacks
- [338] Wasserstein Adversarial Examples on Univariate Time Series Data
- [339] Confidence-Calibrated Adversarial Training Generalizing to Unseen Attacks
- [340] Adversarial Visual Robustness by Causal Intervention
- [341] Efficient Adversarial Training with Transferable Adversarial Examples
- [342] Fast Adversarial Training with Adaptive Step Size
- [343] Learn from the Past A Proxy Guided Adversarial Defense Framework with Self Distillation Regularization
- [344] Secure Instruction and Data-Level Information Flow Tracking Model for RISC-V
- [345] PAGURUS Low-Overhead Dynamic Information Flow Tracking on Loosely Coupled Accelerators
- [346] Information Flow Tracking Methods for Protecting Cyber-Physical Systems against Hardware Trojans – a Survey
- [347] A Survey Report on Hardware Trojan Detection by Multiple-Parameter Side-Channel Analysis
- [348] Evasive Hardware Trojan through Adversarial Power Trace
- [349] A Pragmatic Methodology for Blind Hardware Trojan Insertion in Finalized Layouts
- [350] Programmable EM Sensor Array for Golden-Model Free Run-time Trojan Detection and Localization
- [351] Hardware Software Co-monitoring
- [352] ACFA Secure Runtime Auditing & Guaranteed Device Healing via Active Control Flow Attestation
- [353] TrojanZero Switching Activity-Aware Design of Undetectable Hardware Trojans with Zero Power and Area Footprint
- [354] Hardware Trojan with Frequency Modulation
- [355] Side-Channel Trojan Insertion – a Practical Foundry-Side Attack via ECO
- [356] TRUST XAI Model-Agnostic Explanations for AI With a Case Study on IIoT Security

- [357] Explainable AI, but explainable to whom
- [358] HuntGPT Integrating Machine Learning-Based Anomaly Detection and Explainable AI with Large Language Models (LLMs)
- [359] Painting the black box white experimental findings from applying XAI to an ECG reading setting
- [360] What Do You See Evaluation of Explainable Artificial Intelligence (XAI) Interpretability through Neural Backdoors
- [361] Characterizing Human Explanation Strategies to Inform the Design of Explainable AI for Building Damage Assessment
- [362] Explainable Intrusion Detection Systems (X-IDS) A Survey of Current Methods, Challenges, and Opportunities
- [363] A blockchain-orchestrated Federated Learning architecture for healthcare consortia
- [364] HBFL A Hierarchical Blockchain-based Federated Learning Framework for a Collaborative IoT Intrusion Detection
- [365] Defending Against Poisoning Attacks in Federated Learning with Blockchain
- [366] APPFLChain A Privacy Protection Distributed Artificial-Intelligence Architecture Based on Federated Learning and Consortium Blockchain
- [367] Post Quantum Secure Blockchain-based Federated Learning for Mobile Edge Computing
- [368] Secure and Efficient Federated Learning Through Layering and Sharding Blockchain
- [369] Advancing Federated Learning in 6G A Trusted Architecture with Graph-based Analysis
- [370] Demystifying Swarm Learning A New Paradigm of Blockchain-based Decentralized Federated Learning
- [371] FederatedTrust A Solution for Trustworthy Federated Learning
- [372] AUDITEM Toward an Automated and Efficient Data Integrity Verification Model Using Blockchain
- [373] SmartQC An Extensible DLT-Based Framework for Trusted Data Workflows in Smart Manufacturing
- [374] Orchestrating Product Provenance Story When IOTA ECOSYSTEM Meets The Electronics Supply Chain Space
- [375] Securing OPEN-RAN Equipment Using Blockchain-Based Supply Chain Verification
- [376] Blockchain platform for COVID-19 vaccine supply management
- [377] Private Data Objects an Overview
- [378] A blockchain-based pattern for confidential and pseudo-anonymous contract enforcement
- [379] Fed-DDM A Federated Ledgers based Framework for Hierarchical Decentralized Data Marketplaces
- [380] Spatial Positioning Token (SPToken) for Smart Mobility

- [381] SmartMark Software Watermarking Scheme for Smart Contracts
- [382] IA-CCF Individual Accountability for Permissioned Ledgers
- [383] Intractable Group-theoretic Problems Around Zero-knowledge Proofs
- [384] zk-IoT Securing the Internet of Things with Zero-Knowledge Proofs on Blockchain Platforms
- [385] Trustless Audits without Revealing Data or Models
- [386] Trust the Process Zero-Knowledge Machine Learning to Enhance Trust in Generative AI Interactions
- [387] ZKclaims Privacy-preserving Attribute-based Credentials using Non-interactive Zero-knowledge Techniques
- [388] zkFaith Soonami's Zero-Knowledge Identity Protocol
- [389] Enhancing Ethereum's Security with LUMEN, a Novel Zero-Knowledge Protocol Generating Transparent and Efficient zk-SNARKs
- [390] zksk A Library for Composable Zero-Knowledge Proofs
- [391] Multi-Party Proof Generation in QAP-based zk-SNARKs
- [392] Trustworthy Pre-Processing of Sensor Data in Data On-chaining Workflows for Blockchain-based IoT Applications
- [393] zkTax A pragmatic way to support zero-knowledge tax disclosures
- [394] Zero-Knowledge Proof-based Practical Federated Learning on Blockchain
- [395] Augmenting Zero Trust Architecture to Endpoints Using Blockchain A State-of-The-Art Review
- [396] Application of Data Collected by Endpoint Detection and Response Systems for Implementation of a Network Security System based on Zero Trust Principles and the EigenTrust Algorithm
- [397] Scalable and Secure Architecture for Distributed IoT Systems
- [398] Zero Trust Implementation in the Emerging Technologies Era Survey
- [399] Decentralised Trustworthy Collaborative Intrusion Detection System for IoT
- [400] Zero Trust Real-Time Lightweight Access Control Protocol for Mobile Cloud-Based IoT Sensors
- [401] A Trust Architecture for Blockchain in IoT
- [402] Zero Trust Applications, Challenges, and Opportunities
- [403] Blockchain Federated Learning for Threat Defense
- [404] Trust Management in Decentralized IoT Access Control System
- [405] MeritRank Sybil Tolerant Reputation for Merit-based Tokenomics
- [406] Quantifying the importance of firms by means of reputation and network control
- [407] Trust as a Metric for Resiliency in Signed Social Networks

- [408] To incentivize or not Impact of blockchain-based cryptoeconomic tokens on human information sharing behavior
- [409] How to Rationally Select Your Delegatee in PoS
- [410] Proof-of-Reputation An Alternative Consensus Mechanism for Blockchain Systems
- [411] RepChain A Reputation-based Secure, Fast and High Incentive Blockchain System via Sharding
- [412] Incentives Against Power Grabs or How to Engineer the Revolution in a Pooled Proof of Stake System
- [413] Enabling Opportunistic Users in Multi-Tenant IoT Systems using Decentralized Identifiers and Permissioned Blockchains
- [414] Blockchain based Resource Governance for Decentralized Web Environments
- [415] SoK Explainable Machine Learning for Computer Security Applications
- [416] Meaningful XAI Based on User-Centric Design Methodology
- [417] XRand Differentially Private Defense against Explanation-Guided Attacks
- [418] The privacy issue of counterfactual explanations explanation linkage attacks
- [419] The Disagreement Problem in Faithfulness Metrics
- [420] The future of human-centric eXplainable Artificial Intelligence (XAI) is not post-hoc explanations
- [421] Bridging the Transparency Gap What Can Explainable AI Learn From the AI Act
- [422] Seamful XAI Operationalizing Seamful Design in Explainable AI
- [423] Explainable Artificial Intelligence (XAI) 2.0 A Manifesto of Open Challenges and Interdisciplinary Research Directions
- [424] On the Impact of Explanations on Understanding of Algorithmic Decision-Making
- [425] Beyond Expertise and Roles A Framework to Characterize the Stakeholders of Interpretable Machine Learning and their Needs
- [426] A Methodology and Software Architecture to Support Explainability-by-Design
- [427] Unsupervised Multi-Granularity Summarization
- [428] Multi-Granular Text Encoding for Self-Explaining Categorization
- [429] Progressive Disclosure Designing for Effective Transparency
- [430] Counterfactual Explanations without Opening the Black Box Automated Decisions and the GDPR
- [431] Evaluating Robustness of Counterfactual Explanations
- [432] Disagreement amongst counterfactual explanations How transparency can be deceptive
- [433] Convex optimization for actionable & plausible counterfactual explanations
- [434] Towards Auditability Requirements Specification Using an Agent-Based Approach

- [435] Flexible and Context-Specific AI Explainability A Multidisciplinary Approach
- [436] Explainable Machine Learning in Deployment
- [437] Trinity A Distributed Publish Subscribe Broker with Blockchain-based Immutability
- [438] DeTRM Decentralised Trust and Reputation Management for Blockchain-based Supply Chains
- [439] A Layered Architecture Enabling Metaverse Applications in Smart Manufacturing Environments
- [440] NSSIA A New Self-Sovereign Identity Scheme with Accountability
- [441] MQT-TZ Hardening IoT Brokers Using ARM TrustZone
- [442] The Availability-Accountability Dilemma and its Resolution via Accountability Gadgets
- [443] Security for Machine Learning-based Software Systems a survey of threats, practices and challenges
- [444] Professional Insights into Benefits and Limitations of Implementing MLOps Principles
- [445] Exploring MLOps Dynamics An Experimental Analysis in a Real-World Machine Learning Project
- [446] Mind Your Weight(s) A Large-scale Study on Insufficient Machine Learning Model Protection in Mobile Apps
- [447] MLCapsule Guarded Offline Deployment of Machine Learning as a Service
- [448] MPP Model Performance Predictor
- [449] SafeML Safety Monitoring of Machine Learning Classifiers through Statistical Difference Measure
- [450] We Have No Idea How Models will Behave in Production until Production How Engineers Operationalize Machine Learning
- [451] Adversarial Machine Learning – Industry Perspectives
- [452] MLSMM Machine Learning Security Maturity Model
- [453] ModelCI-e Enabling Continual Learning in Deep Learning Serving Systems
- [454] Integrating MLSecOps in the Biotechnology Industry 5.0
- [455] Automated CVE Analysis for Threat Prioritization and Impact Prediction
- [456] Automated Mapping of CVE Vulnerability Records to MITRE CWE Weaknesses
- [457] ThreatZoom CVE2CWE using Hierarchical Neural Network
- [458] Unveiling Hidden Links Between Unseen Security Entities
- [459] CVE-driven Attack Technique Prediction with Semantic Information Extraction and a Domain-specific Language Model
- [460] Crimson Empowering Strategic Reasoning in Cybersecurity through Large Language Models

- [461] Deep VULMAN A Deep Reinforcement Learning-Enabled Cyber Vulnerability Management Framework
- [462] Using Sequence-to-Sequence Learning for Repairing C Vulnerabilities
- [463] Deep ahead-of-threat virtual patching
- [464] Vulnerability Prioritization An Offensive Security Approach
- [465] On managing vulnerabilities in AI ML systems
- [466] Multiparty Dynamics and Failure Modes for Machine Learning and Artificial Intelligence
- [467] The Reasoning Under Uncertainty Trap A Structural AI Risk
- [468] Empirical Optimal Risk to Quantify Model Trustworthiness for Failure Detection
- [469] PAGER A Framework for Failure Analysis of Deep Regression Models
- [470] DDoD Dual Denial of Decision Attacks on Human-AI Teams
- [471] Blockchain-assisted Undisclosed IIoT Vulnerabilities Trusted Sharing Protection with Dynamic Token
- [472] An Empirical Study of Challenges in Machine Learning Asset Management
- [473] Metadata Representations for Queryable ML Model Zoos
- [474] Developing and Deploying Industry Standards for Artificial Intelligence in Education (AIED) Challenges, Strategies, and Future Directions
- [475] The Artificial Intelligence Ontology LLM-assisted construction of AI concept hierarchies
- [476] Ethics-Based Auditing of Automated Decision-Making Systems Intervention Points and Policy Implications
- [477] Understanding accountability in algorithmic supply chains
- [478] Data Driven Approaches to Cybersecurity Governance for Board Decision-Making – A Systematic Review
- [479] Assessing Supply Chain Cyber Risks
- [480] Automated Compliance Blueprint Optimization with Artificial Intelligence
- [481] Optimal Cyber-Insurance Contract Design for Dynamic Risk Management and Mitigation
- [482] FlipIn A Game-Theoretic Cyber Insurance Framework for Incentive-Compatible Cyber Risk Management of Internet of Things
- [483] Pricing cyber-insurance for systems via maturity models
- [484] A Novel Mutual Insurance Model for Hedging Against Cyber Risks in Power Systems Deploying Smart Technologies
- [485] Accountability and Insurance in IoT Supply Chain
- [486] Post-Incident Audits on Cyber Insurance Discounts
- [487] Security Pricing as an Enabler of Cyber-Insurance A First Look at Differentiated Pricing Markets

- [488] TrustChain Trust Management in Blockchain and IoT supported Supply Chains
- [489] SciChain Trustworthy Scientific Data Provenance
- [490] ESASCF Expertise Extraction, Generalization and Reply Framework for an Optimized Automation of Network Security Compliance
- [491] SmartScan An approach to detect Denial of Service Vulnerability in Ethereum Smart Contracts
- [492] Evaluation of ChatGPT's Smart Contract Auditing Capabilities Based on Chain of Thought
- [493] AGAPECert An Auditable, Generalized, Automated, Privacy-Enabling Certification Framework with Oblivious Smart Contracts
- [494] Fine-grained Data Access Control for Collaborative Process Execution on Blockchain
- [495] Justifying the Dependability and Security of Business-Critical Blockchain-based Applications
- [496] Blockchain-based Digital Twins Research Trends, Issues, and Future Challenges
- [497] Blockchain and Machine Learning for Fraud Detection A Privacy-Preserving and Adaptive Incentive Based Approach
- [498] Algorithmic Fairness Datasets the Story so Far
- [499] Lazy Data Practices Harm Fairness Research
- [500] Collect, Measure, Repeat Reliability Factors for Responsible AI Data Collection
- [501] MEDFAIR Benchmarking Fairness for Medical Imaging
- [502] Understanding Unfairness in Fraud Detection through Model and Data Bias Interactions
- [503] Fairness without Demographics through Adversarially Reweighted Learning
- [504] A Three-Way Knot Privacy, Fairness, and Predictive Performance Dynamics
- [505] FairPrep Promoting Data to a First-Class Citizen in Studies on Fairness-Enhancing Interventions
- [506] Learning with Impartiality to Walk on the Pareto Frontier of Fairness, Privacy, and Utility
- [507] Making Machine Learning Datasets and Models FAIR for HPC A Methodology and Case Study
- [508] Data Collaboration Analysis Over Matrix Manifolds
- [509] The Promise and Peril of Artificial Intelligence – Violet Teaming Offers a Balanced Path Forward
- [510] Reflective Hybrid Intelligence for Meaningful Human Control in Decision-Support Systems
- [511] Compensating for Sensing Failures via Delegation in Human-AI Hybrid Systems
- [512] Enhancing autonomy transparency an option-centric rationale approach
- [513] Hiding Behind Machines When Blame Is Shifted to Artificial Agents
- [514] Meaningful human control actionable properties for AI system development

- [515] Reinforcement Learning Under Moral Uncertainty
- [516] Lifelong Testing of Smart Autonomous Systems by Shepherdng a Swarm of Watchdog Artificial Intelligence Agents
- [517] Dual Governance The intersection of centralized regulation and crowdsourced safety mechanisms for Generative AI
- [518] Bridging the Global Divide in AI Regulation A Proposal for a Contextual, Coherent, and Commensurable Framework
- [519] A Safe Harbor for AI Evaluation and Red Teaming
- [520] Is the U.S. Legal System Ready for AI's Challenges to Human Values
- [521] The risks of risk-based AI regulation taking liability seriously
- [522] A multilevel framework for AI governance
- [523] Particip-AI A Democratic Surveying Framework for Anticipating Future AI Use Cases, Harms and Benefits
- [524] An International Consortium for Evaluations of Societal-Scale Risks from Advanced AI
- [525] Identifying Research Challenges in Post Quantum Cryptography Migration and Cryptographic Agility
- [526] Quantum-Resistant Cryptography
- [527] Quantum Computing Methods for Supply Chain Management
- [528] Cyber-Physical Defense in the Quantum Era
- [529] FIPS Compliant Quantum Secure Communication using Quantum Permutation Pad
- [530] Quantum-Resistant Security for Software Updates on Low-power Networked Embedded Devices
- [531] Code-based Cryptography in IoT A HW SW Co-Design of HQC
- [532] An Energy-Efficient Configurable Lattice Cryptography Processor for the Quantum-Secure Internet of Things
- [533] A 333.9uW 0.158mm² Saber Learning with Rounding based Post-Quantum Crypto Accelerator
- [534] Algorithmic Security is Insufficient A Comprehensive Survey on Implementation Attacks Haunting Post-Quantum Security
- [535] Making existing software quantum safe a case study on IBM Db2
- [536] Quantum Advantage and Y2K Bug Comparison
- [537] Evaluation Framework for Quantum Security Risk Assessment A Comprehensive Study for Quantum-Safe Migration
- [538] Quantum-resistance in blockchain networks
- [539] Towards quantum enhanced adversarial robustness in machine learning

- [540] Experimental quantum adversarial learning with programmable superconducting qubits
- [541] Near-Term Quantum Computing Techniques Variational Quantum Algorithms, Error Mitigation, Circuit Compilation, Benchmarking and Classical Simulation
- [542] Cybersecurity in the Quantum Era Assessing the Impact of Quantum Computing on Infrastructure
- [543] Sustainable AI Regulation
- [544] Making AI Less Thirsty Uncovering and Addressing the Secret Water Footprint of AI Models
- [545] Sustainable AI Environmental Implications, Challenges and Opportunities
- [546] Let it RAIN for Social Good
- [547] Towards Sustainable Artificial Intelligence An Overview of Environmental Protection Uses and Issues
- [548] Artificial Intelligence Development Races in Heterogeneous Settings
- [549] Broadening the perspective for sustainable AI Comprehensive sustainability criteria and indicators for AI systems
- [550] SECure A Social and Environmental Certificate for AI Systems
- [551] GreenFPGA Evaluating FPGAs as Environmentally Sustainable Computing Solutions
- [552] A Systematic Review of Green AI
- [553] Sustainable Artificial Intelligence through Continual Learning
- [554] Towards Environmentally Equitable AI via Geographical Load Balancing
- [555] Image-based Deep Learning for Smart Digital Twins a Review
- [556] Adaptive Fault Detection exploiting Redundancy with Uncertainties in Space and Time
- [557] Hybrid Imitation Learning for Real-Time Service Restoration in Resilient Distribution Systems
- [558] An Intentional Forgetting-Driven Self-Healing Method For Deep Reinforcement Learning Systems
- [559] Disruption Detection for a Cognitive Digital Supply Chain Twin Using Hybrid Deep Learning
- [560] A Cyber Manufacturing IoT System for Adaptive Machine Learning Model Deployment by Interactive Causality Enabled Self-Labeling
- [561] CHESS A Framework for Evaluation of Self-adaptive Systems based on Chaos Engineering
- [562] Self-healing Dilemmas in Distributed Systems Fault Correction vs. Fault Tolerance
- [563] Towards Trusted and Intelligent Cyber-Physical Systems A Security-by-Design Approach
- [564] Blockchain-based Immutable Evidence and Decentralized Loss Adjustment for Autonomous Vehicle Accidents in Insurance
- [565] A Distributed ‘Black Box’ Audit Trail Design Specification for Connected and Automated Vehicle Data and Software Assurance
- [566] DECORAIT – DECentralized Opt-in out Registry for AI Training

- [567] Specification is Law Safe Creation and Upgrade of Ethereum Smart Contracts
- [568] Distributed Ledger for Provenance Tracking of Artificial Intelligence Assets
- [569] New Online Communities Graph Deep Learning on Anonymous Voting Networks to Identify Sybils in Polycentric Governance
- [570] Auto.gov Learning-based On-chain Governance for Decentralized Finance (DeFi)
- [571] A Comprehensive Study of Governance Issues in Decentralized Finance Applications
- [572] SoK A Stratified Approach to Blockchain Decentralization
- [573] Safe Artificial General Intelligence via Distributed Ledger Technology
- [574] Trustworthy Artificial Intelligence and Process Mining Challenges and Opportunities
- [575] The Different Faces of AI Ethics Across the World A Principle-Implementation Gap Analysis
- [576] Principles alone cannot guarantee ethical AI
- [577] POLARIS A framework to guide the development of Trustworthy AI systems
- [578] Power and Play Investigating License to Critique in Teams' AI Ethics Discussions
- [579] From the Ground Truth Up Doing AI Ethics from Practice to Principles
- [580] Enhanced well-being assessment as basis for the practical implementation of ethical and rights-based normative principles for AI
- [581] IEEE 7010 A New Standard for Assessing the Well-being Implications of Artificial Intelligence
- [582] AI Sustainability in Practice Part One Foundations for Sustainable AI Projects
- [583] German AI Start-Ups and AI Ethics Using A Social Practice Lens for Assessing and Implementing Socio-Technical Innovation
- [584] An Ecosystem Approach to Ethical AI and Data Use Experimental Reflections
- [585] Actionable Principles for Artificial Intelligence Policy Three Pathways
- [586] Ethical Assurance A practical approach to the responsible design, development, and deployment of data-driven technologies
- [587] Ethics in conversation Building an ethics assurance case for autonomous AI-enabled voice agents in healthcare
- [588] Beyond Fairness Metrics Roadblocks and Challenges for Ethical AI in Practice